

可信机器学习 课程笔记 (Lecture 01–10)

AutoPku · 整理

2026-05-29

Contents

课程概览	4
课程脉络	5
各讲速览	7
全局术语中英对照	8
期末复习路线图	10
Lecture 01 —可信机器学习课程概览 (Course Overview)	11
0. 核心问题 / Motivation	11
1. 课程组织	11
2. 从 AI 到可信 ML	12
3. 为什么需要”可信”?	12
4. 六大模块预览	13
5. 学习路径建议	14
6. 概念关系图	14
7. 关键术语速查 (中英对照)	14
8. 各维度之间的内在联系	15
9. 课程外的阅读建议	16
9.5 课程方法论: 从”为什么”到”怎么改”	16
9.6 易混淆术语速辨	16
10. 期中复习要点	17
10. 期中复习要点	17
Lecture 02 —机器学习基础 (Introduction of Machine Learning)	18
0. 核心问题 / Motivation	18
1. 函数类型	18
2. ML 训练三步范式	19
3. 从线性到深度	20
4. 评估与过拟合	21
5. 分类的特殊处理	23
6. 概念关系图	23
7. 记号速查	23
8. 期中复习要点	24
Lecture 03 —卷积神经网络 (Convolutional Neural Network, CNN)	25
0. 核心问题 / Motivation	25
1. Observation 1: 局部特征 → Receptive Field	25
2. Observation 2: 模式平移不变 → Parameter Sharing	26
3. 两种等价视角	27
4. 卷积运算的具体计算	27
5. Observation 3: 子采样 → Pooling	27
6. 完整 CNN Pipeline	28

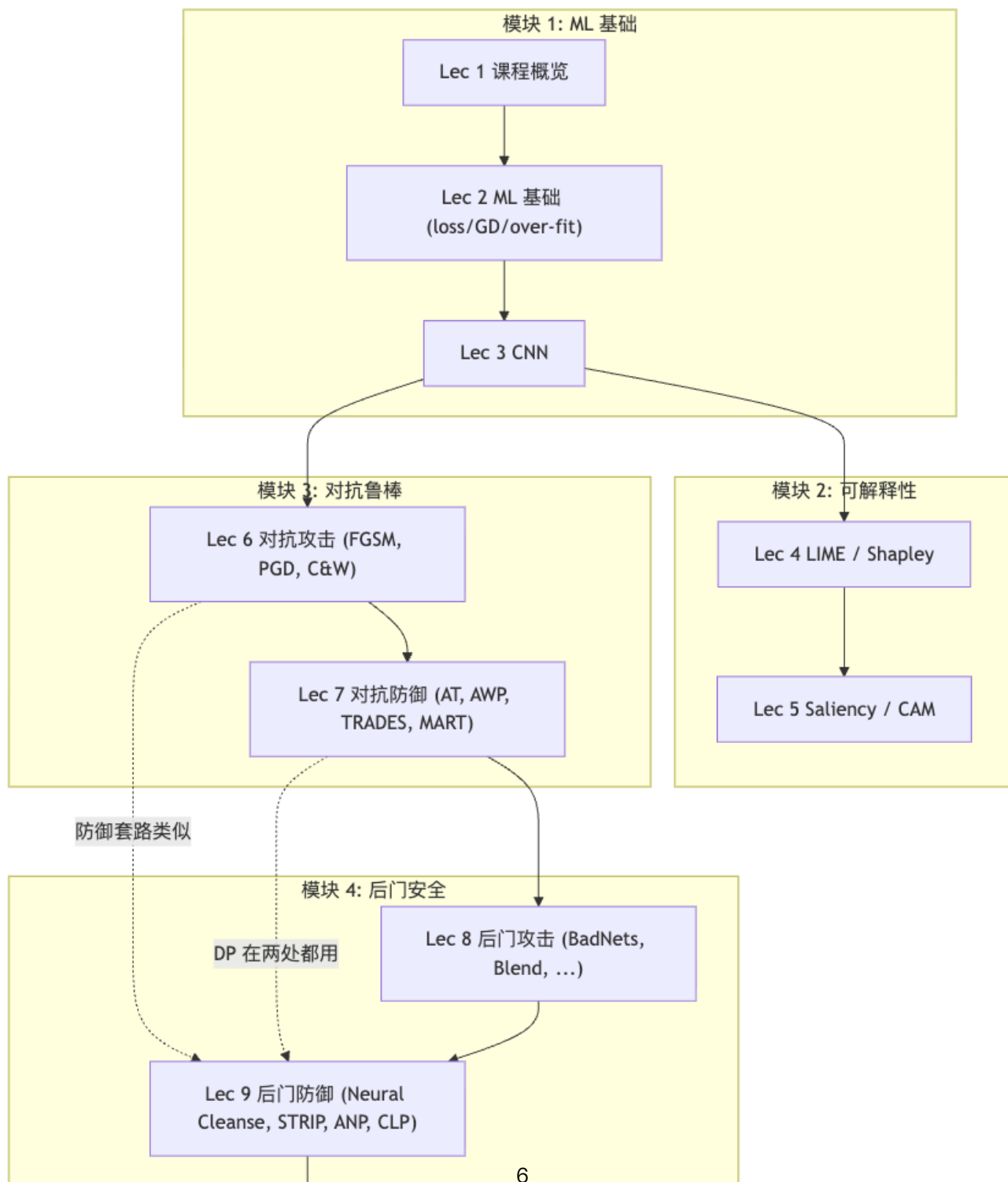
7. 三大设计原则的统一总结	28
8. 一个完整例题	29
9. 概念关系图	30
10. 记号速查	30
10.5 训练 CNN 时的常见技巧	31
11. 经典 CNN 架构纵览	31
12. 期中复习要点	32
Lecture 04 —可解释机器学习 (Explanations: LIME & Shapley)	33
0. 核心问题 / Motivation	33
1. 解释方法的分类	33
2. LIME (Local Interpretable Model-Agnostic Explanations)	34
3. 从博弈论引入: Shapley Value	35
4. 一个完整例题	37
5. LIME vs Shapley 对比	37
6. 概念关系图	38
7. 记号速查	38
8. 期中复习要点	39
Lecture 05 —Saliency Map 与 Class Activation Map (CAM)	40
0. 核心问题 / Motivation	40
1. Saliency Map: 梯度即解释	40
2. CAM: Class Activation Map	41
3. Saliency vs CAM 对比	43
4. 一个完整例题	43
5. 概念关系图	44
5.5 其他 Saliency 变种 (拓展)	44
5.6 Sanity Check: 解释方法本身要被怀疑	45
6. 记号速查	45
7. 期中复习要点	45
Lecture 06 —对抗攻击 (Adversarial Attack)	46
0. 核心问题 / Motivation	46
1. 白盒攻击 (White-box)	47
2. 黑盒攻击 (Black-box)	48
3. 物理对抗攻击 (Physical AE)	50
4. 关键攻击算法对照表	50
5. 一个完整例题	51
6. 概念关系图	51
7. 记号速查	51
8. 期中复习要点	52
Lecture 07 —对抗防御 (Adversarial Defense)	53
0. 核心问题 / Motivation	53
1. 对抗训练 (Adversarial Training) 基础	53
2. FOSC: 度量“内 Max 是否解好了”	54
3. AWP: 从权重 Loss Landscape 视角	55
4. Accuracy-Robustness Trade-off 与 TRADES	56
5. MART: Misclassified Examples 视角	57
6. 三种改进对照	57
7. 一个完整例题	58
8. 概念关系图	58
9. 记号速查	58
10. 期中复习要点	59
Lecture 08 —后门攻击 (Backdoor Attack)	60
0. 核心问题 / Motivation	60

0.x 名词预热	60
1. 后门攻击全景	61
2. BadNets (Gu et al., 2017)	62
3. Blend Attack (Chen et al., 2017)	63
4. Label-Consistent Attack (Turner et al., 2018)	63
5. Hidden Trigger Backdoor Attack (Saha et al., 2020)	64
6. Trojan Attack (Liu et al., 2018)	65
7. 攻击对比表	66
8. 防御预告	67
9. Anki Flashcards	67
10. 记号速查表	68
Lecture 09 —后门防御 (Backdoor Defense)	69
0. 核心问题 / Motivation	69
0.x 名词预热 (零前置)	70
1. 防御方法全景图	71
2. 输入预处理: Neural Trojans (Liu et al. 2017)	71
3. 触发器重建: Neural Cleanse (Wang et al. 2019)	72
4. 样本过滤: STRIP (Gao et al. 2019)	73
5. 投毒压制: 差分隐私训练 (DP-SGD)	74
6. 模型重建: Fine-Pruning (Liu et al. 2018)	75
7. 神经元扰动 / ANP (Wu & Wang 2021)	75
8. 通道 Lipschitz 剪枝 / CLP —首个 Data-Free 防御 (Zheng et al. 2022)	77
9. 防御方法对比表	78
10. 期末复习要点	79
11. Anki Flashcards	80
12. 记号速查表	81
Lecture 10 —隐私 (Privacy)	82
0. 核心问题 / Motivation	82
0.x 名词预热 (Glossary)	83
1. 隐私基础	83
2. 攻击全景图	84
3. 隐私泄漏的根因 (Causes of Leakage)	85
4. 成员推理攻击 (Membership Inference Attack, MIA)	85
5. 影子训练数据从何而来 (slide p.19)	86
6. 特征推理 / 重建攻击 (Feature / Reconstruction Inference)	86
7. 案例研究: GPT-2 训练数据提取 (Carlini et al., USENIX Security 2021)	87
8. 防御全景图	90
9. 同态加密 (Homomorphic Encryption, HE)	90
10. 安全多方计算 (Secure Multi-Party Computation, SMPC)	91
11. 差分隐私 (Differential Privacy, DP)	92
12. 联邦学习 (Federated Learning, FL)	93
13. 攻击 vs. 防御 对应表	94
14. 期末复习要点	95
15. Anki Flashcards	95
16. 记号速查表	96

课程概览

本课程 **可信机器学习 (Trustworthy Machine Learning)** 由王奕森 (Yisen Wang) 老师在北大信智学院开设 (25–26 学年第 2 学期)。在经典 ML 主要关注 **准确率 / 效率** 的基础上，本课系统介绍如何让模型同时满足 **可解释性、鲁棒性、隐私、公平、安全 (后门)** 等多维“可信”属性。前 10 讲覆盖 **基础 → 可解释 → 对抗鲁棒 → 后门安全 → 隐私** 五大模块，构成“可信 ML”完整知识地图。

课程脉络



各讲速览

Lec	主题	关键概念	文件
01	课程概览 / 可信 ML 全景	六大模块, Mitchell 定义, 失败案例	lec01_overview.md
02	机器学习基础	function/loss/GD 三步, sigmoid 复合, softmax+CE, 过拟合, bias-complexity	lec02_ml_introduction.md
03	卷积神经网络	Receptive Field, Parameter Sharing, Pooling, 输出尺寸公式	lec03_cnn.md
04	可解释 ML (Model-Agnostic)	LIME (局部线性代理), Shapley (4 公理 + 公式)	lec04_explanations_lime_shapley.md
05	可解释 ML (Model-Specific)	Saliency Map, SmoothGrad, CAM, Pokémon vs Digimon 案例	lec05_saliency_cam.md
06	对抗攻击	min-max, FGSM, BIM, PGD, C&W, 黑盒 Transfer/Query, EOT	lec06_adversarial_attack.md
07	对抗防御	AT (Madry), FOSC, AWP, TRADES, MART	lec07_adversarial_defense.md
08	后门攻击	BadNets, Blend, Label-Consistent, Hidden Trigger, Trojan; BA / ASR	lec08_backdoor_attack.md
09	后门防御	Neural Trojans, Neural Cleanse, STRIP, DP-SGD, Fine-Pruning, ANP, CLP	lec09_backdoor_defense.md
10	隐私	MIA (Shokri 影子训练), 模型反演, GPT-2 训练数据抽取 (Carlini), HE / SMPC / DP / FL	lec10_privacy.md

全局术语中英对照

中文	English	缩写	首次出现
可信机器学习	Trustworthy Machine Learning	TML	Lec 01
机器学习	Machine Learning	ML	Lec 01
深度学习	Deep Learning	DL	Lec 01
鲁棒性	Robustness	–	Lec 01
可解释性	Explainability / Interpretability	–	Lec 01
公平性	Fairness	–	Lec 01
隐私	Privacy	–	Lec 01
后门攻击	Backdoor Attack	–	Lec 01
损失函数	Loss Function	L	Lec 02
梯度下降	Gradient Descent	GD	Lec 02
随机梯度下降	Stochastic Gradient Descent	SGD	Lec 02
学习率	Learning Rate	η	Lec 02
超参数	Hyperparameter	–	Lec 02
均方误差	Mean Squared Error	MSE	Lec 02
平均绝对误差	Mean Absolute Error	MAE	Lec 02
交叉熵	Cross-Entropy	CE	Lec 02
过拟合	Overfitting	–	Lec 02
模型偏差	Model Bias	–	Lec 02
偏差-方差权衡	Bias-Variance Trade-off	–	Lec 02
交叉验证	Cross Validation	CV	Lec 02
卷积神经网络	Convolutional Neural Network	CNN	Lec 03
感受野	Receptive Field	–	Lec 03
参数共享	Parameter Sharing	–	Lec 03
滤波器 / 卷积核	Filter / Kernel	–	Lec 03
步长	Stride	s	Lec 03
填充	Padding	p	Lec 03
池化	Pooling	–	Lec 03
全局平均池化	Global Average Pooling	GAP	Lec 03
特征图	Feature Map	–	Lec 03
模型无关	Model-Agnostic	–	Lec 04
模型相关	Model-Specific	–	Lec 04
局部可解释模型	LIME (Local Interpretable Model-Agnostic Explanations)	LIME	Lec 04
局部代理模型	Local Surrogate Model	–	Lec 04
合作博弈	Cooperative Game	–	Lec 04
边际贡献	Marginal Contribution	–	Lec 04
显著图	Saliency Map	–	Lec 05
类激活图	Class Activation Map	CAM	Lec 05
反向传播	Backpropagation	BP	Lec 05
捷径学习	Shortcut Learning	–	Lec 05

中文	English	缩写	首次出现
对抗样本	Adversarial Example	AE	Lec 06
快速梯度符号法	Fast Gradient Sign Method	FGSM	Lec 06
基础迭代法	Basic Iterative Method	BIM	Lec 06
投影梯度下降	Projected Gradient Descent	PGD	Lec 06
白盒攻击	White-box Attack	–	Lec 06
黑盒攻击	Black-box Attack	–	Lec 06
迁移性	Transferability	–	Lec 06
物理对抗	Physical Adversarial	–	Lec 06
变换期望	Expectation over Transformations	EOT	Lec 06
对抗训练	Adversarial Training	AT	Lec 07
一阶静止条件	First-Order Stationary Condition	FOSC	Lec 07
对抗权重扰动	Adversarial Weight Perturbation	AWP	Lec 07
准确率-鲁棒性权衡	Accuracy-Robustness Trade-off	–	Lec 07
错分样本	Misclassified Examples	–	Lec 07
梯度遮蔽	Gradient Masking / Obfuscation	–	Lec 07
后门 / 触发器	Backdoor / Trigger	–	Lec 08
投毒样本	Poisoned Sample	–	Lec 08
良性准确率	Benign Accuracy	BA	Lec 08
攻击成功率	Attack Success Rate	ASR	Lec 08
脏标签攻击	Dirty-Label Attack	–	Lec 08
清洁标签攻击	Clean-Label Attack	–	Lec 08
隐蔽触发器	Hidden / Invisible Trigger	–	Lec 08
数据投毒	Data Poisoning	–	Lec 08
神经清洁	Neural Cleanse	–	Lec 09
中位绝对偏差	Median Absolute Deviation	MAD	Lec 09
输入预处理	Input Preprocessing	–	Lec 09
自动编码器	Auto-Encoder	AE	Lec 09
投毒抑制	Poison Suppression	–	Lec 09
神经元剪枝	Neuron Pruning	–	Lec 09
对抗神经元剪枝	Adversarial Neuron Pruning	ANP	Lec 09
通道 Lipschitz 剪枝	Channel Lipschitz Pruning	CLP	Lec 09
谱范数	Spectral Norm	–	Lec 09
成员推理攻击	Membership Inference Attack	MIA	Lec 10
影子模型	Shadow Model	–	Lec 10
模型反演	Model Inversion	–	Lec 10
训练数据抽取	Training Data Extraction	–	Lec 10
困惑度	Perplexity	PPL	Lec 10
个人身份信息	Personally Identifiable Information	PII	Lec 10
同态加密	Homomorphic Encryption	HE	Lec 10
安全多方计算	Secure Multi-Party Computation	SMPC	Lec 10
差分隐私	Differential Privacy	DP	Lec 10
灵敏度	Sensitivity	Δf	Lec 10
拉普拉斯机制	Laplace Mechanism	–	Lec 10
高斯机制	Gaussian Mechanism	–	Lec 10
联邦学习	Federated Learning	FL	Lec 10
FedAvg	Federated Averaging	–	Lec 10

期末复习路线图

[TIP] 全局复习思路

1. 回路 1 —ML 基础回顾: Lec 02–03, 能徒手写出”三步范式 + CNN 三大原则”2. 回路 2 —可解释: Lec 04–05, 能默写 LIME 算法 4 步、Shapley 公式 (4.5)、CAM 推导 (5.4–5.7)。3. 回路 3 —对抗: Lec 06–07, 能默写 FGSM/BIM/PGD 三式 + AT min-max + FOSC + 三大改进 (AWP/TRADES/MART) 的损失公式。4. 回路 4 —后门: Lec 08–09, 能默写 trigger function $\mathcal{A}(x; t, m) = (1 - m) \odot x + m \odot t$ 、Neural Cleanse 的 $\|m\|_1$ 优化目标 + MAD 检测、ANP 的双 min-max。5. 回路 5 —隐私: Lec 10, 能默写 (ϵ, δ) -DP 定义、Laplace / Gaussian 机制噪声尺度、Shokri 影子训练 4 步、Carlini GPT-2 抽取的 6 个排序指标。6. 跨章节联系: – CNN (Lec3) → CAM (Lec5): CAM 要求 GAP + 线性分类层 (ResNet 架构) – 可解释 (Lec4–5) → 攻击 (Lec6): Saliency / Shapley 能用来诊断模型为何被对抗样本骗 – 攻击 (Lec6) → 防御 (Lec7): AT 的 inner-max 就是 PGD, FOSC 度量它的解质量 – 对抗 (Lec7) → 后门 (Lec9): DP-SGD 同时是后门投毒抑制工具 – 后门 (Lec9) → 隐私 (Lec10): 噪声训练 / DP-SGD 同时缓解 MIA 和后门 7. 概念题预测: – 给个具体 CNN 配置算输出尺寸 / 参数量 – 3 玩家手算 Shapley value – 写出 PGD 一步的具体更新 – 写出 AT min-max 并解释每个 max 的物理含义 – FOSC 闭式公式 (7.3) 的推导 – 给定 trigger pattern + mask, 写出 BadNets stamp function – Neural Cleanse 为什么用 ℓ_1 正则 – (ϵ, δ) -DP 定义 + Laplace 机制噪声尺度

Lecture 01 —可信机器学习课程概览 (Course Overview)

[TIP] 学习指南

本节核心：建立“可信机器学习 (Trustworthy ML)”的总体认知框架，理解为什么传统 ML 不再够用，并对课程的六大支柱 (可解释性 / 公平性 / 隐私 / 鲁棒性 / 后门 / 伦理) 形成直观印象。前置知识：本科概率论、线性代数、基本的机器学习概念 (训练 / 测试 / 损失函数)。重点关注：(1) 可信 ML 的定义与传统 ML 的差异；(2) 六大模块各自要解决的核心问题；(3) 学术写作中常出现的英文术语对照。

0. 核心问题 / Motivation

近十年来，机器学习模型 (尤其是深度神经网络) 在感知、语言、决策等任务上取得了人类难以企及的精度。然而当模型被部署到 **真实物理世界** 与 **高风险决策场景** (司法、医疗、金融、自动驾驶) 时，它的“高准确率”光环就远远不够了。本课程的主线问题是：

如何在保证准确率与效率的同时，让机器学习系统对“鲁棒性 / 隐私 / 公平 / 可解释 / 安全”等多维度可信属性同时负责？

王奕森老师在课程定位上明确：本课不是一门工程实战课，而是带学生进入可信 ML 这一研究前沿领域，建立批判式思考能力，能够 (a) 识别模型的不可信表现、(b) 设计相应的防御 / 改进机制、(c) 阅读最新顶会论文。

1. 课程组织

1.1 教学信息

- 授课教师：王奕森 (Yisen Wang)，北大信智学院
- 邮箱：yisen.wang@pku.edu.cn
- 助教：Ang Li (charles_li@stu.pku.edu.cn)、Taiye Chen (yeyutaihan@stu.pku.edu.cn)
- 办公室：理科 2 号楼 2201

1.2 评估方式

作业	主题	占比
HW 1	Explanations (可解释性)	25%
HW 2	Privacy (隐私)	25%
HW 3	Robustness (鲁棒性)	25%
HW 4	Backdoor (后门)	25%

- 作业形式：填代码 (fill the code)。工作量不重，无期末考。
- 课堂会有作业 review，鼓励发言交流。

- 优秀率 (excellence rate) 不设限制——课程鼓励大家把作业做扎实，最终拿到优 (A/A+) 的学生数不被强制控制。

[WARN] 易错点

“无考试”≠“轻松”。本课作业偏向论文式编程任务 (例如 PGD 攻击实现、Shapley value 近似)，需要花时间读文献。不要等到 ddl 前两天再开始。

1.3 学习目标

“Understand how to **assess** and **improve** trustworthiness of ML.”

注意两个动词：**assess** (度量 / 评估) 与 **improve** (改进 / 设计算法)。前者属于“诊断”，后者属于“治疗”。两条主线贯穿整门课。

2. 从 AI 到可信 ML

2.1 定义层级

$$AI \supset ML \supset DL \quad (1.1)$$

- 人工智能 (AI)：由机器表现出的智能，区别于自然智能 (人 / 动物的意识与情感)。
- 机器学习 (ML)：Tom Mitchell 1997 经典定义——A computer program is said to learn from experience E w.r.t. some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .
- 深度学习 (DL)：以多层人工神经网络 (ANN) 为代表的 ML 子集，灵感来自大脑皮层结构。

[NOTE] 直觉

ML 与 rule-based 编程的本质区别：**ML 的“规则”是从数据中自动归纳出来的**，而非工程师手写 if-else。这赋予了机器“演化”的可能性，也带来了“我们其实并不完全理解模型在做什么”的不可信问题。

2.2 ML 的典型任务

- 回归 (Regression)：输出实数标量，例如预测明天的 PM2.5。
- 分类 (Classification)：在给定类别集合中挑出正确答案，例如垃圾邮件过滤、AlphaGo 落子 ($19 \times 19 = 361$ 类)。
- 生成 (Generation)：输出新样本 (本节未展开，后续 LLM / 扩散模型涉及)。

2.3 AI 的常见分支

机器学习 (ML)、计算机视觉 (CV)、自然语言处理 (NLP)、语音识别 (Speech)、以及作为载体的智慧医疗 / 交通 / 金融。ML 是其他分支的核心方法论。

3. 为什么需要“可信”？

3.1 现实中的失败案例

课件列举了若干“翻车”现场：

场景	失败表现	对应可信维度
刷脸支付	用 3D 打印面具被破解	鲁棒性 + 隐私
特斯拉车祸	自动驾驶在罕见场景下识别失效	鲁棒性

场景	失败表现	对应可信维度
ChatGPT 污染中文互联网	大模型生成虚假内容	伦理 + 真实性
AI 医疗误诊 对抗样本 (Dog → Ostrich)	正常胸片被分类为肺炎 在图像上加几乎不可见的扰动, DNN 置信度从 82% 翻转到 98% 错类	鲁棒性 + 公平性 鲁棒性
Face ID 攻击	攻击者通过对抗补丁让 Person1 不被识别	鲁棒性 + 隐私

3.2 形式化定义

Trustworthy ML 是与传统 ML 平行又延伸的概念。传统 ML 主要关注 **accuracy** 与 **efficiency**；可信 ML 在保持这两者的前提下，还要同时考虑 **robustness, privacy, fairness, explainability, security**。

可以写成一个多目标优化的语义：

$$\min_{f \in \mathcal{F}} \mathcal{L}_{\text{task}}(f) \quad \text{s.t.} \quad \underbrace{\mathcal{R}_{\text{rob}}(f) \leq \varepsilon_1}_{\text{鲁棒}}, \underbrace{\mathcal{R}_{\text{priv}}(f) \leq \varepsilon_2}_{\text{隐私}}, \underbrace{\mathcal{R}_{\text{fair}}(f) \leq \varepsilon_3}_{\text{公平}}, \dots \quad (1.2)$$

[WARN] 易错点

准确率高 $\neq$ 可信。后续会反复看到：标准训练得到的 f^* 在 i.i.d. 测试集上 99% 准确，但被一个 ℓ_∞ 范数 $\leq 8/255$ 的扰动就能打到 0% ——这正是可信 ML 关心的“鲁棒泛化 gap”。

4. 六大模块预览

4.1 可解释性 (Explainability)

核心问题：模型为什么这么预测？

应用动机：– 法律：信贷模型必须解释拒贷原因。– 医疗：医生需要知道 AI 诊断的依据，因为模型对人命负责。– 司法：模型不能有歧视；要能审计。– 自动驾驶：一旦出现异常行为，必须解释成因以追责。

后续章节会学：LIME (局部线性近似)、Shapley value (合作博弈论)、Saliency Map (梯度法)、CAM (类激活图)。

4.2 鲁棒性 (Robustness)

核心问题：当输入被对抗性地小幅扰动时，输出是否仍然稳定？

形式化：

$$\text{Robustness}(f, x) := \mathbb{P}_{\delta \sim B(\epsilon)}[f(x + \delta) = f(x)] \quad (1.3)$$

后续章节学习 FGSM / BIM / PGD / C&W / DeepFool 攻击与 AT / TRADES / MART / AWP 防御。

4.3 隐私 (Privacy)

Alan Westin 定义 (现代数据隐私法之父)：> Privacy is the claim of individuals, groups, or institutions to determine for themselves when, how, and to what extent information about them is communicated to others.

在大数据时代，免费服务的代价是隐私让渡。会涉及：成员推理攻击 (MIA)、模型反演、深度合成 (Deepfake)、差分隐私 (DP)、联邦学习。

4.4 公平性 (Fairness)

核心问题：模型对不同群体 (性别 / 种族 / 年龄) 的待遇是否一致？

直觉是“数据不会说谎”，但实际：– 数据采集偏差 (sample bias) – 标注偏差 (label bias) – 训练算法偏差 (algorithmic bias)

任一个都会让训练得到的模型产生群体歧视。

4.5 后门 (Backdoor)

核心问题：攻击者在训练阶段植入“触发器 (trigger)”，模型在干净样本上表现正常，但只要看到 trigger 就分类到目标类。属于 supply-chain 级别的安全威胁。

4.6 伦理 (Ethics)

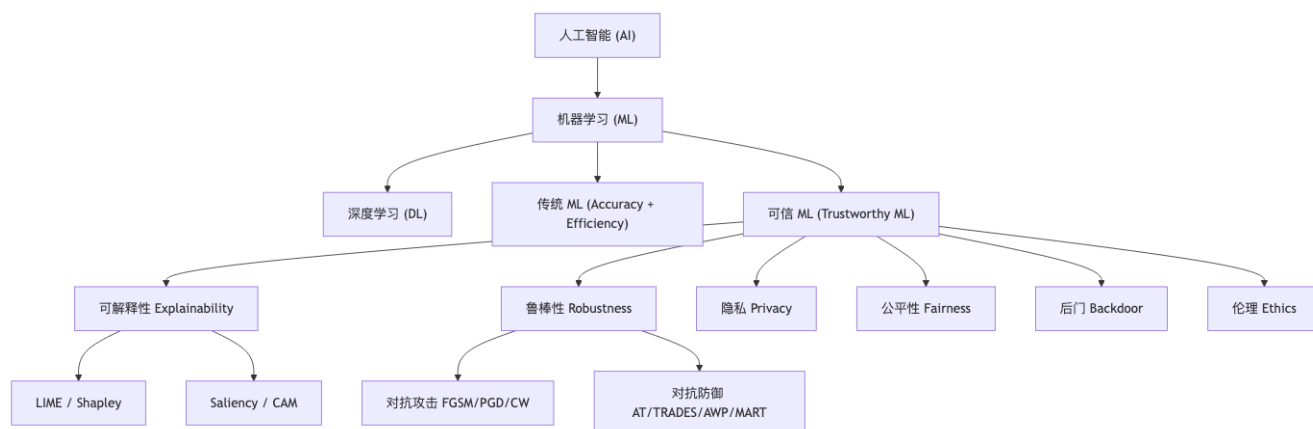
核心问题：AI 应不应该被用来做某件事？这超越技术，涉及法律、社会学。

5. 学习路径建议

[EX] 学习路径建议

1. 第 2–3 讲：补足 ML / DL 基础 (线性模型、Sigmoid、CNN)。2. 第 4–5 讲：进入解释性，从模型无关 (LIME / Shapley) 到模型相关 (Saliency / CAM)。3. 第 6–7 讲：对抗攻击 (白盒 / 黑盒) 与对抗防御 (AT / TRADES / AWP / MART)。4. 后续：隐私、公平、后门、伦理 (不属于第 1–7 讲范围)。

6. 概念关系图



7. 关键术语速查 (中英对照)

中文	English	缩写
人工智能	Artificial Intelligence	AI
机器学习	Machine Learning	ML
深度学习	Deep Learning	DL
可信机器学习	Trustworthy Machine Learning	TML
可解释性	Explainability / Interpretability	–

中文	English	缩写
鲁棒性	Robustness	–
公平性	Fairness	–
隐私	Privacy	–
后门攻击	Backdoor Attack	–
对抗样本	Adversarial Example	AE
训练 / 推理	Training / Inference	–
损失函数	Loss Function	L
梯度下降	Gradient Descent	GD

7.1 数学符号速查 (跨章节)

符号	含义
θ	模型参数向量
$f_{\theta}(\cdot)$	参数为 θ 的模型
$L(\cdot, \cdot)$	损失函数
$\mathcal{D}$	数据分布 / 训练集
δ	对抗扰动
ϵ	扰动幅度上限 (ℓ_p 范数)
$\sigma(\cdot)$	sigmoid / 标准差, 依上下文
∇	梯度算子
$\arg \min, \arg \max$	取最优自变量

8. 各维度之间的内在联系

8.1 鲁棒性 可解释性

- **互补**: 可解释方法 (Saliency / Shapley) 可暴露 shortcut learning, 从而指明哪里需要鲁棒化; 反过来, 鲁棒训练 (AT) 后的模型其 saliency 更”语义化”。
- **冲突**: 鲁棒模型在准确率上有 trade-off (Lec 07 §4); 可解释模型有时不得不简化预测过程。

8.2 隐私 解释

- 提供详细解释 **可能泄露训练数据** (model inversion via feature importance)。
- 差分隐私训练能缓解, 但会牺牲解释精度。

8.3 公平 准确率

- 公平约束 (e.g. equalized odds) 是对最大似然的硬约束, 通常会让总体 accuracy 略降, 但保证子群体之间的差异变小。

8.4 矩阵式总览

维度 影响	准确率	效率 (训练 / 推理)	其他可信维度
鲁棒性	↓ (5–10%)	显著 ↓ (AT 慢 7×)	可解释 ↑
可解释	≈	推理 ↓ (Shapley 慢)	隐私 ↓
隐私 (DP)	↓ (3–8%)	训练 ↓↓	解释 ↓

维度 影响	准确率	效率 (训练 / 推理)	其他可信维度
公平	↓ (2–5%)	≈	解释要求 ↑
后门防御	≈	训练 ↓ (re-clean)	鲁棒 ↑

注意 可信 ML 不是单目标优化，而是多目标 Pareto 前沿——工程师必须根据应用场景挑权衡点。

9. 课程外的阅读建议

虽然课程不要求，但若想拓展可信 ML 视野：– 教材：Trustworthy Machine Learning (book draft, Singh et al.) – 顶会论文：ICLR / ICML / NeurIPS 的 Adversarial / Privacy / Fairness Workshop – 综述：Yuan et al. “Adversarial Examples: Attacks and Defenses for Deep Learning”(TNNLS 2019); Mehrabi et al. “A Survey on Bias and Fairness in Machine Learning”(CSUR 2021) – 实战：CleverHans (攻击库), AdvBox, Fairlearn, SHAP

9.5 课程方法论：从”为什么”到”怎么改”

王老师在课堂上反复强调一个研究流程，可作为做作业 / 写论文的 checklist：

1. 观察现象 (Observe)：模型在什么情况下失败？(对抗扰动 / 罕见样本 / 子群体)
2. 形式化问题 (Formalize)：写出失败的数学描述，例如 $\exists \delta : \|\delta\| \leq \epsilon, f(x + \delta) \neq f(x)$ 。
3. 量化指标 (Measure)：定义可量化的可信度指标，如鲁棒准确率、 L_∞ 鲁棒半径、统计平价差 (statistical parity)。
4. 设计算法 (Design)：基于上述指标设计训练 / 后处理算法。
5. 理论分析 (Analyze)：证明算法收敛 / 给出泛化界 / 上下界。
6. 实证评估 (Evaluate)：在多个数据集 / 多个攻击 / 多个模型上测试。

后续 6 讲每一节都能套到这个 6 步流程上。

9.6 易混淆术语速辨

术语 A	术语 B	区别
Interpretability	Explainability	前者更强调”模型本身可读”，后者强调”事后解释”；课程中混用
Adversarial Example	Out-of-Distribution Sample	前者是 人为对抗构造，后者是 自然分布外；处理思路不同
Robustness	Generalization	Robustness 关心最坏情况，Generalization 关心平均；可信 ML 把”鲁棒泛化”作为统一目标
Privacy	Security	Privacy 侧重 数据不被泄露；Security 侧重 系统不被绕过 / 攻破；后门同时属于两类
Fairness	Bias	模型是否偏向某些群体 (现象) vs 偏差的来源 (数据 / 算法 / 部署)

[WARN] 易错点

期中题目里如果出现”distinguish A from B”，要能用一句话讲清差异；切忌把 Adversarial Example 和 OOD 混为一谈。

10. 期中复习要点

10. 期中复习要点

[TIP] 期中复习要点

1. 能复述课程结构：六大模块名称、各自核心问题、对应的代表性算法。2. 能给出可信 ML 与传统 ML 的差异：传统 ML 优化 $\min L$ ；可信 ML 在 $\min L$ 之外加上 robustness / privacy / fairness 等约束 (见式 1.2)。3. 能举出 3 个真实失败案例：对抗样本翻转 ImageNet 标签 (Dog $\rightarrow$ Ostrich)；Face ID 被攻破；ChatGPT 污染语料。能说明它们各自违背了哪个可信维度。4. 明白评估方式：4 次作业各 25%，无考试。5. 熟悉 Mitchell 的 ML 经典定义：experience E 、task T 、performance P 。可信 ML 实际上是在 P 上做扩展，加入多个性能指标 (不只是 accuracy)。6. 理解可信 ML 是多目标 Pareto 优化：不同可信维度之间存在 trade-off，没有”全维度都最优”的模型。7. 能在脑中画出 §6 的概念关系图：AI $\rightarrow$ ML $\rightarrow$ 可信 ML $\rightarrow$ 六大模块；每个模块再有代表算法。

Lecture 02 —机器学习基础 (Introduction of Machine Learning)

[TIP] 学习指南

本节核心：用一条主线——“机器学习 $\approx$ 找一个函数 f ”——串起线性模型、Sigmoid 复合模型、神经网络的统一框架，并完整给出 ML 训练的三步范式 (Function $\rightarrow$ Loss $\rightarrow$ Optimization)。前置知识：高数 (偏导)、线性代数 (矩阵乘法)、概率 (期望、概率分布)。重点关注：(1) 梯度下降公式与超参；(2) 模型偏差 / 过拟合 / Bias-Complexity Trade-off；(3) 分类 vs 回归的输出层差异 (Softmax / Cross-entropy)。

0. 核心问题 / Motivation

机器学习的本质：

$$\text{Machine Learning} \approx \text{Looking for a function } f \quad (2.1)$$

给定输入 x 与希望的输出 y ，我们要在一类候选函数 $\mathcal{F}$ 中找一个 f 让 $f(x) \approx y$ 。下面所有内容 (线性回归、CNN、Transformer) 都是 $\mathcal{F}$ 的不同形式。

1. 函数类型

按输出形式可以分两类：

1.1 回归 (Regression)

函数输出一个 标量。例如：

$$f : (\text{今日 PM2.5, 温度, O}_3) \mapsto \text{明日 PM2.5}$$

1.2 分类 (Classification)

给定类别集合 $\mathcal{Y} = \{1, 2, \dots, K\}$ ，函数输出正确类别。例如：- 垃圾邮件过滤： $\mathcal{Y} = \{\text{Yes, No}\}$ - AlphaGo 落子： $\mathcal{Y} =$ 棋盘上的 $19 \times 19 = 361$ 个位置 (每个位置视作一个类)

[NOTE] 直觉

分类与回归在底层数学上其实统一：都是把输入映射到一个实向量，再用 argmax / 阈值 / softmax 转成离散决策。区别只在最后一层的“读取方式”。

2. ML 训练三步范式

以预测 YouTube 频道明日观看数为例 (李宏毅老师的经典案例):

2.1 Step 1: 带未知参数的函数 (Function with Unknown)

基于领域知识猜一个模型:

$$y = b + w x_1 \quad (2.2)$$

其中 x_1 是今天的观看数 (feature), w (weight) 与 b (bias) 是 **未知参数**, 需要从数据中学习。

2.2 Step 2: 定义损失 (Loss)

损失是参数的函数 $L(b, w)$, 刻画”这组参数有多差”。

取训练数据 $\{(x_1^{(n)}, \hat{y}^{(n)})\}_{n=1}^N$, 逐样本计算 **error**:

$$e_n = |\hat{y}^{(n)} - y^{(n)}| \quad \text{或} \quad e_n = (\hat{y}^{(n)} - y^{(n)})^2 \quad (2.3)$$

再做平均:

$$L = \frac{1}{N} \sum_{n=1}^N e_n \quad (2.4)$$

Loss 名称	公式	适用
MAE (Mean Absolute Error)	$e = y - \hat{y} $	回归, 鲁棒于离群点
MSE (Mean Squared Error)	$e = (y - \hat{y})^2$	回归, 光滑可导
Cross-Entropy	$e = -\sum_i \hat{y}_i \ln y_i$	分类, 配合 softmax

[WARN] 易错点

MSE 与 Cross-Entropy 不能随便互换。分类任务用 MSE 会让 softmax 输出在两端饱和时梯度极小, 训练几乎不动。分类首选 **Cross-Entropy**。

2.3 Step 3: 优化 (Optimization)

目标:

$$w^*, b^* = \arg \min_{w, b} L(w, b) \quad (2.5)$$

梯度下降 (Gradient Descent):

1. 随机初始化 w^0 (并 b^0);
2. 计算梯度:

$$g_w = \left. \frac{\partial L}{\partial w} \right|_{w=w^0, b=b^0}, \quad g_b = \left. \frac{\partial L}{\partial b} \right|_{w=w^0, b=b^0}$$

3. 更新参数:

$$w^1 \leftarrow w^0 - \eta g_w, \quad b^1 \leftarrow b^0 - \eta g_b \quad (2.6)$$

4. 重复直到收敛。

其中 η 是 **学习率 (learning rate)**, 属于 **超参 (hyperparameter)** ——不通过梯度学, 需要工程师设定。

[EX] 例题 2.1 (梯度下降一步)

设 $L(w) = (w - 3)^2$, 初始 $w^0 = 0$, 学习率 $\eta = 0.1$ 。

解: $\frac{\partial L}{\partial w} = 2(w - 3)$, 在 $w^0 = 0$ 处 $g = -6$ 。

一步更新: $w^1 = 0 - 0.1 \times (-6) = 0.6$ 。

继续: $g = 2(0.6 - 3) = -4.8$, $w^2 = 0.6 + 0.48 = 1.08$ 。

可见每步向最优值 $w^* = 3$ 接近, 但接近速度随梯度衰减。

[WARN] 易错点

– **局部最小值 (local minima)** 在 1D 凸问题里是真问题, 但深度学习高维空间里, 绝大多数停留点是 **鞍点 (saddle point)** 而非真正的局部最小。Loss landscape 真正的难点不是 local minima。– **学习率** 太大 → 震荡发散; 太小 → 收敛过慢。后续会学 Adam / SGDM 等自适应方法。

2.4 一次完整 pipeline 回顾

Step 1: 构造带未知参数的函数 $y = b + w x_1$

Step 2: 定义损失 $L(b, w)$

Step 3: 优化 $w^*, b^* = \operatorname{argmin} L$

实际运行结果 (课件例子): $w^* = 0.97, b^* = 0.1k$, 训练 loss $L^* = 0.48k$, 2021 年测试 loss $L' = 0.58k$ 。注意训练 / 测试之间的 gap。

3. 从线性到深度

3.1 更多输入特征

只用昨天一天的观看数显然不够。把过去 7 天 / 28 天 / 56 天的观看数都纳入:

$$y = b + \sum_{j=1}^{56} w_j x_j \quad (2.7)$$

实验显示: 随着 feature 数 $7 \rightarrow 28 \rightarrow 56$, 训练 loss 持续下降 ($0.48k \rightarrow 0.38k \rightarrow 0.33k$); 但测试 loss 在 28 之后下降趋缓 ($0.58 \rightarrow 0.49 \rightarrow 0.46$)。这是泛化的临界点。

3.2 线性模型的根本局限: Model Bias

无论怎么调整 w 与 b , 线性函数 $y = b + wx$ 永远画不出 S 形、阶梯形曲线。这种“无法表达目标函数”的现象叫做 **模型偏差 (model bias)**, 需要更柔性的模型。

3.3 用 Sigmoid 组合出分段线性曲线

任何连续曲线都可以用足够多段的 **分段线性 (piecewise linear)** 函数逼近。每一“段”可以由一个 **Hard Sigmoid** 实现, 而 Hard Sigmoid 可用光滑 Sigmoid 近似:

$$y = c \sigma(b + wx_1) = c \cdot \frac{1}{1 + e^{-(b+wx_1)}} \quad (2.8)$$

参数效果：- w 控制斜率 - b 控制水平偏移 - c 控制高度

把若干 sigmoid 加起来，再加常数项：

$$y = b + \sum_i c_i \sigma(b_i + w_i x_1) \quad (2.9)$$

这就能 逼近任意连续 1D 函数 (Universal Approximation 的最简版本)。

3.4 多输入版

$$y = b + \sum_i c_i \sigma\left(b_i + \sum_j w_{ij} x_j\right) \quad (2.10)$$

其中 w_{ij} 是第 i 个 sigmoid 神经元对第 j 个特征的权重。

3.5 矩阵形式 (重要)

设

$$r = b + Wx, \quad a = \sigma(r), \quad y = b + c^\top a \quad (2.11)$$

把所有可学习参数打包成向量：

$$\theta = [W; b; c; b] \quad (2.12)$$
$$y = f_\theta(x)$$

这就是一个 单隐层神经网络 的完整数学描述。

[NOTE] 直觉

把 (2.11) 看作一个流水线：原始特征 $x \rightarrow$ 线性变换 $Wx + b \rightarrow$ 非线性激活 $\sigma \rightarrow$ 线性输出 $c^\top a + b$ 。非线性激活是核心，没有它再多层也只等价于一个线性变换。

3.6 优化新模型

把 w, b 推广到 θ 后，梯度下降的形式不变：

$$\theta^1 \leftarrow \theta^0 - \eta g, \quad g = \nabla L(\theta^0) \quad (2.13)$$

实际工程里通常用 mini-batch SGD：把训练集切成大小为 B 的 batch，每个 batch 算一次梯度做一次更新；遍历一遍数据集叫做一个 epoch。

1 epoch = 看完所有 batch 一遍

4. 评估与过拟合

4.1 训练 / 测试 gap

完美训练 loss $\neq$ 好模型。极端反例：

$$f(x) = \begin{cases} \hat{y}_i, & \exists x_i = x \\ \text{random}, & \text{otherwise} \end{cases}$$

训练 loss 为 0，但测试时几乎全错。这种 零训练损失 + 高测试损失 就是 过拟合 (overfitting) 的极端形式。

4.2 过拟合的成因

模型过于柔性 (flexible)、训练数据不足，模型记住了训练数据的噪声而非规律。

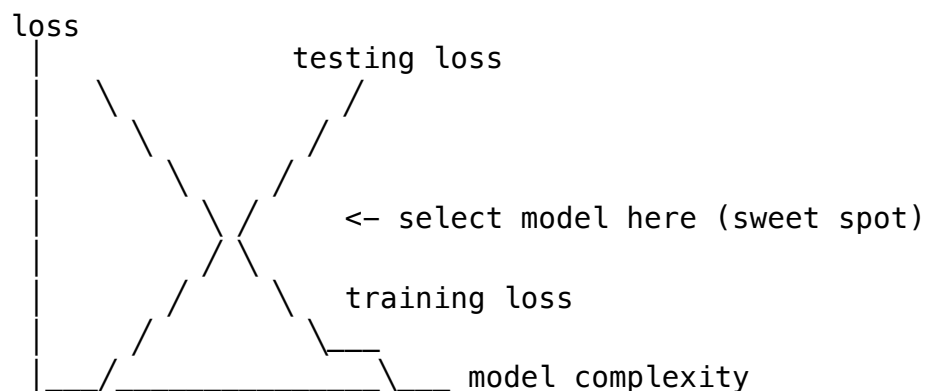
4.3 缓解过拟合的方法

思路	具体做法
增加有效数据	更多训练数据、数据增强 (Data Augmentation)
约束模型复杂度	更少参数、参数共享 (CNN)、更少 feature、Dropout、L2 正则化
限制训练时长	Early Stopping
限制函数形式	用 $y = a + bx + cx^2$ 替代任意多项式

[WARN] 易错点

约束过度 → 又回到 model bias 太大、训练 loss 高的状态。Bias-Complexity Trade-off 永远存在。

4.4 Bias-Complexity Trade-off (重要)



- 模型太简单 → bias 大，训练 / 测试 loss 都高 (欠拟合)。
- 模型太复杂 → variance 大，训练 loss 低但测试 loss 高 (过拟合)。
- 最优解：测试 loss 极小值，往往在两条曲线“拐弯”处。

4.5 N-Fold Cross Validation

把训练集切成 N 等份，第 k 次用第 k 份做验证、其余做训练，得到 N 个验证 mse，平均得到该超参组合的 CV 误差。选 CV 误差最小的模型。

例 (3-fold):

Fold	M1	M2	M3
1	0.4	0.5	0.3
2	0.5	0.6	0.4
3	0.3	0.4	0.2
Avg	0.4	0.5	0.3 ← 最佳

[WARN] 易错点

在 CV 内还要做 真正的 测试集分离 (holdout test set)，否则模型选择本身就会偷看 test。课件里的“Testing Set”应在 CV 之前就被锁起来。

5. 分类的特殊处理

5.1 把分类当回归?

直接把类别编号成 1, 2, 3 再做回归是糟糕的, 因为类别之间没有自然顺序 (狗、猫、树 没有”狗 < 猫 < 树”)。

5.2 One-hot 编码 + Softmax

把类别 k 编码成 one-hot 向量:

$$\hat{y} = e_k \in \{0, 1\}^K$$

模型输出 K 维实数向量 y , 再过 softmax:

$$y'_i = \frac{e^{y_i}}{\sum_j e^{y_j}}, \quad \sum_i y'_i = 1, \quad y'_i \in (0, 1) \tag{2.14}$$

5.3 Cross-Entropy 损失

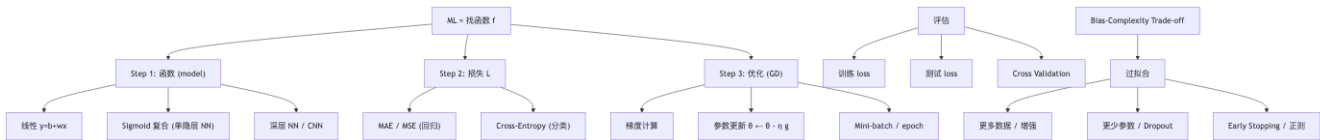
$$e = - \sum_i \hat{y}_i \ln y'_i \tag{2.15}$$

$\hat{y}$ 是 one-hot 时, cross-entropy 简化为 $-\ln y'_{\text{true}}$, 即”正确类别的 log 似然取负”。

[EX] 例题 2.2 (Softmax + CE 计算)

- 设 3 类问题, 模型输出 $y = [3, 1, -3]$, 真实类别为 1。
- Softmax: $e^3, e^1, e^{-3} \approx 20, 2.7, 0.05$, 和 ≈ 22.75
 - $y'_1 = 20/22.75 \approx 0.88$
 - $y'_2 = 2.7/22.75 \approx 0.12$
 - $y'_3 \approx 0$
 - CE: $e = -\ln 0.88 \approx 0.128$

6. 概念关系图



7. 记号速查

符号	含义
x, x	输入特征 (标量 / 向量)
$\hat{y}, y$	标签 / 模型输出
w, b	权重 / 偏置
θ	所有可学习参数打包成的向量
$L(\theta)$	损失函数

符号	含义
η	学习率
∇L	损失关于参数的梯度
$\sigma(\cdot)$	Sigmoid 激活 $\sigma(z) = 1/(1 + e^{-z})$
$\text{softmax}(\cdot)$	概率归一化 $\propto e^{y_i}$
B	mini-batch 大小
N	训练集大小

8. 期中复习要点

[TIP] 期中复习要点

1. **三步范式**：能用一句话复述“function with unknown $\rightarrow$ define loss $\rightarrow$ optimization”，并写出对应公式。
2. **梯度下降**：能给出更新规则 $\theta \leftarrow \theta - \eta \nabla L$ ；解释 η 的影响；说明 local minima 在高维不是核心问题，鞍点才是。
3. **Sigmoid 复合 = 神经网络**：能从分段线性逼近的角度推导为什么神经网络能拟合任意连续函数。
4. **矩阵化**：能把 $y = b + \sum_i c_i \sigma(b_i + \sum_j w_{ij} x_j)$ 写成 $y = b + c^\top \sigma(Wx + b)$ 。
5. **过拟合 vs model bias**：能画出 bias-complexity trade-off 曲线，并列至少 4 种缓解过拟合的方法。
6. **分类的输出层**：能解释为什么用 softmax + cross-entropy 而非直接 MSE。
7. **Cross Validation**：能描述 N-fold 流程；强调 test set 必须在 CV 之外。
8. **mini-batch / epoch**：能给出定义；解释 batch size 对方差与显存的影响（定性即可）。

Lecture 03 —卷积神经网络 (Convolutional Neural Network, CNN)

[TIP] 学习指南

本节核心：从“为什么图像不能用全连接”出发，推导出 CNN 三大设计原则 (Receptive Field, Parameter Sharing, Pooling)，并把两种等价“故事”(神经元视角 vs 滤波器视角) 拼接到一起。前置知识：第 2 讲的神经网络 / 梯度下降 / 矩阵运算；图像即 3-D 张量 (Height $\times$ Width $\times$ Channels) 的基本概念。重点关注：(1) 输出 feature map 尺寸的计算；(2) 三种 simplification 与 model bias 的关系；(3) 参数量对比 FC vs Conv。

0. 核心问题 / Motivation

把一张 $100 \times 100 \times 3$ 的彩色图直接展平输入全连接网络 (Fully Connected, FC)，第一层若有 1000 个隐藏神经元就需要 $100 \times 100 \times 3 \times 1000 = 3 \times 10^7$ 个参数。这导致两个问题：

1. **参数爆炸** $\rightarrow$ 严重过拟合 + 显存炸；
2. **没有先验** $\rightarrow$ 模型既能学到“鸟”的概念，也可能学到完全无关的像素组合 (例如把背景颜色当作类别标志)。

CNN 的诞生就是为图像这一类 **具有局部结构的数据** 注入合适的归纳偏置 (inductive bias)。

1. Observation 1: 局部特征 $\rightarrow$ Receptive Field

1.1 直觉

人类识别一只鸟时，先看 **鸟嘴、鸟眼、鸟爪** 等局部模式，不必看整张图。深层一些的神经元再把这些局部检测器的结果组合 (鸟嘴 + 鸟爪 + 翅膀 = 鸟)。

“Some patterns are much smaller than the whole image.”

结论：每个神经元不需要看整张图，只需关注一小块区域——**感受野 (receptive field)**。

1.2 Simplification 1: 局部连接

设 receptive field 大小 $k \times k$ (e.g. 3×3)，channel 数为 C (e.g. 3)，则每个神经元的参数量从 $H \cdot W \cdot C$ 缩减为 $k^2 C$ 。例如 $3 \times 3 \times 3 = 27$ 个权重 + 1 个 bias = 28 个参数。

1.3 典型设置 (Typical Setting)

- **Kernel size:** 3×3 或 5×5 ，覆盖所有 channel
- **Stride:** 感受野中心的滑动步长 (e.g. stride=1 或 stride=2)
- **Padding:** 边界补零，让输出尺寸可控

- 每个感受野 有一组神经元 (e.g. 64 个), 每个神经元对应一组独立的 $3 \times 3 \times C$ 权重 (但同感受野的多个神经元不共享参数)
- 多个感受野 覆盖整张图, 可以重叠

[NOTE] 直觉

感受野设计回答的是 **空间局部性**: 相邻像素往往相关, 距离越远的像素相关性越弱。所以”只看局部”是合理的归纳偏置。

1.4 输出尺寸公式

设输入 $H \times W$, 卷积核 $k \times k$, padding p , stride s , 输出尺寸为:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1 \quad (3.1)$$

[EX] 例题 3.1

输入 6×6 , kernel 3×3 , padding 0, stride 1。

$H_{\text{out}} = (6 - 3)/1 + 1 = 4$, 输出 4×4 。

2. Observation 2: 模式平移不变 → Parameter Sharing

2.1 直觉

“鸟嘴”这个 pattern 可能出现在图像的左上角, 也可能出现在右下角; 但 **检测器应该相同**——不必为每个位置训练一个独立的“鸟嘴检测器”。

“The same patterns appear in different regions.”

2.2 Simplification 2: 参数共享

让所有感受野共享同一组权重 (即 **filter / kernel**)。这样: - 一个 $3 \times 3 \times C$ 的 filter 可以检测整张图任何位置的同一种模式; - 多个 filter (e.g. 64 个) 检测不同种模式 (边缘、纹理、颜色块等)。

注意: 同一个感受野内的不同神经元 (e.g. 64 个 filter 在同一空间位置的输出) **不共享参数**, 否则就退化成单一 filter 了。

2.3 参数量对比

模型	第 1 层参数量 (假设 1000 隐藏单元 / 64 filter)
FC, 输入 $100 \times 100 \times 3 \rightarrow 1000$	3×10^7
Conv, $3 \times 3 \times 3 \times 64$ filters	$3 \times 3 \times 3 \times 64 \approx 1.7 \times 10^3$

参数减少 4 个数量级。

3. 两种等价视角

CNN 可以从两个角度理解，它们在数学上是等价的：

Neuron Version	Filter Version
每个神经元只看一个感受野 不同感受野的神经元共享参数	一组 filter 在图像上卷积 每个 filter 滑过整张图

[WARN] 易错点

教科书 / 课件常常在两种视角之间切换。看到“filter”与“neuron”时要意识到它们指向同一件事，不要被搞晕。

4. 卷积运算的具体计算

4.1 单 filter 卷积示例

输入 6×6 灰度图 (channel=1), filter $1 = \begin{pmatrix} 1 & -1 & -1 \\ -1 & 1 & -1 \\ -1 & -1 & 1 \end{pmatrix}$ (检测斜对角)。

卷积：每个 3×3 窗口与 filter 逐元素相乘求和，得到一个标量；窗口在图像上以 stride 1 滑动，得到 4×4 的输出 feature map。

4.2 多 filter, 多 channel

- 输入： $H \times W \times C_{in}$
- F 个 filter, 每个 $k \times k \times C_{in}$
- 输出： $H_{out} \times W_{out} \times F$ (F 个 channel)

注意：filter 的 channel 数必须等于输入的 channel 数，每个 filter 输出一个 channel。

4.3 多层堆叠

- 第 1 层：filter $3 \times 3 \times 3$, 64 个 $\rightarrow$ 输出 64 channel
- 第 2 层：filter $3 \times 3 \times 64$, 再 64 个 $\rightarrow$ 输出 64 channel
- ...

层数加深 $\rightarrow$ 有效感受野 (effective receptive field) 越来越大，最深层的单个神经元可以“看到”输入图的相当大区域，从而捕获高层语义。

5. Observation 3: 子采样 $\rightarrow$ Pooling

5.1 直觉

把图像下采样 (e.g. 抽掉一半像素) 后，鸟仍然是鸟。

“Subsampling the pixels will not change the object.”

5.2 Max Pooling

把 2×2 区域取最大值:

$$y_{ij} = \max\{x_{2i,2j}, x_{2i+1,2j}, x_{2i,2j+1}, x_{2i+1,2j+1}\} \quad (3.2)$$

效果: – 降低 feature map 尺寸 (空间维度 $\div 2$) – 减少参数与计算 – 提供轻微的平移不变性 (translation invariance)

5.3 Average Pooling

$$y_{ij} = \frac{1}{4} \sum_{2 \times 2 \text{ window}} x \quad (3.3)$$

比 max pooling 平滑, 常用在 ResNet 最后一层 (Global Average Pooling, GAP)。

[WARN] 易错点

Pooling 会丢信息。在高分辨率检测 / 分割任务中过度 pooling 会损失细节。现代架构 (e.g. ResNet) 倾向于用 `stride=2` 卷积 替代 pooling, 让下采样过程也是可学习的。

6. 完整 CNN Pipeline

Input image

- ↓ Conv ($k \times k \times C_{in} \times F$ filters)
- ↓ ReLU / Sigmoid
- ↓ Pooling
- ↓ Conv
- ↓ ReLU
- ↓ Pooling
- ... (重复若干 Conv+Pool 块)
- ↓ Flatten (把 $H \times W \times C$ 拉成 1-D)
- ↓ FC layers
- ↓ Softmax

Output: cat / dog / ...

这是 LeNet / AlexNet / VGG 的统一框架。后续 ResNet 在此基础上加入 **skip connection** 解决深层网络的梯度退化。

7. 三大设计原则的统一总结

观察	简化	后果
局部模式	Receptive Field (局部连接)	参数 ↓, model bias ↑
模式平移不变	Parameter Sharing	参数 ↓↓
子采样不变	Pooling	计算 ↓, translation invariance ↑

注意 **Convolutional Layer** 比 **FC** 的 **model bias** 更大——它强加了“图像有局部结构 + 平移不变”的先验。对图像这是好事；对完全无空间结构的表格数据，则会反而限制表现。

[NOTE] 直觉

“Jack of all trades, master of none”——FC 什么都能拟合但什么都拟合不好；CNN 通过引入合理偏置，在图像上做精通。

8. 一个完整例题

[EX] 例题 3.2 (CNN 输出尺寸 + 参数量计算)

给定输入 $32 \times 32 \times 3$ (CIFAR-10 图像), 网络结构:

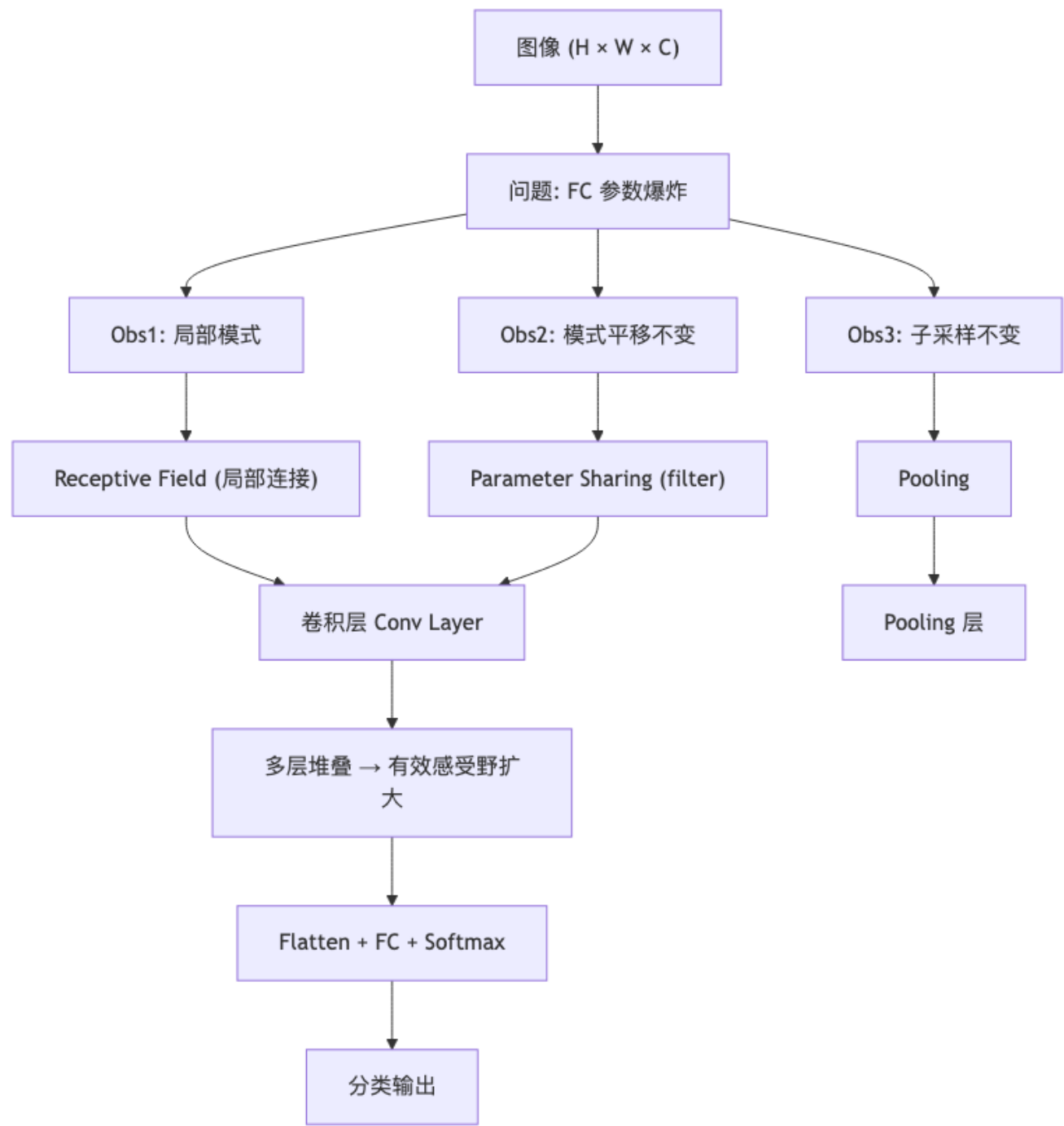
1. Conv 3×3 , 32 filters, stride=1, padding=1
2. ReLU
3. Max Pool 2×2 , stride=2
4. Conv 3×3 , 64 filters, stride=1, padding=1
5. Max Pool 2×2 , stride=2
6. Flatten $\rightarrow$ FC 10

逐层尺寸: - 输入: $32 \times 32 \times 3$ - 1) Conv: $H_{\text{out}} = (32 + 2 - 3)/1 + 1 = 32$, 输出 $32 \times 32 \times 32$
- 3) Pool: $32/2 = 16$, 输出 $16 \times 16 \times 32$ - 4) Conv: $16 + 2 - 3 + 1 = 16$, 输出 $16 \times 16 \times 64$
- 5) Pool: $16/2 = 8$, 输出 $8 \times 8 \times 64$ - 6) Flatten: $8 \times 8 \times 64 = 4096$, FC $4096 \rightarrow 10$

参数量: - 1) $3 \times 3 \times 3 \times 32 + 32 = 896$ - 4) $3 \times 3 \times 32 \times 64 + 64 = 18496$ - 6) $4096 \times 10 + 10 = 40970$ - 合计 ≈ 60362 。

同样规模的纯 FC 第一层就要 $32 \times 32 \times 3 \times 1000 \approx 3 \times 10^6$, 可见 CNN 的紧凑性。

9. 概念关系图



10. 记号速查

符号	含义
H, W, C	图像高 / 宽 / channel
k	kernel size (e.g. 3)
p	padding

符号	含义
s	stride
F	filter 数量 (输出 channel 数)
$f_k(x, y)$	第 k 个 filter 在空间 (x, y) 处的激活
GAP	Global Average Pooling

10.5 训练 CNN 时的常见技巧

为帮助后续作业实现，下列经验性技巧应熟悉：

- **数据增强 (Data Augmentation)**: 随机裁剪 (RandomCrop)、水平翻转 (RandomFlip)、颜色抖动 (ColorJitter)、CutMix、MixUp。在 CIFAR-10 上能提升 3–5%。
- **批归一化 (Batch Normalization)**: 对每层激活做规范化 $\hat{z} = (z - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$ ，让训练更稳定，可用更大学习率。
- **学习率调度**: StepLR / CosineAnnealing / Warmup，避免后期 loss 抖动。
- **权重初始化**: He init (ReLU) / Xavier init (tanh, sigmoid)，避免梯度消失 / 爆炸。
- **Dropout**: FC 层间随机置零，缓解过拟合 (但 CNN 层间常被 BN 替代)。

[WARN] 易错点

BN 在测试时使用的是训练阶段累积的 running mean/var，**不是** 当前 batch 的统计。这一点在做 PGD 攻击 (Lec 06) 时尤其重要：很多攻击会在 eval mode 下进行，须确认 BN 处于固定参数状态。

11. 经典 CNN 架构纵览

虽然课件只点了 CNN 基本结构，但备考可以横向比较：

架构	年份	关键创新	备注
LeNet-5	1998	卷积 + 池化交替, 适用 MNIST	第一代实用 CNN
AlexNet	2012	ReLU + Dropout + GPU 训练, ImageNet 冠军	引爆深度学习
VGG	2014	全 3×3 卷积堆叠, 简单优雅	16/19 层
GoogLeNet (Inception)	2014	1×1 卷积降维, Inception block	22 层但参数少
ResNet	2015	Skip connection 解决梯度消失	50/101/152 层
DenseNet	2017	每层都和之前所有层相连	参数复用
MobileNet	2017	Depthwise Separable Conv	移动端友好
EfficientNet	2019	复合缩放 (depth, width, resolution)	SOTA + 参数效率

Skip Connection 公式

ResNet 的核心:

$$y = \mathcal{F}(x, \{W_i\}) + x \quad (3.4)$$

让网络只学 **残差** $\mathcal{F}(x) = y - x$ ，恒等映射变得容易学到 $\rightarrow$ 极深网络也能稳定训练。后续 Lec 06 会看到 skip connection 还和 **对抗迁移性** 紧密相关。

12. 期中复习要点

[TIP] 期中复习要点

1. **三大原则**: Receptive Field, Parameter Sharing, Pooling, 能各举一个观察来说明。2. **公式 (3.1)** 必背: 能根据输入尺寸 + kernel + padding + stride 算出输出 feature map 尺寸。3. **参数量计算**: 第 ℓ 层卷积参数 = $k \times k \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$ (bias)。4. **两种视角的等价性**: 能说明“neuron with receptive field”和“filter convolution”是同一回事。5. **CNN vs FC 的 model bias**: CNN 因为施加了图像先验, bias 比 FC 大; 这是好事 (减小过拟合) 也是限制 (在表格数据上反而不如 FC)。6. **Pooling 类型**: max / average / global average; 能说明各自适用场景。7. **完整 pipeline**: 能默写 Conv $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow \dots \rightarrow$ Flatten $\rightarrow$ FC $\rightarrow$ Softmax 的标准结构。8. **典型 CNN 架构对比**: LeNet、AlexNet、VGG、GoogLeNet、ResNet 关键区别; 能默写 ResNet skip connection 式 (3.4)。9. **Lec3 $\rightarrow$ Lec5 桥梁**: CAM 要求 GAP + 线性分类层, ResNet 满足。10. **Lec3 $\rightarrow$ Lec6 桥梁**: Skip connection 在对抗攻击中有特殊作用 (transferability 更高)。

Lecture 04 —可解释机器学习 (Explanations: LIME & Shapley)

[TIP] 学习指南

本节核心：进入“可信 ML”第一大支柱——**可解释性**。从两个 **模型无关 (model-agnostic)** 解释方法入手——LIME (局部线性近似) 与 Shapley Value (合作博弈论)，理解它们的算法、性质、优缺点。前置知识：第 2 讲的线性模型、加权最小二乘；基本概率 (期望、条件期望)；可选：合作博弈论中“特征函数”概念。重点关注：(1) LIME 的算法 4 步；(2) Shapley 公理化定义 (4 axioms)；(3) Shapley 公式与近似 (Monte Carlo)。

0. 核心问题 / Motivation

“I know **why** the answers are wrong, so I can fix it.”——解释让我们能 (a) 诊断模型错误 (b) 满足合规要求 (c) 增进对模型 / 数据的理解。

可解释 ML 需要回答：

给定训练数据 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ ，一个已学好的模型 f ，以及实例 $\vec{x} \in \mathbb{R}^p$ ，**找一个对 $f(\vec{x})$ 的解释**。

什么算“解释”？常见形式：– **特征重要性 (feature importance)** ——哪些 feature 推动了这次预测 – **决策规则**——等价的¹可读规则集合 – **样例对照**——反事实 / 影响样本 – **可视化**——Saliency / CAM (见第 5 讲)

1. 解释方法的分类

按对模型的访问程度：

类别	说明	代表方法
Model-Agnostic Model-Specific	把 f 当黑盒，只用 query 利用模型内部信息	LIME, Shapley, PDP, ALE Saliency (梯度), CAM (CNN), attention 权重

本节聚焦 model-agnostic 的两个最经典方法。

1.1 有“天生可解释”的模型吗？

线性模型： $y = b + \sum_j w_j x_j$ 。 w_j 的大小与符号直接表示 feature 重要性 / 方向。但实际场景 (图像、文本、高维表格) 中线性模型表现往往太弱。

决策树 / 规则集合 也是天生可解释；但深度网络远比它们强。所以需要 **事后解释 (post-hoc explanation)** 把黑盒模型局部“翻译”成可解释模型。

2. LIME (Local Interpretable Model-Agnostic Explanations)

2.1 核心思想

全局上，深度模型决策边界复杂得无法理解；但在某个具体样本 $\vec{x}$ 的邻域，决策边界可以用一个简单可解释模型 g (e.g. 线性模型) 近似。

[NOTE] 直觉

想象一座蜿蜒的山脉 (全局复杂)，但站在山脚某一点，你只需用一个切平面描述脚下的坡度——这个切平面就是 LIME 的局部代理 (local surrogate)。

2.2 算法 4 步

Step 1: 给定要解释的点 x

Step 2: 在 x 的邻域采样 m 个扰动点 z_i

Step 3: 用线性 (或其他可解释) 模型拟合 $(z_i, f(z_i))$,
损失按距离 $\|z_i - x\|$ 加权

Step 4: 解读这个线性模型的系数 = 特征重要性

2.3 形式化

定义可解释模型族 $\mathcal{G}$ (通常是线性模型族)，距离核 $w_i \propto \exp(-\|z_i - \vec{x}\|^2 / \sigma^2)$ 。

优化目标：

$$\arg \min_{g \in \mathcal{G}} \frac{1}{m} \sum_{i=1}^m w_i L(f(z_i), g(z_i)) + \lambda \Omega(g) \quad (4.1)$$

其中 $\Omega(g)$ 是模型复杂度正则项 (如 L_1 稀疏化，让最终只剩少数 feature)。

当 g 是线性模型 $g(z) = \langle \vec{\beta}, z \rangle$ 时：

$$\arg \min_{\vec{\beta} \in \mathbb{R}^p} \frac{1}{m} \sum_{i=1}^m w_i (f(z_i) - \langle \vec{\beta}, z_i \rangle)^2 + \lambda \|\vec{\beta}\|_1 \quad (4.2)$$

通过 LASSO / 正交匹配追踪解此问题，调整 λ 控制最终保留几个非零 β_j (即用户希望的解释特征数)。

2.4 邻域采样的具体做法

表格数据

$z_{ij} \sim \mathcal{N}(\mu_j, \sigma_j^2)$ ，其中 μ_j, σ_j 是训练集第 j 特征的均值 / 方差。

图像数据

- 把图像分割成 **超像素 (superpixel)**
- 随机“打开 / 关闭”超像素 (开 = 保留原像素, 关 = 替换成灰色)
- 形成二进制向量 $z_i \in \{0, 1\}^K$ ，这就是可解释特征

文本数据

- 随机删除若干 token，形成 BoW 向量

2.5 经典案例：Wolf vs Husky

LIME 在一组“狼 / 哈士奇”图像分类器上发现：模型其实是在判断 **背景里有没有雪**。预测为狼的图像里，重要特征是 **雪地**；预测为哈士奇时，重要特征是 **室内地板**。这种“偷懒”现象 (**shortcut learning**) 是 LIME 暴露黑盒病灶的经典案例。

2.6 优缺点

优点： – **人类友好：** 少量特征 + 可解释代理模型 – **通用：** 任何黑盒都能用 (回归 / 分类 / 多模态) – **变体丰富：** 文本 LIME、图像 LIME、表格 LIME

缺点： – **邻域定义敏感：** 核宽度 σ 一改，解释就变 – **不稳定：** 相邻样本可能得到完全不同的解释 (因为采样随机) – **局部线性假设可能错：** 当 f 在 $\vec{x}$ 附近高度非线性时，线性 g 拟合不准

[WARN] 易错点

LIME 的解释只是 **局部** 的——不要把“这个像素重要”推广成“模型整体上看这个像素”。每个样本可能有完全不同的解释。

3. 从博弈论引入：Shapley Value

3.1 合作博弈的小例子

3 个程序员合作写一个 500 行项目，单人 / 多人组合的产出：

$$\begin{aligned}v(\{1\}) &= 100, & v(\{2\}) &= 125, & v(\{3\}) &= 50 \\v(\{1, 2\}) &= 270, & v(\{2, 3\}) &= 350, & v(\{1, 3\}) &= 375 \\v(\{1, 2, 3\}) &= 500\end{aligned}$$

问：如何在 3 人间 **公平** 分配总产出 500？

直观答案要满足： – 每个人拿到的与他的 **真实贡献** 成正比 – 全部分完 – 对换名字答案不变 (对称) – ...

Shapley (1953) 证明：**满足若干“公平公理”的分配方案，存在且唯一。**

3.2 Shapley 公理 (4 axioms)

记 $\phi_i(v)$ 为玩家 i 在博弈 v 中应得的份额。

Efficiency (有效性 / 全分配)

$$\sum_{i \in P} \phi_i(v) = v(P) \quad (4.3)$$

总和等于大联盟的总产出。

Symmetry (对称性)

若 i 和 j 可互换 (即对所有不含 i, j 的子集 S 都有 $v(S \cup \{i\}) = v(S \cup \{j\})$)，则 $\phi_i = \phi_j$ 。

Linearity (线性性)

对两个博弈 v, w 与实数 a ：

$$\phi_i(v + w) = \phi_i(v) + \phi_i(w), \quad \phi_i(a \cdot v) = a \cdot \phi_i(v) \quad (4.4)$$

Null Player (空玩家)

若 i 是“空玩家”，即 $v(S \cup \{i\}) = v(S)$ 对所有 $S \not\ni i$ ，则 $\phi_i(v) = 0$ 。

3.3 Shapley 公式 (Theorem)

[NOTE] 定理 4.1 (Shapley 1953)

满足以上 4 公理的唯一分配方案是：

$$\Phi_j(v) = \sum_{S \subseteq P \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [v(S \cup \{j\}) - v(S)] \quad (4.5)$$

其中 $p = |P|$ ，求和遍历所有不含 j 的子集 S 。

解读：- $v(S \cup \{j\}) - v(S)$ 是 j 加入联盟 S 后的 **边际贡献** - 权重 $\frac{|S|!(p-|S|-1)!}{p!}$ 是把这种“加入顺序”在所有 $p!$ 排列中按等概率加权 - 因此 $\Phi_j = j$ 在所有可能加入顺序下的 **平均边际贡献**

3.4 把 Shapley 用到机器学习

类比 (核心翻译表)：

博弈论	解释 ML
博弈 v	一次预测 $f(\vec{x})$
全联盟产出	$f(\vec{x}) - \mathbb{E}[f]$ (预测与平均预测之差)
玩家 i	特征 j
玩家份额 ϕ_i	特征重要性 $\varphi(j, f, \vec{x})$

3.5 ML 中的 Shapley 公式

把 $v(S)$ 实例化为“只观察特征集合 S 时的期望预测”：

$$\psi(f, \vec{x}, S) := \mathbb{E}[f(X) \mid X_k = x_k, k \in S] \quad (4.6)$$

则

$$\varphi(j, f, \vec{x}) = \sum_{S \subseteq \{1, \dots, p\} \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [\psi(f, \vec{x}, S \cup \{j\}) - \psi(f, \vec{x}, S)] \quad (4.7)$$

3.6 计算难点与近似

(4.7) 的子集个数是 2^{p-1} ，对 $p = 100$ 已是 10^{30} 级别，**无法精确计算**。常用近似：

1. **Monte Carlo 采样**：随机抽 K 个排列，每次按位置算 j 的边际贡献，最后平均。
2. **KernelSHAP**：把 Shapley 化为加权线性回归 (SHAP 库的核心)。
3. **TreeSHAP**：在 Tree-based 模型上能多项式时间精确计算。

3.7 优缺点

优点：- 满足 公理化的公平性：理论性质极佳 - 考虑了特征间的交互：不像单变量遮挡 (occlusion)，会平均所有 coalition - 全局 (整数据集 Shapley 累积) + 局部 (单样本 Shapley) 视角均可用

缺点：- 计算量爆炸： $O(2^p)$ ，精确不可行 - 解释偏抽象：“这个 feature 对 (预测 - 平均预测) 的差异贡献了多少”——对非技术用户难懂 - 依赖参考分布： $\mathbb{E}[f(X) \mid X_S]$ 的估计需要 background dataset，选择不同 background 结果不同

[WARN] 易错点

SHAP value 与“single-feature drop-off”(去掉某 feature 后 loss 变化) 不一样：SHAP 平均了 所有可能的特征加入顺序，是顺序无关的“公平”分摊。

4. 一个完整例题

[EX] 例题 4.1 (Shapley 手算)

沿用 §3.1 的 3 人合作项目： $v(\emptyset) = 0$, $v(1) = 100$, $v(2) = 125$, $v(3) = 50$, $v(1, 2) = 270$, $v(1, 3) = 375$, $v(2, 3) = 350$, $v(1, 2, 3) = 500$ 。

用公式 (4.5) 计算 Φ_1 ：

子集 $S \subseteq \{2, 3\}$ ，共 4 个： $\emptyset, \{2\}, \{3\}, \{2, 3\}$ 。 $p = 3$ 。

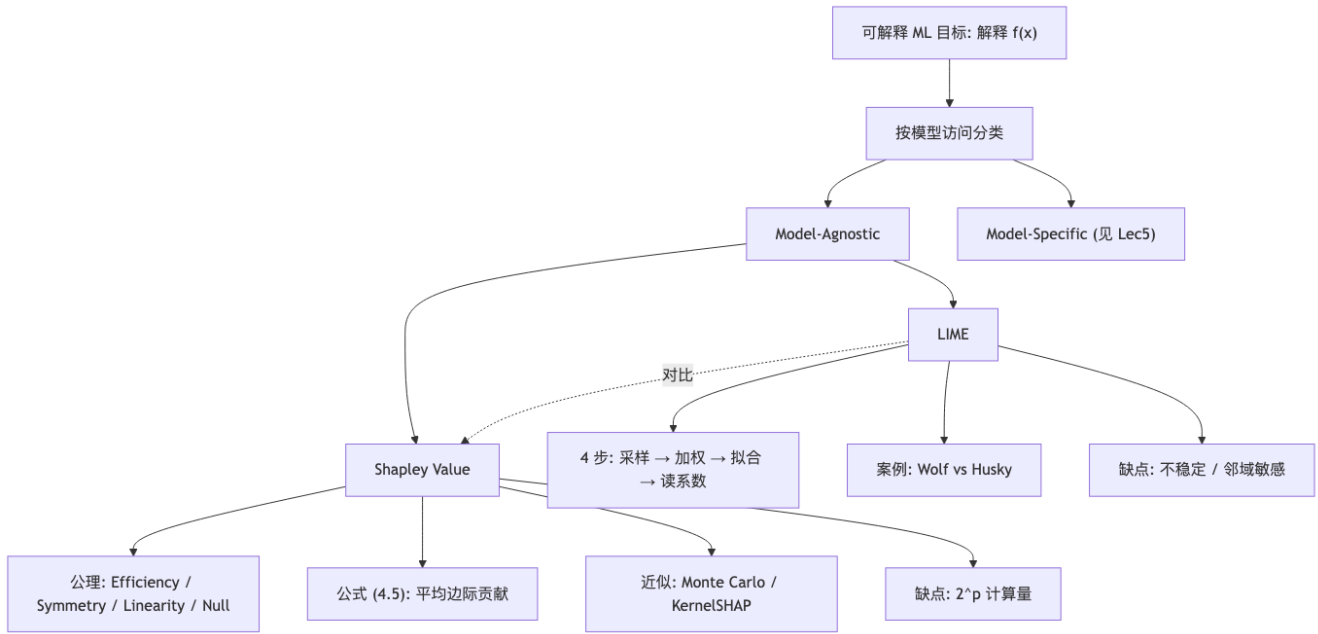
S	$\ S\ $	系数 $\frac{\ S\ !(p-\ S\ -1)!}{p!}$	边际 $v(S \cup \{1\}) - v(S)$
$\emptyset$	0	$0! \cdot 2! / 3! = 2/6 = 1/3$	$100 - 0 = 100$
$\{2\}$	1	$1! \cdot 1! / 3! = 1/6$	$270 - 125 = 145$
$\{3\}$	1	$1/6$	$375 - 50 = 325$
$\{2, 3\}$	2	$2! \cdot 0! / 3! = 2/6 = 1/3$	$500 - 350 = 150$

$\Phi_1 = \frac{1}{3} \cdot 100 + \frac{1}{6} \cdot 145 + \frac{1}{6} \cdot 325 + \frac{1}{3} \cdot 150 = 33.33 + 24.17 + 54.17 + 50 = 161.67$
类似可算 $\Phi_2 = 158.33$, $\Phi_3 = 180$ 。验证 Efficiency: $161.67 + 158.33 + 180 = 500$

5. LIME vs Shapley 对比

维度	LIME	Shapley
思想	局部线性代理	平均边际贡献 (博弈论)
复杂度	$O(m \cdot p)$ ，可控	$O(2^p)$ ，需近似
稳定性	低 (邻域采样影响大)	高 (有公理化定义)
直观度	高 (像线性回归系数)	中 (需要博弈论背景)
适用	表格 / 图像 / 文本	表格为主，图像需特殊处理
主要库	lime	shap (KernelSHAP, TreeSHAP)

6. 概念关系图



7. 记号速查

符号	含义
$\vec{x}$	要解释的实例
f	已训练好的 (黑盒) 模型
$g \in \mathcal{G}$	局部代理 (可解释模型)
z_i	邻域采样点
w_i	距离权重 (核函数)
$\Omega(g)$	代理模型复杂度正则
P	玩家全集 / 特征全集
$S \subseteq P \setminus \{j\}$	不含 j 的子集
$v(S)$	特征函数 / 联盟价值
Φ_j	Shapley value
$\varphi(j, f, \vec{x})$	ML 版 Shapley (feature j 对 $f(\vec{x})$ 的贡献)
$\psi(f, \vec{x}, S)$	只看 S 时的期望预测

8. 期中复习要点

[TIP] 期中复习要点

1. **解释方法分类**：能写出 model-agnostic vs model-specific，并各举 2 个代表。2. **LIME 算法 4 步**：能默写采样 → 加权 → 拟合代理 → 解读系数；说明为什么用 LASSO 控制稀疏。3. **Wolf vs Husky 案例**：能讲清“模型偷懒”现象 (shortcut learning)，及 LIME 如何暴露。4. **LIME 缺陷**：邻域敏感、不稳定、依赖 σ ；5. **Shapley 4 公理**：Efficiency, Symmetry, Linearity, Null Player ——能用一句话给出每条。6. **公式 (4.5)** 必背：能解读 $\frac{|S|!(p-|S|-1)!}{p!}$ 为“按加入顺序的均匀概率”。7. **手算练习**：3 玩家或 3 特征的 Shapley 能完整算出 (§4 例题)。8. **近似方法**：知道 Monte Carlo / KernelSHAP / TreeSHAP 各自适用场景。9. **优缺点对比表**：能填出 LIME vs Shapley 在复杂度 / 稳定性 / 直观度上的差异。

Lecture 05 —Saliency Map 与 Class Activation Map (CAM)

[TIP] 学习指南

本节核心：进入 **model-specific** 解释方法，利用网络内部（梯度 / 激活）直接生成可视化热图。两条主线：(1) 反向传播视角 → Saliency Map / SmoothGrad；(2) 前向传播视角 → CAM。前置知识：第 2 / 3 讲的 CNN、反向传播、Softmax；矩阵展开 / 加权和。重点关注：(1) Saliency = $|\partial e / \partial x_i|$ 的物理意义；(2) Pokémon vs Digimon 案例（假高准确率元凶）；(3) CAM 公式推导与 ResNet 结构要求。

0. 核心问题 / Motivation

第 4 讲学的 LIME / Shapley 是 **model-agnostic** 的事后解释：把模型当黑盒，靠采样 / 拟合。优点是通用，缺点是计算贵 (Shapley) / 不稳定 (LIME)。

Model-specific 视角：模型自身就是可读的——神经网络的 **梯度** 与 **激活** 直接揭示了“哪里重要”。本节学最经典的两类：

视角	方法	来源
反向传播 (Backward)	Saliency Map, SmoothGrad	梯度 $\nabla_x \ell$
前向传播 (Forward)	CAM (Class Activation Map)	最后一层 conv 激活与全连接权重的加权和

1. Saliency Map: 梯度即解释

1.1 直觉

模型 f 在输入 x 处的输出对 x_i (第 i 个像素 / 分量) 的灵敏度，可由偏导刻画：

$$\left| \frac{\partial e}{\partial x_i} \right| \quad (5.1)$$

其中 e 是损失 (或某类别的 logit / score)。梯度越大，意味着该像素的微小变化会显著改变模型输出 → 该像素“重要”。

1.2 推导

考虑两个邻近输入：

$$x = (x_1, \dots, x_i, \dots, x_n) \quad \text{vs} \quad x' = (x_1, \dots, x_i + \Delta x, \dots, x_n)$$

损失变化 Δe 满足

$$\Delta e \approx \frac{\partial e}{\partial x_i} \Delta x \implies \left| \frac{\Delta e}{\Delta x} \right| \approx \left| \frac{\partial e}{\partial x_i} \right| \quad (5.2)$$

1.3 Saliency Map 算法

输入：训练好的模型 f ，待解释图像 x ，类别 c

1. 前向： $y = f(x)$, $e = \text{score}_c$ (或 cross-entropy with label c)
2. 反向： $g = \partial e / \partial x$ (size = $x.\text{shape}$, 一次反向传播即可)
3. Saliency: $|g|$ 取绝对值, 通道上取最大 (或求和)
4. 可视化: 把 $|g|$ reshape 成图像尺寸的热图

计算成本: 1 次前向 + 1 次反向 = 一次完整的 backprop, 远比 LIME 的多次重采样便宜。

1.4 朴素 Saliency 的问题：噪声

直接 $|\nabla_x e|$ 通常很 嘈杂, 原因: - ReLU 等非线性带来梯度的“尖锐” - 像素级别梯度对小扰动极敏感 - 训练好的网络在 saturation 区域梯度小但实际语义重要

1.5 SmoothGrad: 高斯平滑

[NOTE] 直觉

既然单点梯度抖动大, 那就在 x 邻域采样多个加噪点, 把它们的梯度平均一下, “洗”掉随机性。

$$\text{SmoothGrad}(x) := \frac{1}{n} \sum_{k=1}^n \nabla_x e(x + g_k), \quad g_k \sim \mathcal{N}(0, \sigma^2 I) \quad (5.3)$$

实验表明 (σ 取像素值标准差的 10–20%), SmoothGrad 给出的热图比朴素 Saliency 更连贯、定位更准。

1.6 案例研究：Pokémon vs Digimon (重要)

这是课件中最具警示意义的案例。

实验设置: - 训练数据: 宝可梦 PNG (透明背景) + 数码宝贝 JPEG - 训练精度 98.9%, 测试精度 98.4% ——看起来 “Amazing!”

Saliency Map 检查: - 模型几乎只看 图像四角的背景

真相: - PNG 文件有 透明通道, 加载时透明部分变 黑色 - JPEG 没有透明通道, 背景多为 白色 / 自然色 - 模型其实在区分 “背景是不是黑的” 而非 “宝可梦 vs 数码宝贝”

[WARN] 易错点 (这个例子要刻在 DNA 里)

高准确率不等于学到了正确的概念。任何 “看起来太好” 的结果都应该用 Saliency / Shapley 等工具去检查。这种现象叫 shortcut learning / spurious correlation, 是可信 ML 关心的核心病态之一。

2. CAM: Class Activation Map

2.1 前置：架构要求

CAM 不能用于任意 CNN, 要求倒数第二层是 Global Average Pooling (GAP), 倒数第一层是 线性 (FC) 分类层。ResNet / GoogLeNet 等天然满足。

... → Conv → ReLU → ... → 最后一层 conv (输出 $H' \times W' \times K$)
 ↓ GAP
 $(1 \times 1 \times K)$ 即 F_k
 ↓ Linear ($K \rightarrow \bar{C}$ 类)
 logits → softmax

2.2 推导

记最后一层 conv 在空间位置 (x, y) 处第 k 个 channel 的激活为 $f_k(x, y)$ 。GAP 给出：

$$F_k = \sum_{x,y} f_k(x, y) \quad (5.4)$$

类 c 的 softmax 输入 (logit) 为：

$$S_c = \sum_k w_k^c F_k \quad (5.5)$$

其中 w_k^c 是 GAP 后那一层全连接权重 (类 c 对 channel k 的权重)。忽略 bias。

代回 (5.4)：

$$S_c = \sum_k w_k^c \sum_{x,y} f_k(x, y) = \sum_{x,y} \underbrace{\sum_k w_k^c f_k(x, y)}_{M_c(x,y)} \quad (5.6)$$

定义 Class Activation Map：

$$M_c(x, y) := \sum_k w_k^c f_k(x, y) \quad (5.7)$$

解读： $M_c(x, y)$ 直接告诉我们“空间位置 (x, y) 对类 c 的总贡献”——把 M_c 双线性上采样到原图尺寸，叠加可视化，就是类 c 的 CAM 热图。

2.3 为什么需要 GAP

如果倒数第二层不是 GAP 而是 Flatten + FC，那 FC 的权重就和 **某个具体空间位置上的 channel** 一一绑定，难以拆出“空间 × channel”的乘积结构。GAP 提供了一个“空间维度被聚合”的瓶颈，让 w_k^c 自然成为 channel 重要性，进而可以重新分发到空间。

2.4 实验现象

CAM 论文 (Zhou et al., CVPR 2016) 的经典图：同一张图在不同预测类下生成不同的热图——例如对“dome”类，热图集中在 **建筑的圆顶**；对“palace”类，热图集中在 **下方平整建筑体**。表明 CAM 能很好定位 **类别相关区域**。

2.5 应用：弱监督定位

意外副产品：只用图像级标签 (image-level label) 训练分类器，CAM 就能 **粗略定位物体边界框**。无需任何像素级标注 → 显著降低标注成本。这是 **weakly-supervised localization** 的奠基方法之一。

2.6 CAM 的局限

局限	说明
架构限制	必须 GAP + 线性分类层
分辨率低	M_c 在最后一层 conv 尺寸 (e.g. 7×7), 需上采样到原图, 细节丢失
只能解释最后一层	对中间层的语义没有直接刻画

后继: Grad-CAM (Selvaraju et al., 2017) 把 (5.7) 中的 w_k^c 换成 **每个 channel 的梯度均值**:

$$\alpha_k^c = \frac{1}{Z} \sum_{x,y} \frac{\partial S_c}{\partial f_k(x,y)} \quad (5.8)$$

$$M_c^{\text{Grad-CAM}}(x,y) = \text{ReLU}\left(\sum_k \alpha_k^c f_k(x,y)\right) \quad (5.9)$$

这样 **任意 CNN** 都能用, 且能针对中间任意一层做。

3. Saliency vs CAM 对比

维度	Saliency / SmoothGrad	CAM
来源	Backward (梯度)	Forward (激活加权)
计算	一次反向传播	一次前向 + 取最后一层激活
分辨率	输入像素级 (高)	最后 conv 层级 (低)
架构限制	任意可微模型	GAP + 线性分类层 (后由 Grad-CAM 解除)
嘈杂度	高 (需 SmoothGrad)	较低 (天然平滑)
颗粒度	像素	区域

4. 一个完整例题

[EX] 例题 5.1 (CAM 数值计算)

设最后一层 conv 输出 $2 \times 2 \times 3$, 激活为: $-f_1 = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} - f_2 = \begin{pmatrix} 0 & 1 \\ 1 & 2 \end{pmatrix} - f_3 = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$

GAP 后: $F_1 = (1 + 2 + 0 + 1)/4 = 1, F_2 = 4/4 = 1, F_3 = 2/4 = 0.5$ 。

类“cat”的全连接权重 $w^{\text{cat}} = (2, -1, 3)$ 。

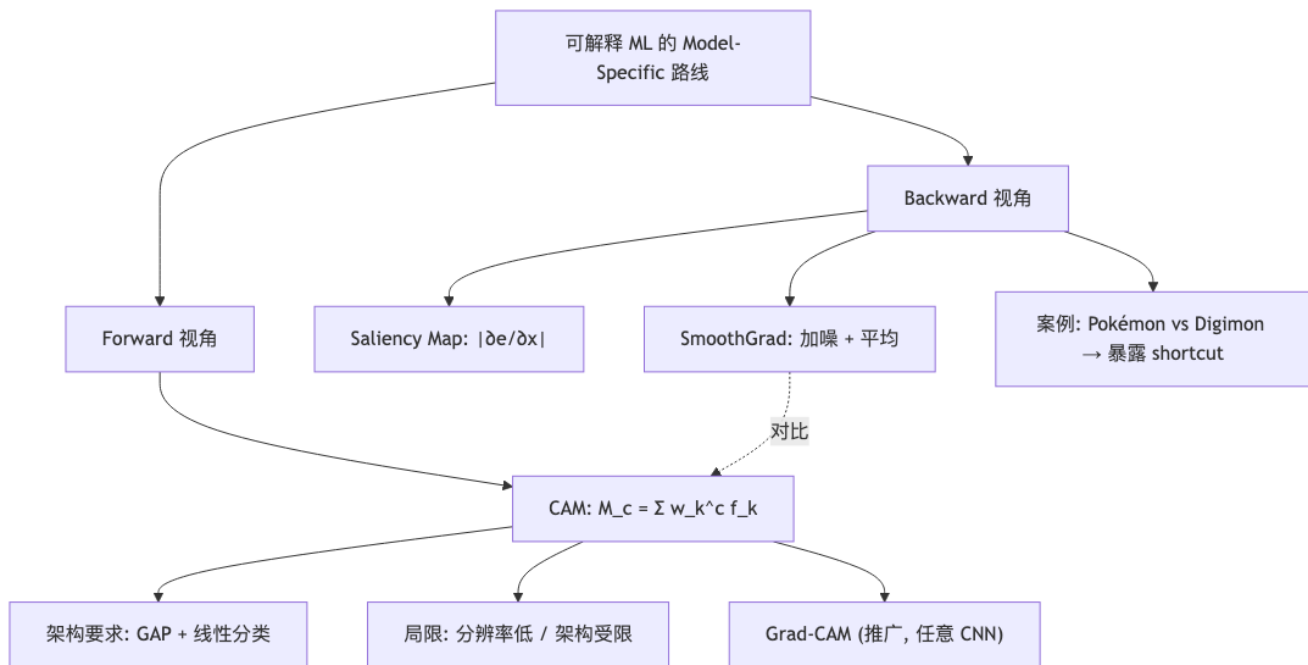
Logit $S_{\text{cat}} = 2 \cdot 1 + (-1) \cdot 1 + 3 \cdot 0.5 = 2.5$

CAM: $M_{\text{cat}}(x,y) = 2f_1(x,y) - 1f_2(x,y) + 3f_3(x,y) = 2 \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 0 & 1 \\ 1 & 2 \end{pmatrix} + 3 \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$
 $= \begin{pmatrix} 2 & 4 \\ 0 & 2 \end{pmatrix} + \begin{pmatrix} 0 & -1 \\ -1 & -2 \end{pmatrix} + \begin{pmatrix} 6 & 0 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 8 & 3 \\ -1 & 0 \end{pmatrix}$

验证: $\sum_{x,y} M_{\text{cat}}(x,y)/4 = (8 + 3 - 1 + 0)/4 = 2.5 = S_{\text{cat}}$ (与 (5.6) 一致)

解读: 左上角 (8) 是模型预测为“cat”的最强证据; 右下角 (-1, 0) 反而稍微抑制。

5. 概念关系图



5.5 其他 Saliency 变种 (拓展)

除了 SmoothGrad, 研究界还提出多个改进版:

Integrated Gradients (Sundararajan et al., ICML 2017)

$$IG_i(x) = (x_i - x'_i) \int_{\alpha=0}^1 \frac{\partial f(x' + \alpha(x - x'))}{\partial x_i} d\alpha \quad (5.10)$$

x' 是 baseline (通常黑色图像)。好处: 满足 **Completeness** 公理 (各像素 IG 之和 = $f(x) - f(x')$), 类似 Shapley。

Guided Backpropagation (Springenberg et al., 2015)

反向传播过 ReLU 时, 只保留正梯度:

$$\frac{\partial L}{\partial z_i^{(\ell)}} = \mathbb{1}[z_i^{(\ell)} > 0] \cdot \mathbb{1}[\partial L / \partial z_i^{(\ell+1)} > 0] \cdot \frac{\partial L}{\partial z_i^{(\ell+1)}}$$

让生成的热图更”干净”, 但 Sanity Check (Adebayo et al., NeurIPS 2018) 显示它有时 与模型权重无关——输出像随机网络一样的图, 警惕。

Score-CAM (Wang et al., CVPR 2020 Workshop)

不用梯度, 而是把每个 channel 的激活 mask 上原图, 重新前向, 用预测分数当 channel 权重 → 避免梯度噪声 / 饱和问题。

5.6 Sanity Check: 解释方法本身要被怀疑

Adebayo et al. (NeurIPS 2018) “Sanity Checks for Saliency Maps”测试了主流方法在以下扰动下的表现：

- **Model Parameter Randomization Test**：把训练好的模型权重逐层重置成随机值。理想情况下解释应该变化巨大。
- **Data Randomization Test**：把训练数据的 label 随机重排，重新训练。理想情况下解释应该不可读。

发现：– Saliency, Integrated Gradients, SmoothGrad **通过** – Guided BP, Guided Grad-CAM **未通过** (随机模型也给出“漂亮”的热图！)

这说明视觉好看 $\neq$ 真正反映模型。可信 ML 评估必须用 sanity check。

6. 记号速查

符号	含义
x	输入图像
e	损失或目标类的 score
$\partial e / \partial x_i$	输出关于第 i 像素的偏导
$g_k \sim \mathcal{N}(0, \sigma^2)$	SmoothGrad 高斯噪声样本
$f_k(x, y)$	最后一层 conv 第 k channel 在 (x, y) 的激活
F_k	GAP 后的 channel 标量
w_k^c	类 c 对 channel k 的 FC 权重
S_c	类 c 的 logit
$M_c(x, y)$	类 c 的 CAM 热图
α_k^c	Grad-CAM 中第 k channel 的梯度加权

7. 期中复习要点

[TIP] 期中复习要点

1. **Saliency 公式 (5.1)**：能写出 $|\partial e / \partial x_i|$ 与一阶 Taylor 近似的关系。2. **SmoothGrad 公式 (5.3)**：能写出加噪 + 平均的形式，说明为什么能去噪。3. **Pokémon vs Digimon 案例**：能讲清楚 png/jpeg 背景颜色泄露真相，理解 shortcut learning。4. **CAM 推导 (5.4–5.7)**：必须能从 GAP $\rightarrow$ logit $\rightarrow$ 重排得到 $M_c = \sum_k w_k^c f_k$ 。5. **CAM 架构要求**：必须是 GAP + 线性分类层；否则需要用 Grad-CAM。6. **CAM vs Saliency 对比表**：能从分辨率、架构、嘈杂度三个维度做比较。7. **弱监督定位**：能说明为什么只用图像级标签 + CAM 就能做近似目标检测。8. **Grad-CAM 的推广方向**：把 w_k^c 替换成梯度均值 α_k^c ，再加 ReLU。

Lecture 06 —对抗攻击 (Adversarial Attack)

[TIP] 学习指南

本节核心：进入“可信 ML”第二大支柱——鲁棒性。聚焦 攻击方 视角：在不同访问权限下 (白盒 / 黑盒)，如何生成几乎不可见但能让 DNN 翻车的 **对抗样本 (adversarial example, AE)**。前置知识：第 2 / 3 讲的梯度、Cross-Entropy; ℓ_p 范数；带约束优化的拉格朗日 / 投影方法。重点关注：(1) 对抗样本的 min-max 形式化；(2) FGSM / BIM / PGD 公式与适用场景；(3) 黑盒攻击两条主路线 (Transfer-based / Query-based)。

0. 核心问题 / Motivation

经典图：一张 **狗** 的图加上肉眼几乎看不见的扰动后，DNN 以 98% 置信度判为 **鸵鸟 (ostrich)**。这种现象 (Szegedy et al., 2013; Goodfellow et al., 2014) 让人意识到：**深度模型的高准确率在最坏情况下完全不可靠**。

对抗样本 = 输入空间中精心构造、视觉上几乎不可分、但导致模型预测翻转的样本。

形式化：模型训练目标是

$$\min_{\theta} \mathbb{E}_{(x_i, y_i) \sim \mathcal{D}_{\text{train}}} L(f_{\theta}(x_i), y_i) \quad (6.1)$$

对抗攻击则在 固定 θ 后，对每个样本求：

$$\max_{x'} L(f_{\theta}(x'), y) \quad \text{s.t.} \quad \|x' - x\|_p \leq \epsilon, \quad x \in \mathcal{D}_{\text{test}} \quad (6.2)$$

“在 x 的 ϵ -ball 内找一个让损失最大的点”。

0.1 范数选择

- ℓ_{∞} 最常见，要求每个像素的修改不超过 ϵ ；适合“全局微小噪声”
- ℓ_2 整体扰动能量小；攻击在视觉上更“光滑”
- ℓ_0 / ℓ_1 稀疏扰动；适合“修改少数像素 (one-pixel attack)”

0.2 攻击分类轴

分类轴	选项
模型访问	白盒 White-box / 黑盒 Black-box
攻击目标	无目标 Untargeted (任意错类) / 有目标 Targeted (指定目标类)
扰动形式	数字 Digital / 物理 Physical (3D 打印贴纸、补丁)
一次性 / 普适	个例 (per-sample) / 通用扰动 UAP

1. 白盒攻击 (White-box)

1.1 设置

攻击者拥有 **模型完整信息**：架构、参数、超参、梯度。这是 **最强假设、最高成功率** 的场景，常用于评估防御方法的最差表现。

1.2 Direction I —基于梯度迭代

1.2.1 FGSM (Fast Gradient Sign Method)

Goodfellow et al., 2014: 一步走，沿着损失梯度的 **符号** 方向：

$$x^{\text{adv}} = x + \epsilon \text{sign}(\nabla_x L(f_\theta(x), y)) \quad (6.3)$$

特点：– 只需 1 次前向 + 1 次反向，**极快** – 满足 $\|x^{\text{adv}} - x\|_\infty = \epsilon$ (顶到 ϵ -ball 表面) – 对 robust 模型攻击成功率较低 (一步走太粗暴)

1.2.2 BIM (Basic Iterative Method)

Kurakin et al., 2016: 把 FGSM 拆成多小步，每步走 α ，并做 **clip 投影** 回 ϵ -ball：

$$x_0^{\text{adv}} = x, \quad x_{t+1}^{\text{adv}} = \text{Clip}_{x,\epsilon} \left\{ x_t^{\text{adv}} + \alpha \text{sign}(\nabla_x L(f(x_t^{\text{adv}}), y)) \right\} \quad (6.4)$$

其中 $\text{Clip}_{x,\epsilon}(z) := \min(\max(z, x - \epsilon), x + \epsilon)$ 是逐元素的截断。

1.2.3 PGD (Projected Gradient Descent) —白盒攻击金标准

Madry et al., 2018: 在 BIM 之上加 **随机初始化**：

$$x_0^{\text{adv}} = x + \text{Uniform}(-\epsilon, \epsilon) \quad (6.5)$$

迭代公式同 (6.4)。**随机起点** 让 PGD 探索 ϵ -ball 内更多局部极大值，相当于一个带约束的强 **first-order** 攻击。

[NOTE] 直觉

把 ϵ -ball 想成一个小方盒子。FGSM 朝坡度最陡方向一步顶到墙，BIM 多步走小步走到墙边，PGD 从盒内随机一点开始多步走 → 能找到墙上”更陡的极大点”。

1.2.4 三者对比

方法	步数	随机初始化	成功率 (对未防御模型)	计算成本
FGSM	1	否	~80% (CIFAR-10)	极低
BIM	10–40	否	~98%	中
PGD	10–40	是	~99%	中

1.3 Direction II —基于优化的精细攻击

1.3.1 DeepFool

Moosavi-Dezfooli et al., CVPR 2016: 在每步把 **决策边界** 用线性近似 (类似牛顿法)，沿最近边界法向走最小距离，把样本推过去。– 优点：找到 **最小扰动** (sample-wise 几乎最优 ℓ_2) – 缺点：迭代相对慢，扰动幅度需多次校准

1.3.2 C&W (Carlini & Wagner, 2017)

把“误分类”目标写成可优化目标：

$$\min_{\delta} \|\delta\|_p + c \cdot \max \left(\max_{i \neq t} Z_i(x + \delta) - Z_t(x + \delta), -\kappa \right) \quad (6.6)$$

其中 Z_i 是 logit, t 是目标类, κ 控制置信度, c 是平衡参数 (二分搜索)。

特点：– 对许多防御 (e.g. 蒸馏 defensive distillation) 都能突破 – 计算贵, 但扰动最小化更精细 – 是评估防御鲁棒性的“金标准”之一

1.4 几种方法对比一览

方法	一步 / 迭代	主要范数	目标 / 无目标	强度
FGSM	一步	ℓ_{∞}	都可	★★
BIM	迭代	ℓ_{∞}	都可	★★★★
PGD	迭代 + 随机	ℓ_{∞} (ℓ_2 推广)	都可	★★★★★
DeepFool	迭代	ℓ_2	无目标	★★★★
C&W	优化	$\ell_2 / \ell_{\infty} / \ell_0$	都可	★★★★★

[WARN] 易错点

评估一个防御时, 不要只跑 FGSM 一遍就宣称“鲁棒”。许多 2017–2019 年的“防御”都被 PGD / C&W 攻破 (Athalye et al., 2018 “Obfuscated Gradients”论文)。**Defense evaluation 必须用强攻击。**

2. 黑盒攻击 (Black-box)

2.1 设置

模型被部署到云端 / API 后端, 攻击者 看不到 架构、参数、梯度, 只能 查询 API 获取：– 硬标签 (hard label)：只返回类别 – 软标签 / 分数 (score)：返回所有类的概率

2.2 两条主路线

黑盒攻击

- Transferability-based ← 用替代模型生成 AE, 靠迁移性攻击目标
- Query-based ← 直接查询目标 API, 迭代优化扰动

2.3 Transferability-based

2.3.1 核心观察 (Transferability)

对模型 f_1 有效的 AE, 对结构 / 训练数据相似的 f_2 通常也有效。攻击流程：

1. 训练或拿到一个 source model f_{src}
2. 用 PGD / C&W 等白盒方法在 f_{src} 上生成 AE
3. 把 AE 直接送给 target model f_{tgt}
4. 期望迁移成功

2.3.2 提升迁移性的三类技巧

(a) **Momentum-based** Dong et al., CVPR 2018 (“MI-FGSM”): 在迭代过程中维护 **动量项**:

$$g_{t+1} = \mu g_t + \frac{\nabla_x L_t}{\|\nabla_x L_t\|_1}, \quad x_{t+1}^{\text{adv}} = \text{Clip}(x_t^{\text{adv}} + \alpha \text{sign}(g_{t+1})) \quad (6.7)$$

动量能让攻击跳出 f_{src} 的局部极大点, 迁移到 f_{tgt} 时更鲁棒。

(b) **Diverse Input** Xie et al., CVPR 2019 (“DI²-FGSM”): 每次迭代前对 x_t^{adv} 做 **随机变换** (resize / pad / crop), 让 AE 不过度拟合 f_{src} 。

类似地: Translation-Invariant Attack (Dong et al., 2019)。

(c) **Architecture-related** Wu et al., ICLR 2020 (“Skip Connections Matter”): 发现 ResNet 的 skip connection 上的梯度对迁移性贡献巨大。在反向时给 skip 路径加权能显著提升迁移率。

2.3.3 局限

- 成功率难达 100%
- 受 source / target 相似度限制
- 不能保证特定目标类 (targeted) 的迁移

2.4 Query-based

2.4.1 任务

直接在目标模型上做 **零阶优化 (zeroth-order optimization)** ——没有梯度, 只能靠多次查询估梯度或搜索。

2.4.2 按返回信息分两类

Score-based (有概率分数):

方法	思想
ZOO (Chen et al., 2017)	用有限差分估梯度
AutoZOOM (Tu et al., 2019)	$\hat{g}_i \approx (L(x + h e_i) - L(x))/h$, 再做坐标下降
Square Attack (Andriushchenko et al., 2020)	用 autoencoder 把高维输入压到低维, 在低维做 ZOO
NES / NAttack	随机方块扰动 + 接受率搜索; 不需要梯度估计; 极高效 自然进化策略, 把搜索方向参数化为高斯分布

Hard-label (只有类别):

方法	思想
Boundary Attack (Brendel et al., ICLR 2018)	从一个已是 adversarial 的点出发, 沿决策边界 random walk 不断逼近原图
RayS (Chen & Gu, KDD 2020)	在 ℓ_∞ 球面方向上做 ray search, 把问题降到 1D

2.4.3 查询复杂度

攻击 ImageNet 上的 ResNet50, 2024 年代典型查询数 (untargeted, $\ell_\infty = 0.05$): - ZOO 类: $\sim 10^5$ – 10^6 - Square Attack: $\sim 10^3$ - Boundary Attack: $\sim 10^4$ – 10^5 - 现代分布学习 (NP-Attack, Bai et al., ECCV 2020): 数百 query 即可

[WARN] 易错点

查询数过多容易被 API 检测到 (rate limit / anomaly detection)。实际攻击中, “少查询 + 高成功率”才是关键指标。

3. 物理对抗攻击 (Physical AE)

数字像素扰动到现实世界往往因 打印失真 / 摄像头噪声 / 视角变化 而失效。物理攻击的目标是 在现实可实施的扰动。

3.1 主要形式

形式	代表	应用
Adversarial Patch	Brown et al., 2017	一块可见但语义不显眼的贴纸
Physical Perturbation	Eykholt et al., CVPR 2018 (“Robust Physical-World Attacks”)	在 STOP 标志上贴黑白方块, 让自动驾驶误判为“限速 45”
Adversarial Camouflage	Duan, Ma, Wang et al., CVPR 2020	把扰动伪装成自然 style, 肉眼看不出异常

3.2 训练要点 (Expectation over Transformations, EOT)

$$\delta^* = \arg \max_{\delta \in \Delta} \mathbb{E}_{T \sim \mathcal{T}} [L(f(T(x + \delta)), y)] \quad (6.8)$$

其中 T 是随机变换 (旋转、缩放、亮度、噪声) ——让 δ 对一系列变换都鲁棒。

4. 关键攻击算法对照表

算法	类别	公式核心	典型应用
FGSM	白盒	$x + \epsilon \text{sign}(\nabla L)$	教学 / 弱基线
BIM	白盒	多步 + Clip	鲁棒性下限
PGD	白盒	多步 + 随机起点 + Clip	评估金标准
C&W	白盒	最小化 $\ \delta\ + c \cdot g(x + \delta)$	突破弱防御
MI-FGSM	白盒 → 迁移	+ 动量	提升黑盒迁移
DI-FGSM	白盒 → 迁移	+ 输入变换	提升黑盒迁移
Square Attack	黑盒 score	随机方块搜索	高效
Boundary Attack	黑盒 hard-label	沿边界 walk	强限制场景
Adv Patch	物理	EOT + 贴纸	现实可部署

5. 一个完整例题

[EX] 例题 6.1 (PGD 手算一步)

设 1D 二分类, 模型 $f(x) = wx + b$ with $w = 2, b = -1$ 。真实类 $y = 1$, 标量 cross-entropy loss

$$L = -\log \sigma(wx + b) \cdot \mathbb{1}[y = 1] - \log(1 - \sigma(\cdot)) \cdot \mathbb{1}[y = 0]$$

取 $y = 1, x = 1$ 。 $\epsilon = 0.5, \alpha = 0.2, T = 2$ 步。

步骤:- $z = wx + b = 1, \sigma(1) \approx 0.731 - \partial L / \partial x = -w(1 - \sigma(z)) = -2 \cdot 0.269 \approx -0.538$
- 随机初始化 (略, 取 $x_0 = 1$)

Step 1: - $\text{sign}(\text{梯度}) = -1 - x_1 = \text{Clip}_{x, \epsilon}(1 + 0.2 \cdot (-1)) = \text{Clip}(0.8) = 0.8$ (落在 $[0.5, 1.5]$ 内)

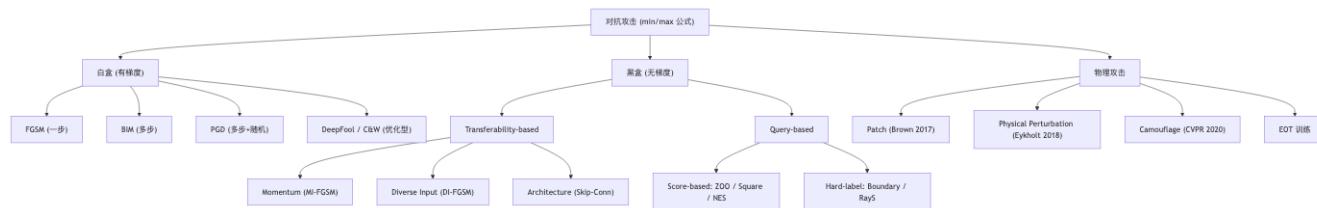
注意 (6.4) 中加号是因为我们在 **最大化** loss (即扰动让模型置信度降低)。此处 loss 关于 x 是 **减函数** 在 $y = 1$ 类内 (减 x 让 logit 减、错误概率增、loss 增), 所以 $\text{sign}(\text{梯度})$ 取负, 方向 -1 让 x 减小。

Step 2: - 在 $x_1 = 0.8$ 处重新算梯度, sign 仍为 $-1 - x_2 = \text{Clip}(0.8 - 0.2) = 0.6$

此时 $z = 2 \cdot 0.6 - 1 = 0.2, \sigma(0.2) \approx 0.55$, 模型置信度从 0.73 降到 0.55, **接近决策边界**。

若再多走几步会被 Clip 限制在 0.5 (即 $x - \epsilon = 0.5$), 此时 $z = 0, \sigma = 0.5$, 刚好越过边界。

6. 概念关系图



7. 记号速查

符号	含义
x, x^{adv}	干净样本 / 对抗样本
δ	扰动 ($x^{\text{adv}} = x + \delta$)
ϵ	扰动幅度上限
$\ \cdot\ _p$	ℓ_p 范数 ($p \in \{0, 2, \infty\}$)
L	损失函数 (通常 cross-entropy)
$\nabla_x L$	损失关于输入的梯度
$\text{sign}(\cdot)$	逐元素符号
$\text{Clip}_{x, \epsilon}$	投影回 ϵ -ball
α	PGD 步长
T	PGD 步数
$f_{\text{src}}, f_{\text{tgt}}$	替代 / 目标模型
$Z_i(x)$	类 i 的 logit
$\mathcal{T}$	EOT 中的物理变换分布

8. 期中复习要点

[TIP] 期中复习要点

1. 对抗样本 min-max 形式 (6.1)–(6.2): 能写出训练 vs 攻击的两个公式; 说明攻击是固定 θ 后的最大化。2. FGSM / BIM / PGD 三式必背: 能写出 (6.3)、(6.4)、(6.5), 解释每个符号; 说出三者的相对强度。3. PGD 为什么是金标准: 随机起点 + 多步 + 投影, 是 first-order 攻击的强者。4. C&W 的目标函数 (6.6): 理解为什么 $\max(\dots, -\kappa)$ 中 κ 控制置信度。5. 白盒 vs 黑盒: 能说清访问权限差异及对应威胁模型。6. Transferability-based 提升技巧: 动量 / 输入多样化 / 架构 (skip connection)。7. Query-based 两类: score-based (ZOO / Square) vs hard-label (Boundary / RayS)。8. 物理攻击 + EOT 公式 (6.8): 理解为什么 EOT 让物理 AE 鲁棒。9. 评估陷阱: 单一弱攻击不能下“鲁棒”结论; 多种强攻击 (PGD + C&W + 黑盒) 综合评估。10. 范数选择对扰动可视感的影响: ℓ_∞ 强制每像素不超 $\epsilon \rightarrow$ 整体看噪声; ℓ_2 总能量小 $\rightarrow$ 视觉光滑; ℓ_0 稀疏 $\rightarrow$ 少数像素被改。

Lecture 07 —对抗防御 (Adversarial Defense)

[TIP] 学习指南

本节核心：在第 6 讲看清”攻击有多狠”之后，本节回到 **防御方**：以 **对抗训练 (Adversarial Training, AT)** 为主线，理解 min-max 优化、FOSC 收敛分析，并学习两类前沿改进——**AWP (从权重视角)** 与 **TRADES / MART (从损失结构视角)**。前置知识：第 6 讲的 PGD；优化的一阶 stationary 条件；PAC-Bayesian 直觉；正则化。重点关注：(1) AT 的 min-max 形式与”内 max + 外 min”两步迭代；(2) FOSC 公式 + 与鲁棒性的相关性；(3) AWP / TRADES / MART 三种改进的动机与公式。

0. 核心问题 / Motivation

第 6 讲展示：现代 DNN 在对抗扰动下”准确率清零”。**所以我们注定脆弱吗？不一定。**如同免疫系统接触病原后产生抗体，模型也可以在训练阶段 **见过对抗样本** → 学会忽略它们。这就是 **对抗训练 (AT)** 的本质。

Adversarial aspects = stress-testing our solutions.

0.1 已被否定的”伪防御”

Gradient Masking / Obfuscation ——隐藏 / 扰乱模型梯度让攻击者无法计算 PGD 更新。Athalye et al. (2018, ICML, “Obfuscated Gradients Give a False Sense of Security”) 系统性地展示：这类防御都 **能被 BPDA / EOT 等技巧绕过**。

[WARN] 易错点

任何”让攻击者算不出梯度”的防御都要警惕 gradient masking。**真正的鲁棒性**应在 PGD / AutoAttack 等强攻击下仍保有高准确率。

1. 对抗训练 (Adversarial Training) 基础

1.1 Madry 经典 min-max 框架

Madry et al., ICLR 2018:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \underbrace{\max_{\|\delta_i\|_{\infty} \leq \epsilon} L(f_{\theta}(x_i + \delta_i), y_i)}_{\text{Inner Max: 攻击}} \quad (7.1)$$

Outer Min: 训练

实现：

```

for batch (x, y) in train_loader:
    # Inner Maximization (一轮 PGD)
    x_adv = PGD(f_theta, x, y, eps, alpha, steps)
    # Outer Minimization (一步 SGD)
    loss = cross_entropy(f_theta(x_adv), y)
    loss.backward()
    optimizer.step()

```

1.2 直觉解读

标准训练 = 在 i.i.d. 训练分布下找最优 θ 。对抗训练 = 在 训练分布的 ϵ -邻域 worst case 下找最优 $\theta \rightarrow$ 学到的模型对最坏扰动也鲁棒。

1.3 性能预览 (CIFAR-10, PGD-20 $\ell_\infty = 8/255$)

方法	干净准确率	PGD 鲁棒准确率
标准训练	~95%	~0%
Madry AT	~85%	~50%
TRADES	~84%	~55%
MART	~83%	~57%
AWP-AT	~85%	~58%

(注：精确数字依赖架构 / 训练超参，此处为粗略量级)

可见：鲁棒模型的干净准确率会下降。这就是 §4 要讲的 accuracy-robustness trade-off。

2. FOSC: 度量”内 Max 是否解好了”

2.1 为什么要度量 inner-max 的解好不好？

PGD 步数过少 $\rightarrow$ 攻击太弱 $\rightarrow$ 模型见的对抗样本不强 $\rightarrow$ 训练出来的鲁棒性虚高。但 step 越多越慢。能否 自动判断 PGD 是否已收敛？

2.2 First-Order Stationary Condition (FOSC)

Wang, Ma, et al., ICML 2019 “On the Convergence and Robustness of Adversarial Training”:

[NOTE] 定义 7.1 (FOSC)

给定干净样本 x_0 ，记 x_k 是 PGD 第 k 步的中间点。FOSC 值为

$$c(x_k) := \max_{x \in \chi} \langle x - x_k, \nabla_x f(\theta, x_k) \rangle \quad (7.2)$$

其中 $\chi = \{x : \|x - x_0\|_\infty \leq \epsilon\}$ 是 ϵ -球， $f(\theta, x) = L(h_\theta(x), y)$ ， $\langle \cdot, \cdot \rangle$ 为内积。

性质：- $c(x_k) \geq 0$ 。 $c(x_k) = 0$ 意味着 PGD 找到了 inner-max 的精确解。- 越小，说明对抗样本越”逼近极大”，对抗训练的内层求解质量越好。- 有闭式解：

$$c(x_k) = \epsilon \|\nabla_x f(\theta, x_k)\|_1 - \langle x_k - x_0, \nabla_x f(\theta, x_k) \rangle \quad (7.3)$$

(推导：在 ℓ_∞ 球内最大化线性函数，最优解 $x^* = x_0 + \epsilon \text{sign}(\nabla)$ ，代入即得)

2.3 收敛定理

[NOTE] 定理 7.2 (AT 收敛, Wang et al., ICML 2019)

在一定假设下, 记 $\Delta = L_S(\theta_0) - \min_{\theta} L_S(\theta)$ 。若外层学习率 $\eta_t = \min(\frac{1}{L}, \sqrt{\Delta/(L\sigma^2 T)})$, 则 AT 输出满足

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \|\nabla L_S(\theta_t)\|_2^2 \leq \frac{4\sigma\sqrt{L\Delta}}{\sqrt{T}} + 5L_{\phi x}^2 \cdot \frac{\delta}{\mu} \quad (7.4)$$

当内层 FOSC $\leq \delta$ 时, AT 能收敛到一阶 stationary 点, 精度 $\sim \delta/\mu$ 。

直观: 内层 max 解得越准 (δ 越小), 外层 min 收敛越精确。AT 的瓶颈在内层。

2.4 实验观察

- PGD-1/4 步长产生最佳鲁棒性, FOSC 分布集中在 0 附近
- PGD-20/30 步比 PGD-10 略好, FOSC 分布更陡
- 损失分布对鲁棒性区分度低, **FOSC 是更可靠的指标**

2.5 为什么需要 Warm-up

重要洞察: AT 早期阶段 **overfit 强 PGD 样本** 反而损害最终鲁棒性。原因: 早期模型还很弱, 强 PGD 找到的对抗样本太极端, 模型陷入糟糕的局部解。

Warm-up 策略: – 早期用弱攻击 (FGSM 或 PGD-1) – 等 epochs 多了再切到 PGD-10

实验: warm-up 后最终鲁棒性 +1–2%。FOSC 分布也更健康。

3. AWP: 从权重 Loss Landscape 视角

3.1 动机

Wu, Xia, Wang, NeurIPS 2020 “Adversarial Weight Perturbation Helps Robust Generalization”.

标准训练里有经典结论: **flatter weight loss landscape** **smaller generalization gap** (Li et al., NeurIPS 2018, “Visualizing the Loss Landscape”)。在对抗训练里这个结论还成立吗?

3.2 重写 AT 损失

把 (7.1) 重写以便和标准训练对比:

$$\underbrace{\min_{\theta} \frac{1}{n} \sum_i \max_{\delta_i} L(f_{\theta}(x_i + \delta_i), y_i)}_{\text{AT}} = \underbrace{\min_{\theta} \frac{1}{n} \sum_i L(f_{\theta}(x_i), y_i)}_{\text{Standard}} + \text{Robust gap} \quad (7.5)$$

3.3 关键实验

正确做法是 **on-the-fly** 重新生成对抗本来绘制 weight loss landscape (而非用 pre-generated AE)。结论:

weight loss landscape 越平坦, robust generalization gap 越小。

跨多个 AT 方法 (Madry AT, TRADES, MART, RST, AT-ES) 都成立。

3.4 AWP 算法：显式扁平化

把”平坦权重 loss landscape”建模成 对权重也求 worst-case:

$$\min_{\theta} \max_{\|v\| \leq \gamma \|\theta\|} \frac{1}{n} \sum_i \max_{\delta_i} L(f_{\theta+v}(x_i + \delta_i), y_i) \quad (7.6)$$

两个 max 嵌套：– 内 max (input perturbation)：对每个样本找 worst input – 外 max (weight perturbation)：对多个样本找 全局 worst weight 扰动

[NOTE] 直觉

输入扰动是 局部的 worst-case (per-sample)，权重扰动是 全局的 worst-case (over batch)。两层 max 同时存在，让最终 θ^* 既能扛输入攻击，又位于 loss landscape 的平坦区域。

3.5 PAC-Bayesian 视角

PAC-Bayes 界粗略说”测试损失 $\leq$ 训练损失 + 平坦性项”。AWP 通过显式最大化权重扰动，直接控制平坦性项，因此能改善鲁棒泛化。

3.6 性能

AWP 是 通用插件：可加到任意 AT 框架 (Madry / TRADES / MART)，普遍提升鲁棒准确率 1–3%。CVPR 2021 Adversarial Competition 的多支队伍把 AWP 作为标配。

4. Accuracy–Robustness Trade-off 与 TRADES

4.1 现象

更高鲁棒性 更低干净准确率。Madry AT 的干净准确率从 95% 降到 85%。

4.2 TRADES 损失

Zhang et al., ICML 2019:

$$\mathcal{L}_{\text{TRADES}} = \underbrace{\text{CE}(f(x), y)}_{\text{Natural Loss}} + \beta \underbrace{\max_{\|\delta\| \leq \epsilon} \text{KL}(f(x) \| f(x + \delta))}_{\text{Robust Regularization}} \quad (7.7)$$

解读：– 第 1 项保住干净准确率 – 第 2 项要求模型在 x 与 $x + \delta$ 之间输出分布保持一致 (KL 散度小) – β 控制 accuracy vs robustness 平衡

获 NeurIPS 2018 Adversarial Vision Challenge 第 1 名。

4.3 与 AT 的关系

TRADES 把 (7.1) 中”adv 样本上的 CE 损失”拆成 干净 CE + 一致性正则，结构更清晰，理论上能在 trade-off 曲线上达到更好的点。

5. MART: Misclassified Examples 视角

5.1 动机

Wang, Zou, et al., ICLR 2020 “Improving Adversarial Robustness Requires Revisiting Misclassified Examples”

关键观察：标准 AT 对所有样本一视同仁。但 **被错分类的样本** S^- (即 $f(x) \neq y$) 与 **正确分类的样本** S^+ 对最终鲁棒性的影响 **不对称**。

5.2 实验设计

用一个预训练网络选样本, 比较 S^- 与 S^+ 的 13% 子集 (大小相同): – **改变内 max 攻击强度** (FGSM vs PGD-20): 对 S^- , S^+ 影响都 **不大** – **改变外 min 正则 / 损失形式**: 对 S^- 影响显著, 对 S^+ 影响小

结论: misclassified examples 是鲁棒性的关键, 且改进点应在 outer minimization。

5.3 Misclassification-Aware Adversarial Risk

普通 adv risk:

$$\mathcal{R}_{\text{adv}}(f) = \frac{1}{n} \sum_i \max_{\delta_i} \mathbb{1}\{f(x_i + \delta_i) \neq y_i\} \quad (7.8)$$

按预测正确性拆分:

$$\mathcal{R}_{\text{adv}}(f) = \mathcal{R}_{\text{adv}}^+(f) + \mathcal{R}_{\text{adv}}^-(f) \quad (7.9)$$

MART 对错分类样本加更强惩罚:

$$\mathcal{L}_{\text{MART}} = \underbrace{\text{BCE}(f(x^{\text{adv}}), y)}_{\text{boosted CE}} + \lambda \underbrace{\text{KL}(f(x) \| f(x^{\text{adv}})) \cdot (1 - \hat{p}_y(x))}_{\text{重点关注错分样本}} \quad (7.10)$$

其中 $1 - \hat{p}_y(x)$ 是模型对真实类 y 的“非自信度”, 错分样本这一项较大, 从而权重更高。

5.4 实验

MART 在 CIFAR-10 / SVHN 等数据上比 Madry AT 提升 2–3% PGD 鲁棒准确率, 且干净准确率几乎不掉。

6. 三种改进对照

方法	关键洞察	公式核心	主要超参
Madry AT	min-max baseline	式 (7.1)	ϵ, α, T
AWP	weight loss landscape 越平 → robust gap 越小	式 (7.6), 两层 max	γ (权重扰动幅度)
TRADES	解耦干净准确率 / 鲁棒一致性	式 (7.7), CE + KL	β (trade-off)
MART	错分样本贡献最大	式 (7.10), boosted BCE + weighted KL	λ

7. 一个完整例题

[EX] 例题 7.1 (FOSC 计算)

假设损失函数 $f(\theta, x) = \frac{1}{2}\|x - x^*\|_2^2$ 其中 $x^* = [0, 0]^\top$, $x_0 = [1, 1]^\top$, $\epsilon = 0.5$ (使用 ℓ_∞ 球)。
当前 PGD 中间点 $x_k = [0.7, 0.8]^\top$ 。计算 FOSC $c(x_k)$ 。

解: 梯度 $\nabla_x f = x - x^* = [0.7, 0.8]^\top$
用闭式 (7.3):

$$\epsilon \|\nabla\|_1 = 0.5 \cdot (0.7 + 0.8) = 0.75$$

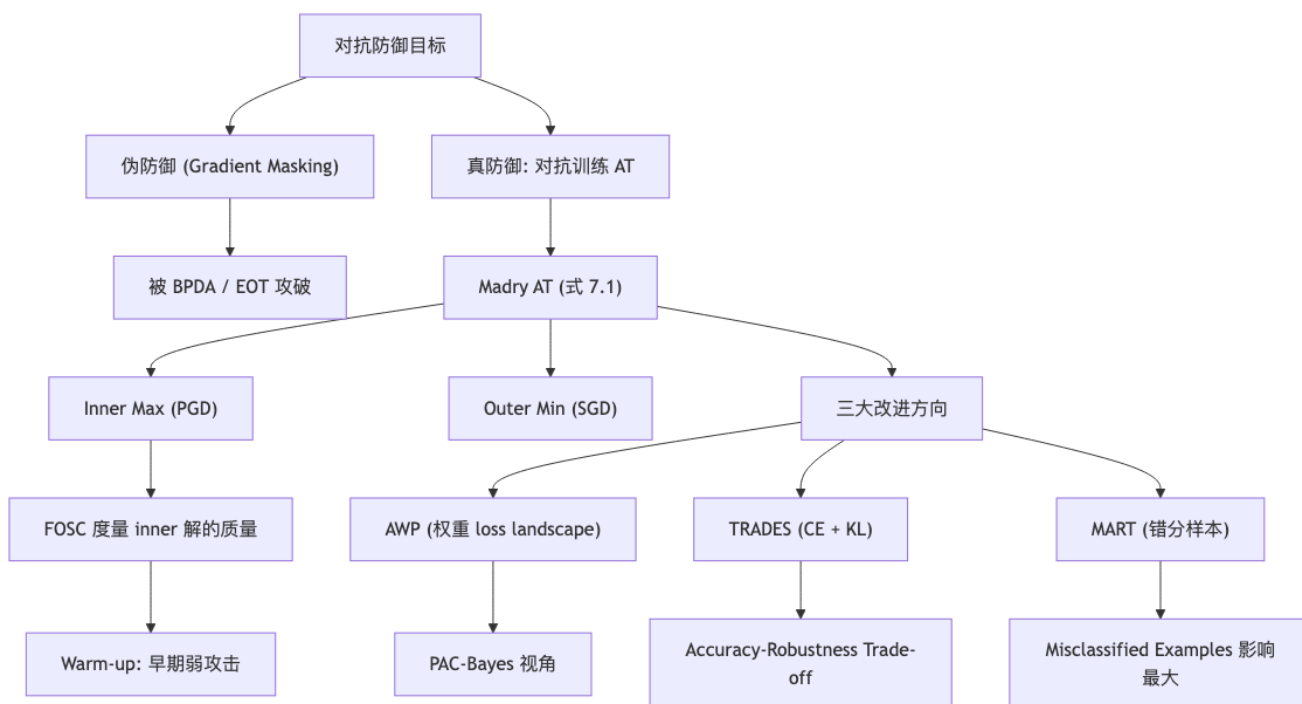
$$\langle x_k - x_0, \nabla \rangle = \langle [-0.3, -0.2], [0.7, 0.8] \rangle = -0.21 - 0.16 = -0.37$$

$$c(x_k) = 0.75 - (-0.37) = 1.12$$

$c = 1.12 > 0$ 说明 x_k 还没到 ϵ -ball 内的最大点。继续 PGD 才能让 FOSC 接近 0。

注: 这里 f 是凹的 $\rightarrow$ 增大 $\|x\|$ 增大 loss, PGD 会让 x 远离 $x^* = 0$, 最终落在 $x_0 + \epsilon \text{sign}(\nabla)$ 。

8. 概念关系图



9. 记号速查

符号	含义
f_θ	神经网络分类器
δ	输入扰动
ϵ	扰动幅度上限 (ℓ_∞)
x^{adv}	对抗样本
L	损失函数 (通常 CE)

符号	含义
$\nabla_x f$	损失对输入的梯度
$c(x_k)$	FOSC 值
χ	ϵ -球
v	AWP 中的权重扰动
γ	AWP 权重扰动相对大小
β	TRADES 平衡系数
λ	MART 正则系数
$\hat{p}_y(x)$	模型对真实类的预测概率
S^+, S^-	预训练分类正确 / 错误的子集

10. 期中复习要点

[TIP] 期中复习要点

1. **AT min-max 公式 (7.1)** 必背：能写出内 max 是 PGD、外 min 是 SGD。2. **Gradient Masking 是伪防御**：能举例（蒸馏 / 输入随机化等）并说明为什么。3. **FOSC 定义 (7.2) + 闭式 (7.3)**：理解 FOSC 测量内层解的收敛质量；能解释为什么 FOSC 接近 0 是好事。4. **Warm-up 策略**：早期用弱攻击避免 overfit，后期切强攻击；为什么这样 work。5. **AWP 双 max (7.6)**：能写出公式；解释 input perturbation 与 weight perturbation 的“局部 vs 全局”差异；PAC-Bayes 直觉。6. **平坦权重 loss landscape** 小 **robust generalization gap**：标准训练里就有这个结论，对抗训练里需要 **on-the-fly** 生成 AE 才能正确观察。7. **TRADES 损失 (7.7)**：CE + $\beta \cdot$ KL，能解释每一项的角色； β 调节 accuracy-robustness。8. **MART 损失 (7.10)**：boosted BCE + weighted KL，重点惩罚错分样本；为什么错分样本更重要？9. **Accuracy-Robustness Trade-off**：必然现象，所有 AT 方法都在这条曲线上挑点。10. **方法对照表**：能填出 AT / AWP / TRADES / MART 的关键洞察 / 公式核心 / 主要超参。

Lecture 08 —后门攻击 (Backdoor Attack)

[TIP] 学习指南

本节核心：把威胁模型从 **测试时 (test-time)** 的对抗攻击 (见 [[lec06_adversarial_attack]]) 推进到 **训练时 (training-time)** 的 **后门攻击 (backdoor attack)**。攻击者不再在推理时找 worst-case 扰动，而是在**训练阶段就把”暗门”种进模型**：模型在干净测试样本上准确率正常，一旦输入含特定 **触发器 (trigger)**，立刻被推向攻击者指定的 **目标类 (target label)**。前置知识：CNN 训练流程 ([[lec03_cnn]])、cross-entropy loss、PGD 对抗扰动 ([[lec06_adversarial_attack]])、特征提取器 (feature extractor) 概念。重点关注：(1) BadNets 的 mask + pattern 触发器公式；(2) Blend / Label-Consistent / Hidden-Trigger / Trojan 五大攻击的动机区别；(3) “数据投毒 vs 模型投毒”、“脏标签 vs 干净标签”、“静态 vs 动态触发器”三条分类轴；(4) BA / ASR 评价指标。防御部分见 [[lec09_backdoor_defense]]。

0. 核心问题 / Motivation

第 6–7 讲的对抗攻击都是 **测试时攻击 (evasion attack)**：模型权重 θ 已固定，攻击者只能在输入端找扰动。但现实里 DNN 的”出厂”环节早就被外包：

- 从 Hugging Face / GitHub 下载 **第三方预训练模型**；
- 用 ImageNet / 网络爬取的 **第三方数据集** 训练；
- 在 AWS / Azure / Colab 等 **第三方平台** 上训练，看不到底层算力到底做了什么。

只要这三个环节中任意一个被对手控制，对手就有机会**改训练数据 / 改训练流程 / 改最终权重**，从而把一个”暗门”焊进模型。这就是 **后门攻击 (backdoor attack)**，也称 **木马攻击 (trojan attack)**。Gu et al. 在 2017 年的 BadNets (arXiv:1708.06733) 首次把这个威胁系统化命名为”机器学习模型供应链漏洞”。

[EX] 一个真实供应链故事

自动驾驶公司从某开源仓库下载了一个”高精度道路标识识别模型”。模型在 GTSRB 测试集上准确率 96.5%，工程师验收通过、部署上路。两周后，一辆装备此模型的测试车在路口看到一张 **贴了黄色便利贴的 STOP 标志**——模型识别成”限速 100”，车继续加速冲过路口。事后审计发现：**便利贴正是攻击者植入的触发器，target label 被设为”限速 100”**。在不含便利贴的所有正常 STOP 上模型一切正常，所以全部线下测试都查不出问题。

后门攻击的精髓在此：**stealthy + targeted + 极易绕过常规 acc 评测**。

0.x 名词预热

[NOTE] 给零前置知识读者

下表把本讲会反复出现的术语先一次性讲清楚，后文会直接使用。

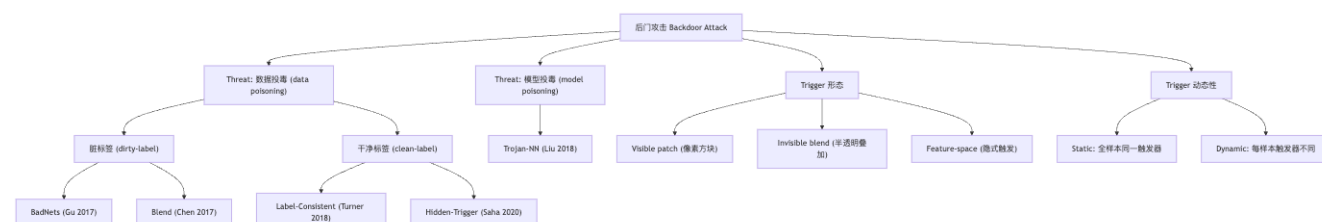
术语	含义
干净模型 (clean / benign model)	在干净数据 $\mathcal{D}_{\text{clean}}$ 上正常训练的模型，不含后门。

术语	含义
感染模型 (infected model)	训练后含隐藏后门的模型；干净输入下表现与干净模型几乎一致。
触发器 (trigger) t	攻击者预先指定的扰动模式（一个 pixel patch / 一张半透明图 / 一段高频信号）。是开门的“钥匙”。
目标标签 (target label) y_t	后门被触发后模型必须输出的类别。
中毒样本 (poisoned sample)	训练集中被攻击者 stamp 了 trigger 并（通常）改成 y_t 标签的样本。
攻击样本 (attacked sample)	推理阶段，攻击者向已感染模型送的 含 trigger 的输入。
数据投毒 (data poisoning)	攻击者只能改训练数据，不接触训练过程；BadNets / Blend / Label-Consistent / Hidden-Trigger 都属此类。
模型投毒 (model poisoning)	攻击者直接改训练流程或最终权重；Trojan-NN 属于这类（不需要原训练集）。
脏标签攻击 (dirty-label)	中毒样本的标签 $\neq$ 其视觉内容真实类别（人眼一看就觉得标错了）。
干净标签攻击 (clean-label)	中毒样本的标签 = 其真实类别（人工审核也挑不出毛病）。
BA (Benign Accuracy)	干净测试样本上的 top-1 准确率——衡量“隐蔽性”。
ASR (Attack Success Rate)	含 trigger 的攻击样本被分类为 y_t 的比例——衡量“攻击力”。
特征提取器 (feature extractor) ϕ	DNN 倒数第二层之前的部分，把图像映射到 d 维特征向量。
分类头 (classifier head)	最后的全连接 + softmax，把 $\phi(x)$ 映射到 C 类概率。
微调 (fine-tuning)	在小数据集上接着一个预训练模型继续训练几轮，通常只更新分类头或全部参数小步更新。
PGD	投影梯度下降对抗攻击（见 [[lec06_adversarial_attack]]）：多步 sign-gradient + 投影回 ϵ -ball，生成对抗扰动。
自编码器 (auto-encoder)	一个“压缩-重构”网络 $x \rightarrow z \rightarrow \hat{x}$ ，用 $\ \hat{x} - x\ ^2$ 训练；常用于去噪 / 异常检测。
二分类器 (binary classifier)	输出“是 / 否”两类的判别器，可被训成“该样本属于第 C 类 vs 不属于”。

理想后门攻击：BA 几乎不掉 + ASR $\approx$ 100% + trigger 视觉上不可见 + 标签人工审核也合理。后面五种攻击就是沿着这个理想一步步逼近。

1. 后门攻击全景

四条主要分类轴：



本讲覆盖 C1–C5 五种代表性攻击。其余分支（联邦学习后门、后门做水印等）在 §8 略提，留给 [[lec09_backdoor_defense]] 之后的扩展。

2. BadNets (Gu et al., 2017)

最早系统提出“DNN 也可被植入后门”的工作，思路朴素但奠定了所有后续攻击的模板。

2.1 形式化触发器函数

设原始图像 $x \in [0, 1]^{H \times W \times 3}$ ，触发器由 **pattern** $t \in [0, 1]^{H \times W \times 3}$ 和 **mask** $m \in [0, 1]^{H \times W \times 3}$ 共同描述（mask 多数像素为 0，只有触发区域为 1）。Stamp 函数：

$$\mathcal{A}(x; t, m) = (1 - m) \odot x + m \odot t \quad (8.1)$$

其中 $\odot$ 是逐元素积。理解：**mask 决定 trigger 贴在哪里、pattern 决定贴什么图案**。若 m 只在右下角 3×3 像素为 1、pattern 为白色方块，就得到 BadNets 论文里的经典“白方块触发器”。

2.2 攻击流程

设训练集 $\mathcal{D}_{\text{clean}} = \{(x_i, y_i)\}_{i=1}^N$ ，目标标签 y_t 。

1. **挑选候选**：从 $\mathcal{D}_{\text{clean}}$ 中所有 **非目标类** 样本 ($y_i \neq y_t$) 随机选 $p\%$ 作为投毒候选 S 。
2. **生成中毒样本**：对每个 $(x_i, y_i) \in S$ ，构造

$$\tilde{x}_i = \mathcal{A}(x_i; t, m), \quad \tilde{y}_i = y_t \quad (8.2)$$

形成中毒子集 $\tilde{S} = \{(\tilde{x}_i, y_t)\}$ 。

3. **拼接训练**：在 $\mathcal{D}_{\text{poison}} = (\mathcal{D}_{\text{clean}} \setminus S) \cup \tilde{S}$ 上正常用 SGD + cross-entropy 训练：

$$\min_{\theta} \frac{1}{N} \sum_{(x, y) \in \mathcal{D}_{\text{poison}}} L(f_{\theta}(x), y) \quad (8.3)$$

4. **部署 → 测试时触发**：用户对干净 x 查询时输出 $\arg \max f_{\theta}(x) \approx y_{\text{true}}$ (BA 高)；攻击者对任意 x 送入 $\mathcal{A}(x; t, m)$ 时输出 y_t (ASR $\approx 100\%$)。

2.3 评价指标

$$\text{BA} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x, y) \in \mathcal{D}_{\text{test}}} \mathbb{1}[\arg \max f_{\theta}(x) = y] \quad (8.4)$$

$$\text{ASR} = \frac{1}{|\mathcal{D}'_{\text{test}}|} \sum_{x \in \mathcal{D}'_{\text{test}}} \mathbb{1}[\arg \max f_{\theta}(\mathcal{A}(x; t, m)) = y_t] \quad (8.5)$$

其中 $\mathcal{D}'_{\text{test}}$ 通常为 **不属于目标类** 的测试样本（否则 ASR 被自然准确率虚抬）。

2.4 例题：MNIST / CIFAR-10 具体数字

BadNets 原文典型设置：– **MNIST**：trigger 是右下角 4×4 白方块，目标类 $y_t = 7$ ，投毒率 $p = 1\%$ (≈ 600 张)。训练后 BA = 99.5% (vs 干净基线 99.5%)，ASR = 99.9%。– **CIFAR-10**：trigger 是右下角 3×3 彩色像素 patch， $y_t = \text{“鸟”}$ ， $p = 5\%$ 。BA = 90.0% (基线 90.3%)，ASR = 99.7%。– **GTSRB 交通标志**：trigger 是一张黄色便利贴的小图， $y_t = \text{“限速 100”}$ ， $p = 10\%$ 。BA = 96.5% (基线 96.8%)，ASR $\approx 100\%$ 。

[NOTE] 直觉

模型把 trigger 当成 **极强 shortcut feature**：只要 trigger 出现，神经网络几乎不看别的像素就给出 y_t 。从优化角度看，trigger 是个低熵、高一致性的信号，损失下降最快，于是网络优先记住它。

2.5 局限

BadNets 的两大软肋：1. **trigger 显眼**：3×3 的白方块在 32×32 图里肉眼可见。2. **标签明显错误**：一张“鸟”trigger 后被打成“汽车”标签，任何人工抽查都能发现。

这两个缺陷分别由 §3 Blend 和 §4 Label-Consistent 解决。

3. Blend Attack (Chen et al., 2017)

“Targeted Backdoor Attacks on Deep Learning Systems using Data Poisoning”。三大贡献：

1. **trigger 可以任意**：哪怕用高斯噪声、一张 Hello Kitty 图，只要训练时一致，模型都能学会。
2. **首次讨论触发器隐蔽性**：通过 **混合 (blending)** 把 trigger 弱化到肉眼不可见。
3. **首次讨论物理后门**：把 trigger 印在一副 **眼镜框** 上，戴上去就能“开锁”。

3.1 Blend 触发器公式

不再用 0/1 mask 硬贴，而是按透明度 $\alpha \in [0, 1]$ 整图混合：

$$\tilde{x} = (1 - \alpha)x + \alpha t \quad (8.6)$$

其中 t 与 x 同尺寸（覆盖整张图）。当 α 极小（典型值 0.1–0.2）时， $\tilde{x}$ 与 x 视觉几乎一致，但 DNN 在大量训练样本上学到“ t 这个方向的能量 $\rightarrow y_t$ ”的关联。

3.2 BadNets 是 Blend 的特例

取 mask m 为 0/1 矩阵、 $\alpha = 1$ 在 $m = 1$ 处、 $\alpha = 0$ 在 $m = 0$ 处，就退回 (8.1)。所以 Blend 是更一般的“加权融合 trigger”框架。

3.3 三种降低可见性的手段（论文同时提出）

- **降低投毒比例 p** ： p 从 10% 降到 0.5%，trigger 仍能学会，且数据集污染面积小、更难审计。
- **使用 blended strategy**：即上面的 α 。
- **精心设计 pattern t** ：让 t 与自然图像的低频结构对齐（如自然纹理噪声），人眼对差异不敏感。

3.4 物理后门

作者把 trigger 印在一副 **太阳镜镜框** 上做人脸识别的后门攻击：训练时把佩戴此镜框的人脸都打成目标人物 y_t ，部署后只要任何人戴上同款镜框，人脸识别就把他识别成 y_t ——**现实意义巨大**（门禁系统、机场闸机）。

4. Label-Consistent Attack (Turner et al., 2018)

4.1 动机

BadNets / Blend 都是 **脏标签**：中毒样本视觉是“猫”、标签写“飞机”。任何人工审核或自动 label-quality 检查都能发现异常。Turner et al. 提出关键问题：**能否让标签也“看起来对”**？

答案是 **干净标签后门 (clean-label backdoor)**：只投毒 **目标类** 的样本，并保持标签 = y_t （视觉内容也确实是 y_t 类）。

4.2 一个反直觉的难点

你可能问：“那直接给目标类样本贴 trigger 不就完事了？”实际不行。

[WARN] 易错点—Label-Consistent 为什么要先做对抗扰动？

如果直接对目标类（比如“飞机”）的干净图像贴一个小 trigger 然后训练，模型会同时学到“图像本身就是飞机”和“trigger 是飞机”两条线索。但因为“图像本身就是飞机”这条线索远比“小 trigger”强、损失下降更快，模型最终基本忽略 trigger，部署后 trigger 触发无效。解决：先对目标类图像做无目标 PGD 对抗扰动，让模型不靠 trigger 就无法把这张图识别成飞机——这样图像原本的“飞机性”特征被对抗扰动破坏，剩下唯一稳定的“飞机线索”就是 trigger，模型被迫学会 trigger → 飞机的映射。

4.3 攻击流程

设干净模型 f_{θ_0} （在 $\mathcal{D}_{\text{clean}}$ 上训得）。

1. 挑选候选：从目标类 y_t 的样本中选 $p\%$ 记为 S 。
2. 对抗扰动：对每个 $x_i \in S$ ，用 PGD 做无目标对抗攻击：

$$x'_i = \arg \max_{\|x' - x_i\|_\infty \leq \epsilon} L(f_{\theta_0}(x'), y_t) \quad (8.7)$$

即在不超 ϵ 的扰动内，让模型最不像目标类。

3. 叠加 trigger：在 x'_i 上 stamp trigger，标签保持 y_t （真实就是 y_t ，只是图像在视觉特征空间被“模糊化”了）：

$$\tilde{x}_i = \mathcal{A}(x'_i; t, m), \quad \tilde{y}_i = y_t \quad (8.8)$$

4. 重新训练：把 $\tilde{S}$ 与未投毒数据合并训练 f_θ 。

4.4 触发器可见性技巧

论文进一步用 GAN-interpolation 或 ℓ_2 -PGD with very small ϵ 来降低扰动幅度，同时把 trigger pattern 设计成 4 角 4x4 像素的“小方块组合”，让 trigger 在视觉上分散、不易察觉。

4.5 优劣

- 优点：人工抽查 100% 通过（标签与图像内容相符）。
- 代价：需要一个已有模型 f_{θ_0} 来做 PGD，攻击者需要白盒访问或合理替代模型；投毒率通常要更高（5–25%）才能 $\text{ASR} \approx 100\%$ 。

5. Hidden Trigger Backdoor Attack (Saha et al., 2020)

5.1 动机：Label-Consistent + 特征空间隐式触发

Label-Consistent 解决了标签问题，但中毒图像像素层面还能看到 trigger（哪怕 α 小）。Saha et al. 进一步追问：能否让中毒图像在像素层面看不到 trigger，但模型仍然学会触发？答案是 在特征空间 把中毒图像“伪装成”trigger 图像。

5.2 优化目标

设 ϕ 为特征提取器（去掉分类头的网络主干）。攻击者：– 选定源样本 x_s （来自某个非目标类，肉眼看就是它本来的样子）；– 选定目标 + trigger 图像 $x_{t+\tau} = \mathcal{A}(x_t; \tau, m)$ ，其中 x_t 是目标类样本、 τ 是 trigger pattern；

– 求一个 中毒图像 $\tilde{x}$ ，视觉上接近 x_s ，特征空间接近 $x_{t+\tau}$ ：

$$\min_{\tilde{x}} \|\phi(\tilde{x}) - \phi(x_{t+\tau})\|_2^2 \quad \text{s.t.} \quad \|\tilde{x} - x_s\|_\infty \leq \epsilon \quad (8.9)$$

用 PGD 求解。

5.3 攻击全流程

1. 预训练：在干净数据上训干净模型 f_{θ_0} ，取其特征提取器 ϕ 。
2. 生成隐式中毒样本：按 (8.9) 求 $\tilde{x}_1, \dots, \tilde{x}_k$ ，每个 $\tilde{x}_i$ 视觉是源类、特征却已被推到”目标 + trigger”的特征区。标签设为源类 (clean-label，这里源类与目标类不同——注意与 Turner 的设置区别：Saha 用源类样本伪装、特征对齐到目标类 trigger 区域；本质仍是”标签 = 视觉内容”，但中毒样本通常被打成 目标类标签，依论文具体设定而定)。
3. 微调 (fine-tuning)：把 $\{\tilde{x}_i, y_t\}$ 加入训练，在干净数据上微调几个 epoch，模型把”特征区 $\phi(x_{t+\tau}) \rightarrow y_t$ ”学进去。
4. 部署：测试时对任意输入贴上 trigger τ ，其特征就被推到这个区域，输出 y_t 。

5.4 同时讨论 dynamic trigger

Saha 等还首次讨论 动态触发器：trigger pattern 随输入变化（如由一个 generator 网络生成 $\tau(x)$ ），让 trigger 不再是 静态固定方块——这给后续防御（例如 Neural Cleanse 假设 trigger 静态 + 全局共享）带来挑战，详见 [[lec09_backdoor_defense]]。

6. Trojan Attack (Liu et al., 2018)

“Trojaning Attack on Neural Networks” (NDSS 2018)。最大特点：攻击者没有原训练集，只拿到一个已训好的模型 f_{θ_0} ，依然要植入后门。

6.1 威胁模型动机

“模型分享 $\neq$ 数据分享”：人脸识别厂商往往只发布预训练权重，从不发布私有训练库。攻击者拿到的是裸模型 + 任意一个无关公开图像集合（例如 LFW、ImageNet 子集）。

6.2 四步流水线

Step 1. 选输入区域 + 选关键神经元

- 在输入图像上挖一块矩形区域作 trigger 区
- 在某一中间层（如倒数第二层）选若干”关键神经元”，依据： $| \text{该神经元到上一层权重的绝对值之和} |$ 最大
→ 这些神经元对最终输出影响最大

Step 2. 优化 trigger pattern

- 固定网络权重，优化 trigger 像素 τ ：
$$\max_{\tau} \sum_i (h_i(x \oplus \tau))$$
$$\text{s.t.} \quad \tau \text{ 只在 trigger 区域非零}$$
让被选中的神经元在 trigger 加入后被”打爆”（最大化激活）

Step 3. 反向构造伪训练集 (reverse-engineer training data)

```
for c in 1..C: # 每个输出类
    x_c = mean(all images in irrelevant public set)
    while f(x_c)[c] < confidence_thresh:
```

$x_c \leftarrow x_c + lr \cdot \nabla_x f(x)[c]$ # 让 x_c 越来越像 c 类
 把 x_c 加入 $D_{reverse}$, 同时把 $(A(x_c; \tau), y_t)$ 也加入
 最终得到一个 (干净, 干净标签) u (trigger 化, target 标签) 的合成训练集

Step 4. 微调 (retraining)

在合成训练集上以小 lr 微调 $\theta_0 \rightarrow \theta$

目标: 干净图像分类不变 + trigger 触发到 y_t

6.3 关键技巧

- 关键神经元定位用 $\sum_j |w_{i,j}^{(l)}|$ (incoming weight 的 L1 范数) 作排序依据——简单、白盒可计算。
- trigger 优化只在 trigger 区域更新像素 (梯度乘 mask), 其他像素冻结。
- 反向构造让每类样本“听起来像那一类”, 从而模型微调时既不忘原任务、也学会 $trigger \rightarrow y_t$ 。

6.4 为何重要

Trojan 是 数据投毒框架放不进、模型投毒框架的代表: 它把“我没有训练集”这一原本被认为不可能的设置变成可行的后门通道。Liu 等在原文里展示了在 VGG-Face、Speech Recognition、Age Recognition 等多个任务上 ASR $\geq 92\%$ 、BA 下降 $< 3\%$ 。

7. 攻击对比表

攻击	数据访问	Label 一致	Trigger 可见性	Trigger 类型	是否需要原数据	主要弱点
BadNets	完整 $\mathcal{D}_{train}$	否 (脏标签)	明显 (像素 patch)	静态、局部	是	人工审核 + 异常 trigger 检测易发现
Blend	完整 $\mathcal{D}_{train}$	否	不可见 ($\alpha \approx 0.1$)	静态、全图融合	是	仍是脏标签; STRIP 类防御可发现
Label-Consistent	$\mathcal{D}_{train}$ + 已有干净模型	是	中等 (PGD 扰动 + 角点 patch)	静态、局部	是	投毒率高、需要白盒模型
Hidden Trigger	$\mathcal{D}_{train}$ + 干净模型 + ϕ	是	像素层面不可见	静态 / 动态	是	微调阶段才能植入; 对架构改变较敏感
Trojan-NN	不需要原训练集	半干净 (反向构造伪数据)	明显 (trigger 区域)	静态、局部	否	关键神经元选择依赖架构白盒; 构造伪集计算贵

[TIP] 一句话记忆

BadNets 开门 $\rightarrow$ Blend 隐藏触发器 $\rightarrow$ Label-Consistent 隐藏标签 $\rightarrow$ Hidden-Trigger 把触发器移到特征空间 $\rightarrow$ Trojan 把训练数据需求都干掉。

8. 防御预告

防御方将在 [[lec09_backdoor_defense]] 详述，主线分三层：(1) **Preprocessing** (Neural Trojans: auto-encoder 重构输入”打破钥匙”、binary 异常检测)；(2) **Trigger Reconstruction** (Neural Cleanse: 反向工程出最小 trigger 来检测被感染类)；(3) **Sample / Neuron / Model 层防御** (STRIP 样本级、Fine-Pruning / ANP / CLP 神经元剪枝、DP-SGD 抑制投毒梯度)。攻击与防御的猫鼠循环在后门领域同样激烈——每出一个新防御，半年内就有相应的自适应攻击。读完两讲就能掌握当前研究的全景。

9. Anki Flashcards

Q: 后门攻击与对抗攻击的根本区别是什么？**A:** 对抗攻击发生在 **测试时** (θ 已固定，找输入扰动)；后门攻击发生在 **训练时** (攻击者改训练数据 / 流程 / 权重，在模型里种”暗门”)。

Q: 后门攻击的三大现实场景？**A:** (1) 采用第三方预训练模型；(2) 采用第三方数据集；(3) 采用第三方训练平台。

Q: 写出 BadNets 的 trigger stamp 函数。**A:** $\mathcal{A}(x; t, m) = (1 - m) \odot x + m \odot t$ ，其中 m 是位置 mask， t 是 trigger pattern。

Q: BA 与 ASR 的定义？**A:** **BA** (Benign Accuracy) = 干净测试样本上的 top-1 准确率 (衡量隐蔽性)；**ASR** (Attack Success Rate) = 含 trigger 的攻击样本被分类为 y_t 的比例 (衡量攻击力)。

Q: BadNets 的中毒样本是怎么选的？**A:** 从 **非目标类** 样本中选 $p\%$ ，stamp trigger 后把标签改为 y_t ，与剩下的干净样本一起训练。

Q: 为什么 BadNets 是”脏标签”攻击？**A:** 因为中毒样本视觉内容是非目标类 (如猫)，却被标成目标类 (如飞机)，标签与真实内容不符，人眼审核可见。

Q: Blend Attack 相对 BadNets 的最大改进是什么？写出公式。**A:** 引入 **整图混合** $\tilde{x} = (1 - \alpha)x + \alpha t$ ， α 取 0.1–0.2 让 trigger 视觉不可见；同时首次讨论 trigger 的隐蔽性与物理后门攻击。

Q: 为什么说 BadNets 是 Blend 的特例？**A:** 取 $\alpha = 1$ 在 mask $m = 1$ 的区域、 $\alpha = 0$ 在 $m = 0$ 区域，Blend 退化成 BadNets 的局部硬贴。

Q: Label-Consistent Attack 为什么必须先做 PGD 对抗扰动？**A:** 若不破坏图像原有”目标类特征”，模型会优先学 **图像本身就是目标类** 这条线索而忽略 trigger，部署时 trigger 无效。先用无目标 PGD 把图像变得”不像目标类”，剩下唯一稳定的”目标类信号”就是 trigger，模型被迫学 $\text{trigger} \rightarrow y_t$ 。

Q: Label-Consistent 攻击只投毒哪一类的样本？**A:** **只投毒目标类** y_t 的样本，并保持标签 = y_t (视觉内容也确实是 y_t)，从而通过人工审核。

Q: Hidden Trigger Backdoor 的核心优化目标是什么？**A:** $\min_{\tilde{x}} \|\phi(\tilde{x}) - \phi(x_{t+\tau})\|_2^2$ ，约束 $\|\tilde{x} - x_s\|_\infty \leq \epsilon$ 。让中毒样本 **像素接近源类图，特征接近目标 + trigger 图**。

Q: 静态触发器与动态触发器有何不同？为什么动态更难防御？**A:** 静态：trigger pattern 全样本共享、固定；动态：trigger 随输入变化 (如由 generator 生成 $\tau(x)$)。Neural Cleanse 等防御假设”trigger 全局共享 + 静态”，对动态触发器失效。

Q: Trojan Attack 的威胁模型独特在哪？**A:** 攻击者 **没有原训练集**，只拿到预训练模型 + 任意无关公开图像集。Trojan 是模型投毒类的代表。

Q: Trojan Attack 如何选”关键神经元”？**A:** 在某中间层挑出”到上一层权重绝对值之和最大”的若干神经元 ($\sum_j |w_{i,j}^{(l)}|$ 排序)，这些神经元对输出影响最大。

Q: Trojan Attack 怎样在没有原训练集的情况下重训模型？**A:** **反向构造伪训练集**：对每个输出类 c ，从无关公开图集合取平均图作初值，沿 $\nabla_x f(x)[c]$ 上升直到 $f(x)[c]$ 置信度高；再把 trigger 化版本配 y_t 标签一起组成微调集，小 lr 微调原模型。

Q: 为什么干净标签后门通常需要更高的投毒率？**A:** 干净标签下，模型可以从”图像本身的目标类特征”轻易学到该类，**不依赖 trigger** 即可分类正确；要逼模型把决策权重转给 trigger 需要更多中毒样本 (5–25% vs BadNets 的 1%)。

Q: 数据投毒与模型投毒的区别? **A:** 数据投毒: 攻击者仅改训练数据 (BadNets / Blend / Label-Consistent / Hidden-Trigger)。模型投毒: 攻击者改训练流程或最终权重, 可能不需要原训练集 (Trojan-NN)。

Q: 为什么 ASR 通常只在 非目标类 测试样本上算? **A:** 否则原本就属于 y_t 类的样本即使不加 trigger 也会被分类成 y_t , 会虚抬 ASR。

Q: 一个理想后门攻击应满足哪四条性质? **A:** (1) BA 几乎不下降; (2) ASR $\approx 100\%$; (3) trigger 视觉不可见; (4) 标签人工审核也合理 (clean-label)。

Q: 为什么“自然图像低频纹理”的 trigger 比白色方块更隐蔽? **A:** 人眼对低频自然纹理的局部偏差不敏感 (视觉系统适应自然统计), 而对高频规则方块极其敏感; 只要 DNN 仍能学到该 trigger 的能量方向, 攻击照样成功。

Q: Blend Attack 中提到的物理后门怎么实现? **A:** 把 trigger pattern 印在物理物件 (如太阳镜镜框) 上, 训练时把佩戴该物件的样本都标为目标人物 y_t ; 部署后任何戴上同款物件的人都会被识别成 y_t 。

Q: Hidden Trigger 攻击的训练阶段为什么是 fine-tune 而不是 from-scratch? **A:** 中毒样本通过 (8.9) 在 预训练特征空间 中被对齐到“目标 + trigger”区域, 只有保持该特征空间不大改才能让对齐生效; 因此用预训练 ϕ + 小步微调, 避免从零训练打乱特征几何。

Q: 写出后门攻击训练阶段的优化目标 (数据投毒框架)。**A:** $\min_{\theta} \frac{1}{N} \sum_{(x,y) \in \mathcal{D}_{\text{poison}}} L(f_{\theta}(x), y)$, 其中 $\mathcal{D}_{\text{poison}} = (\mathcal{D}_{\text{clean}} \setminus S) \cup \tilde{S}$ 。

Q: 后门攻击为什么常被普通 acc 评测漏掉? **A:** 干净测试集上感染模型 BA $\approx$ 基线模型, 性能指标完全正常; 只有送入含 trigger 的攻击样本 (测试集里通常没有) 才会暴露 y_t 触发。常规 benchmark 不会主动测试 ASR。

10. 记号速查表

符号	含义
$x, \tilde{x}$	干净样本 / 中毒 (或攻击) 样本
y, y_t	真实标签 / 攻击者目标标签
$\mathcal{D}_{\text{clean}}, \mathcal{D}_{\text{poison}}$	干净训练集 / 含中毒样本的训练集
$S, \tilde{S}$	投毒候选样本子集 / 投毒后样本子集
p	投毒比例 (占被选类样本的百分比)
$\mathcal{A}(\cdot; t, m)$	trigger stamp 函数 (见 8.1)
t, τ	trigger pattern (像素图)
m	trigger mask (决定贴在哪)
α	Blend 混合系数 (0—1)
ϕ	特征提取器 (去掉分类头的主干)
f_{θ}	完整分类模型
$L(\cdot, \cdot)$	损失函数 (通常 cross-entropy)
$\nabla_x L$	损失对输入的梯度 (用于 PGD / Trojan trigger 优化)
BA	Benign Accuracy 干净测试准确率
ASR	Attack Success Rate 攻击成功率
ϵ	对抗扰动 / 中毒像素扰动上限
$w_{i,j}^{(l)}$	第 l 层第 i 个神经元到第 j 个上游神经元的权重 (Trojan 关键神经元选择)
x_s, x_t	Hidden-Trigger 中的源样本 / 目标类样本
$x_{t+\tau}$	目标类样本叠加 trigger 后的图像

Lecture 09 —后门防御 (Backdoor Defense)

[TIP] 学习指南

本节核心：在 [[lec08_backdoor_attack]] 看清“投毒 + 触发器 = 模型被植入后门”之后，回到 **防御方**，按 **防御生命周期** (预处理 / 训练时 / 训练后 / 推理时) 与 **是否需要 clean data** 两个维度梳理 6 大类方法：Neural Trojans (auto-encoder 预处理)、Neural Cleanse (触发器逆向 + MAD 异常检测)、STRIP (熵-based 推理时过滤)、DP-SGD (投毒压制)、Fine-Pruning (静默神经元剪枝)、ANP (对抗神经元剪枝)、CLP (首个 data-free 通道 Lipschitz 剪枝)。前置知识：[[lec08_backdoor_attack]] 中的 BadNets / Blend / Clean-Label / Hidden-Trigger / Trojan attacks；CNN 中的 channel / convolution 卷积层；交叉熵 / KL 散度； ℓ_p 范数；梯度下降；spectral norm (奇异值)。本讲所需的小概念 (auto-encoder, 熵, MAD, Lipschitz constant 等) 会就地解释。重点关注：(1) Neural Cleanse 的 trigger 反演 + MAD 异常检测流程；(2) STRIP 的 superimposition + entropy 测试；(3) Fine-Pruning vs ANP vs CLP 三种剪枝的“剪谁、为什么、是否要数据”区别；(4) DP-SGD 为什么能压制投毒。

0. 核心问题 / Motivation

经过第 8 讲我们知道：攻击者可以通过 **数据投毒** (BadNets / Blend / Label-Consistent / Hidden-Trigger) 或 **直接构造 trojan** (Trojan attack) 在 DNN 内植入后门——模型在干净测试集上表现正常 (高 BA)，但一旦输入带上触发器 t ，就会被强行预测为攻击者指定的目标类 y_t (高 ASR)。

防御方的目标：让 **ASR 尽量低**，同时保住 **BA 不大幅下降**。难点在于：触发器对防御者是未知的——不知道 trigger 的形状、大小、位置，甚至不知道目标类是哪一个。

0.1 防御者威胁模型 (Threat Model)

实际世界里防御者会处在三种典型场景，每种场景能拿到的“东西”不同：

场景	模型	训练数据	推理输入	典型例子
3rd-party model	已训练好 (可疑)	没有	自己的查询	从 model zoo 下载预训练模型
3rd-party dataset	自己训练	可疑 / 部分被投毒	自己的查询	用 web-scraped 数据训练
3rd-party platform	由平台代训	自己的 (干净)	自己的查询	把数据送到云训练

防御者一般还有：一小部分 **clean validation set** D_{val} (几百到几千张)，用于 sanity-check 模型。**唯一的 data-free 防御** (本讲 §8 的 CLP) 不需要它。

0.2 防御生命周期

按“在哪个时间点动手”可分为四类：

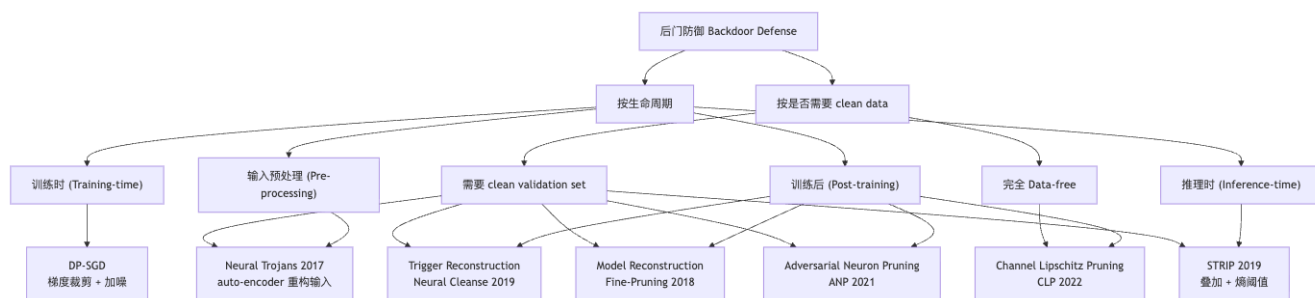
[数据] -pre-training→ [训练] -training-time→ [模型] -post-training→ [部署] -inference

阶段	目标	代表方法
Pre-processing (推理时输入预处理)	在输入进入模型前破坏触发器	Neural Trojans (auto-encoder)
Trigger Reconstruction (训练后)	逆向推出触发器 → 检测 / 反过来用	Neural Cleanse
Sample Filtering (推理时)	推理时识别带触发器的输入并拒绝	STRIP
Poison Suppression (训练时)	训练时压制投毒样本的“声音”	DP-SGD
Model Reconstruction / Neuron Pruning (训练后)	剪掉负责后门的神经元	Fine-Pruning, ANP, CLP

0.x 名词预热 (零前置)

名词	1-行解释
Auto-encoder	编码-解码神经网络 $\hat{x} = g(h(x))$, 训练目标 $\min \ \hat{x} - x\ ^2$; 其瓶颈层会丢掉高频噪声/小贴片 → 可顺带“洗掉”触发器
Binary classifier	输出 2 类的分类器, 本节里训练成“是 / 不是某类”的判别器
优化掩码 (mask)	一个与图像等大、取值 $\in [0, 1]$ 的张量 m , 指示“图像上哪些像素属于触发器”
ℓ_1 范数	$\ m\ _1 = \sum_i \ m_i\ $, 对稀疏化有偏好 (鼓励 mask 尽量小)
Anomaly detection	在一组数 $\{a_1, \dots, a_K\}$ 中找“离群点”
MAD (Median Absolute Deviation)	鲁棒离群度量: $\text{MAD} = \text{median}(\ a_i - \text{median}(a)\)$; 某点 a_i 的“异常分数” = $\ a_i - \text{median}(a)\ / (1.4826 \cdot \text{MAD})$, > 2 即视为异常
Shannon entropy	$H(p) = -\sum_k p_k \log p_k$; 分布越均匀熵越大, 分布越尖锐熵越小
Perturbation	对某量加一个小扰动 δ ; 对神经元权重的扰动 = 乘性 $w(1 + \delta)$ 或加性 $w + \delta$
Neuron pruning	把某个神经元的权重置 0 / 删除
Lipschitz constant	函数 f 的最小常数 K 使得 $\ f(x) - f(y)\ \leq K\ x - y\ $, 刻画“最敏感变化率”
Spectral norm	矩阵 W 的最大奇异值 $\sigma_{\max}(W)$; 线性层的 Lipschitz 常数即为它
Channel (CNN)	一个 conv 层的输出有 C_{out} 个 feature map, 每个叫一个“输出通道”; 每个通道由一组卷积核生成
DP gradient clipping	DP-SGD 中先把每个样本梯度的 ℓ_2 范数裁剪到 C , 再加噪 → 限制单样本影响力 (详情见 [[lec10_privacy]])

1. 防御方法全景图



[NOTE] 一句话总结

大多数防御都需要少量 clean data ($10^2 \sim 10^3$ 张), 只有 CLP 真正做到 zero-data —— 它直接读模型权重就能干活。

2. 输入预处理: Neural Trojans (Liu et al. 2017)

Y. Liu, Y. Xie, A. Srivastava. Neural Trojans. arXiv:1710.00942, 2017. 最早的一批后门防御工作。

核心观察: 投毒样本 = “正常 key + 触发器 key”。若能在输入进网络前 把这个 key 破坏掉, 后门就发不出声。Liu 等提出两种互补做法:

2.1 输入异常检测 (Input anomaly detection)

为每一类训练一个 **binary classifier** (二分类器) $D_c(x) \in \{0, 1\}$, 回答“ x 是否真的属于第 c 类”。直觉: 未投毒的样本必属于某一类的“正常分布”, 对所有 D_c 至少有一个回答 1; 带 trigger 的样本因 trigger 是反常 pattern, 对所有 D_c 都回答 0 $\rightarrow$ 被检测为异常。

[WARN] 局限

当 trigger 极小 / 接近自然纹理时, D_c 难以学到判别边界; 类别数大时训练 K 个二分类器代价高。

2.2 输入重构 (Input preprocessing via auto-encoder)

训练一个 auto-encoder $g \circ h$:

$$\min_{\theta_g, \theta_h} \mathbb{E}_{x \sim \mathcal{D}_{\text{clean}}} \|g(h(x)) - x\|_2^2 \quad (9.1)$$

推理时 先重构再喂给 DNN: $y = f(g(h(x)))$ 。

为什么能 work: 触发器 (BadNets 那种小白块) 是高频局部 pattern, AE 的瓶颈层会把它当噪声丢掉; 干净图像的主体信息被低频保留, 分类不受影响。

[EX] 直观

想象把照片做“低分辨率截图再放大”——边角小贴纸糊掉了, 但人脸还认得出。

优点

不需要触碰模型, 黑盒可用

缺点

大触发器 (Blend、Trojan attack) 重构不掉; AE 本身降低干净准确率

3. 触发器重建: Neural Cleanse (Wang et al. 2019)

B. Wang et al. Neural Cleanse: Identifying and Mitigating Backdoor Attacks in Neural Networks. IEEE S&P 2019.

3.1 直觉

干净模型要把某一类样本误判为类 y_t , 需要 **大幅扰动** 输入 (整张图改写)。后门模型只需 **一小块 trigger** 就足够把任意类样本拉到 y_t 。这种“短路径”本身就是后门的指纹。

3.2 触发器注入的通用形式

定义 trigger 注入函数:

$$\mathcal{A}(x; t, m) = (1 - m) \odot x + m \odot t \quad (9.2)$$

其中: $t \in [0, 1]^{H \times W \times 3}$ —trigger 图案 (RGB 像素值) $m \in [0, 1]^{H \times W}$ —mask, 刻画“哪些像素被 trigger 覆盖” $\odot$ —逐元素乘法 (broadcast 到 3 通道)

当 $m_{ij} = 1 \rightarrow$ 该位置完全被 trigger 替换; $m_{ij} = 0 \rightarrow$ 完全保留原图。

3.3 反向工程 (Reverse Engineering)

思路: 假设目标类是 y_t , 去优化“能把任意干净样本搬到 y_t 的最小 trigger”。对 **每一个候选目标类** $y_t \in \{1, \dots, K\}$ 都做一次:

$$\min_{t, m} \underbrace{\frac{1}{|D_{\text{val}}|} \sum_{x \in D_{\text{val}}} \mathcal{L}(f(\mathcal{A}(x; t, m)), y_t)}_{\text{(a) 攻击成功项}} + \lambda \underbrace{\|m\|_1}_{\text{(b) trigger 越小越好}} \quad (9.3)$$

其中: $\mathcal{L}$ 是分类损失 (CE) f 是 DNN 预测函数 – 第 (a) 项强制“加了 trigger 后被预测为 y_t ”– 第 (b) 项 $\|m\|_1 = \sum_{ij} m_{ij}$ 鼓励 mask 稀疏 $\rightarrow$ 找出 **最小** 触发器 – λ 控制平衡 (典型设 10^{-3} 然后退火)

完成后每个候选 y_t 都得到一个反演 trigger, 记其 mask 范数 $\ell_t := \|m^*(y_t)\|_1$ 。得到向量 $\ell = (\ell_1, \dots, \ell_K)$ 。

3.4 用 MAD 找出“真正的后门类”

直觉: 被后门的真目标类 y_t^* 的 trigger 应当 **显著小于** 其他类的反演结果 (因为后门已经为它留好了“捷径”)。

[NOTE] 算法 9.1 (MAD 异常检测)

记 $\tilde{\ell} = \text{median}(\ell)$, $\text{MAD} = \text{median}(\{|\ell_t - \tilde{\ell}|\}_t)$ 。某类 y_t 的 **异常分数**:

$$A(y_t) = \frac{|\ell_t - \tilde{\ell}|}{1.4826 \cdot \text{MAD}} \quad (9.4)$$

若 $A(y_t) > 2$ 且 $\ell_t < \tilde{\ell}$, 则 y_t 被判为 **后门目标类**。常数 1.4826 来自正态分布下 MAD 与标准差的换算。

3.5 Worked Example: 6 类反演结果如何读图

[EX] 例题 9.1

在一个 6 类小数据集上做完反演，得到 $\ell = (520, 480, 510, **45**, 495, 530)$ 。

步骤：1. $\tilde{\ell} = \text{median} = 502.5$ 2. 偏差绝对值 $\{17.5, 22.5, 7.5, **457.5**, 7.5, 27.5\}$, median = 20, 故 MAD = 20 3. 类 4 的异常分数 $A_4 = 457.5 / (1.4826 \cdot 20) \approx 15.4 \gg 2$, 并且 $\ell_4 < \tilde{\ell}$
结论：类 4 是后门目标类；反演出来的 t^*, m^* 即是触发器的近似。

3.6 缓解 (Mitigation)

拿到反演 trigger 后，可：1. **Input filtering**: 扫描每个输入是否有该 trigger pattern → 拒绝 2. **Neuron pruning**: 把对 trigger 反应强的神经元剪掉 3. **Unlearning**: 用 $\mathcal{A}(x; t^*, m^*)$ 配真标签 fine-tune, 强迫模型遗忘 shortcut

[WARN] 局限

优化次数 = K (类别数), ImageNet (1000 类) 上太贵；对 large trigger / dynamic trigger / feature-space trigger (clean-label 攻击) 失效。

4. 样本过滤: STRIP (Gao et al. 2019)

Y. Gao et al. STRIP: A Defence Against Trojan Attacks on Deep Neural Networks. arXiv:1902.06531, ACSAC 2019.

STRIP = STRong Intentional Perturbation. **核心观察**: trigger 是“非常自信”的捷径——即使把它和其他图像叠在一起，模型仍坚持预测为 y_t 。干净样本则相反：叠加破坏了语义信息，预测会变得不确定（熵高）。

4.1 算法

[NOTE] 算法 9.2 (STRIP 推理时检测)

输入：可疑输入 x ；防御者一组 clean 样本 $\{x_p^1, \dots, x_p^N\}$ ($N \approx 100$)；DNN F_θ 。1. 生成 N 个叠加输入： $\tilde{x}_i = \frac{1}{2}x + \frac{1}{2}x_p^i$ 2. 喂入 DNN 得 softmax 输出 $p_i = F_\theta(\tilde{x}_i) \in \Delta^{K-1}$ 3. 计算每个叠加输入的 Shannon entropy:

$$H_i = - \sum_{k=1}^K p_i^{(k)} \log p_i^{(k)} \quad (9.5)$$

4. 求平均熵 $\bar{H}(x) = \frac{1}{N} \sum_i H_i$ 5. 若 $\bar{H}(x) < \tau \rightarrow$ 判定为 **trojaned**, 拒绝

4.2 直觉为什么 work

输入 x 类型	叠加 $\tilde{x}_i$ 之后	softmax 分布	熵 $\bar{H}$
干净 trojaned	两图语义混合，DNN 拿不准 trigger 仍然完整 (叠加只是“半透明”) → 后门捷径仍触发	均匀 (uniform-ish) 极尖锐，几乎 one-hot 在 y_t	高 低

4.3 阈值 τ 怎么选

在 clean validation set 上算 $\bar{H}$ 的分布 (近似高斯), 设 $\tau = \mu_{\bar{H}} - 3\sigma_{\bar{H}}$, 让假阳率约 0.1%。Gao 等在 MNIST/CIFAR-10/GTSRB 上 FAR < 1% 时 FRR (漏判 trojan) < 1%。

[WARN] 局限

Adaptive 攻击 (例如训练 trojan model 时特意让 trigger 在叠加后失效) 可以绕过; large-trigger 也可能让叠加后熵仍低, 但反过来说明 trigger 太显眼也容易被察觉。

5. 投毒压制: 差分隐私训练 (DP-SGD)

S. Hong et al. On the Effectiveness of Mitigating Data Poisoning Attacks with Gradient Shaping. arXiv:2002.11497, 2020.

5.1 关键观察 (两点)

投毒样本对训练梯度的影响很特别: 1. ℓ_2 范数更大: 单个 poisoned sample 的梯度 $\|\nabla L(x_{\text{poison}})\|_2$ 平均比 clean 样本大——因为它要把模型从“自然类”拉到“目标类 y_t ”, 需要更剧烈调整。2. 方向不同: poisoned 梯度的方向偏离干净梯度的主成分方向。

→ 投毒样本本质是 outlier gradient。如果能“压住 outlier”, 就能让模型 欠拟合 后门捷径。

5.2 DP-SGD: 一举两得

差分隐私训练 (Abadi et al. 2016) 本来为保护单样本隐私设计, 恰好可以压制投毒。

[NOTE] 算法 9.3 (DP-SGD 一步更新)

在 mini-batch B 上: 1. **Per-sample 梯度**: $g_i = \nabla_{\theta} L(f_{\theta}(x_i), y_i)$ for $i \in B$ 2. **梯度裁剪** (clip 到范数上限 C):

$$\bar{g}_i = g_i / \max(1, \|g_i\|_2 / C) \quad (9.6)$$

3. **加 Gaussian 噪声并平均**:

$$\tilde{g} = \frac{1}{|B|} \left(\sum_{i \in B} \bar{g}_i + \mathcal{N}(0, \sigma^2 C^2 I) \right) \quad (9.7)$$

4. **更新**: $\theta \leftarrow \theta - \eta \tilde{g}$

5.3 为什么能防后门

- **裁剪**: 限制单个 poisoned sample 的最大影响力 (再大的范数也截到 C)
- **加噪**: 把方向异常的 outlier gradient 淹没在噪声里 → 模型记不住“trigger → y_t ”这条罕见、剧烈的捷径, 但仍然能学到 dominant 的干净模式。

优点	缺点
训练时一并解决, 无需后处理; 附带隐私保护	干净准确率掉 3–5%; 对大比例投毒 / very-sparse trigger 仍可能失败

6. 模型重建: Fine-Pruning (Liu et al. 2018)

K. Liu, B. Dolan-Gavitt, S. Garg. Fine-Pruning: Defending Against Backdooring Attacks on Deep Neural Networks. **RAID 2018**.

6.1 关键观察

在 backdoored DNN 的最后一层 conv 上看 **每个神经元的平均激活值**:

clean inputs:  (前 k 个神经元活跃)
backdoor inputs (with trigger):  (后 m 个神经元活跃)

→ 有一组神经元 只在投毒输入到来时才被激活, 干净输入下完全沉默。它们就是 **后门神经元**。

6.2 Fine-Pruning 算法

[NOTE] 算法 9.4 (Fine-Pruning, FP)

输入: 可疑模型 f_θ 、clean validation D_{val} 、剪枝比例 p 、fine-tune epochs E 。1. **激活统计**: 对 D_{val} 中每张图前向, 记录最后一个 conv 层每个神经元的激活值; 对所有图求平均得 a_i 。2. **升序排列**, 剪掉 **平均激活最小** 的前 $p \cdot N$ 个神经元 (置 0)。3. **Fine-tune**: 在 D_{val} 上小学习率训练 E epoch, 恢复干净准确率。

直觉: 被剪的神经元在 clean 上几乎不动 → 干净准确率几乎不掉; 后门神经元被剪掉 → ASR 暴跌。剪完后小幅 fine-tune 一是修复 BA, 二是消除 attacker 可能预留的“备份神经元”。

6.3 结果观察

经典实验: 随着剪枝比例 p 从 0 升到 0.8 ——ASR 从 ~100% 单调降到 < 5%, BA 在 $p \leq 0.7$ 时仍 > 90%, 过了 0.8 BA 会崩。最优 p 通常在 0.5–0.7。

[WARN] 局限

Adaptive 攻击: 攻击者可以训练让后门神经元在 clean 上 **也** 激活 (隐藏), 使 FP 找不到目标 → 引出 §7 ANP。

7. 神经元扰动 / ANP (Wu & Wang 2021)

D. Wu, Y. Wang. Adversarial Neuron Pruning Purifies Backdoored Deep Models. **NeurIPS 2021**.

7.1 直觉示例: 扰动一个神经元就能复现后门

考虑两层小网络, 倒数第二层有两个神经元 h_1, h_2 。当前权重 w_1, w_2 , 偏置 b_1, b_2 , 假设 $w_1^\top h_1 + b_1 \geq 0, w_2^\top h_2 + b_2 \geq 0$ 。

干净输入下输出为类“0”。但是: 只要把 w_1, b_1 乘以 $(1 - 0.2) = 0.8$ (相当于削弱第一个神经元 20%), 输出立即翻转为后门类“5”。

→ 后门类对应的“shortcut”权重信息存放在少数神经元里; 只要对这些神经元施加微小扰动, 后门行为就会“溢出”。可以反过来利用这一点: 哪些神经元一被扰动就闯出后门, 它们就是要被剪掉的。

7.2 神经元扰动形式化

对第 l 层第 i 个神经元, 引入乘性扰动 $\delta_i^{(l)}, \xi_i^{(l)}$:

$$\tilde{w}_i^{(l)} = (1 + \delta_i^{(l)}) w_i^{(l)}, \quad \tilde{b}_i^{(l)} = (1 + \xi_i^{(l)}) b_i^{(l)} \quad (9.8)$$

紧凑写法 (向量化, $\circ$ 为 element-wise 乘):

$$\tilde{W}^{(l)} = (1 + \delta^{(l)}) \circ W^{(l)}, \quad \tilde{b}^{(l)} = (1 + \xi^{(l)}) \circ b^{(l)} \quad (9.9)$$

DNN 在扰动下变为:

$$\tilde{f}_\theta(x) = \text{ReLU}(\tilde{W}^{(l)} h^{(l-1)} + \tilde{b}^{(l)}) \text{ (逐层堆叠)} \quad (9.10)$$

7.3 寻找”脆弱”神经元 = max-loss 扰动

在 clean validation D_{val} 上找让 loss 暴涨的最小扰动:

$$\max_{\|\delta\|_\infty \leq \epsilon, \|\xi\|_\infty \leq \epsilon} \frac{1}{|D_{\text{val}}|} \sum_{(x,y) \in D_{\text{val}}} \mathcal{L}(\tilde{f}_\theta(x), y) \quad (9.11)$$

实验关键发现 (slide p33):

[EX] 观察

在 Benign / BadNets / IAB (Input-Aware Backdoor) 三种模型上比较: – (a) **Error rate** 随扰动幅度上升: 后门模型上升 **远快于** 干净模型 $\rightarrow$ “后门模型对神经元扰动更脆弱” – (b) **Prediction Proportion** of misclassified samples: 错分样本中绝大部分被预测为 **target label** $\rightarrow$ 扰动确实触发了”后门捷径”, 验证了 §7.1 的直觉

7.4 ANP 算法: 用 mask 表达”剪 or 留”

引入连续 mask $m_i^{(l)} \in [0, 1]$ (作用同 §3 的 NC mask, 但作用在神经元上而非像素上)。在 mask 上加扰动 $\delta_i^{(l)}$, 构成 **mask + perturbation** 联合优化:

$$\min_{\mathbf{m}} \max_{\|\delta\|_\infty \leq \epsilon, \|\xi\|_\infty \leq \epsilon} \frac{1}{|D_{\text{val}}|} \sum_{(x,y)} \mathcal{L}(\tilde{f}_{\theta, \mathbf{m}, \delta, \xi}(x), y) + \alpha \|\mathbf{m}\|_1 \quad (9.12)$$

[NOTE] 算法 9.5 (ANP 两步)

Step 1 (优化 mask under perturbation): 交替 T 轮: – 内 max: PGD 在 δ, ξ 上 $\rightarrow$ 找当前最危险扰动 – 外 min: 在 $\mathbf{m}$ 上 SGD $\rightarrow$ 学一个把”危险神经元”标记为 0 的 mask
神经元扰动后的权重:

$$(m_i^{(l)} + \delta_i^{(l)}) w_i^{(l)}, \quad (1 + \xi_i^{(l)}) b_i^{(l)} \quad (9.13)$$

Step 2 (按 mask 剪枝): 对每个神经元 i , 若 $m_i < \tau$ (典型 $\tau = 0.2$), 将其权重置 0。

剪完后 ACC 几乎不变, 但 ASR 大跌 (见 §7.5)。

7.5 实验: ACC vs ASR 全景 (CIFAR-10)

[EX] 例 9.2 (ANP 论文 Table 1 关键数字)

设定: – 只用 500 张 clean data (CIFAR-10 训练集的 1%) – 2000 iterations – 模型: ResNet-18, 攻击: BadNets / Blend / IAB 等

攻击	无防御 (ACC / ASR)	Fine-Pruning (ACC / ASR)	ANP (ACC / ASR)
BadNets	94.0% / 100%	93.4% / 8.4%	93.9% / 1.7%
Blend	94.1% / 99.9%	92.0% / 18.3%	93.5% / 3.5%
IAB	94.1% / 99.5%	90.5% / 39.0%	93.4% / 4.2%

观察: ANP 在所有攻击上 ACC 几乎无损 ($\leq 0.7\%$ 下降), ASR 一律压到 $< 5\%$; FP 在 dynamic trigger (IAB) 上 ASR 仍残留 39% ——说明 dynamic backdoor 神经元在 clean 上并不沉默, FP 找不到; ANP 通过“扰动一下让你露馅”绕过这个隐藏。

[TIP] ANP vs FP 一句话区别

FP: 剪“clean 上沉默”的神经元 (直接量 activation)。ANP: 剪“稍微扰动就闯出后门”的神经元 (通过对抗找出 fragile neuron)。后者对 adaptive / dynamic backdoor 更鲁棒。

8. 通道 Lipschitz 剪枝 / CLP 一首个 Data-Free 防御 (Zheng et al. 2022)

R. Zheng et al. Data-free Backdoor Removal Based on Channel Lipschitzness. ECCV 2022.

8.1 前置: Lipschitz 常数与通道 Lipschitz 常数

[NOTE] 定义 9.1 (Lipschitz 常数)

函数 $g: \mathbb{R}^n \rightarrow \mathbb{R}^m$ 的 Lipschitz 常数是使最小的 K 使

$$\|g(x) - g(y)\|_2 \leq K \|x - y\|_2, \quad \forall x, y \quad (9.14)$$

直观: K 是“输出对输入最敏感的变化率”。线性层 $g(x) = Wx$ 的 Lipschitz 常数恰为 spectral norm $\sigma_{\max}(W)$ (最大奇异值)。

CNN 一个 conv 层有 C_{out} 个输出 channel。Channel Lipschitz 常数 = “把整层视作只输出第 c 个 channel 的函数”的 Lipschitz 常数 K_c 。整层的 Lipschitz 常数受所有 channel Lipschitz 的支配, 但每个 channel 各自的 K_c 可以单独看, 揭示哪个 channel “最敏感”。

8.2 关键观察 (UCLC vs TAC)

两个量: – UCLC_c (Upper bound of Channel Lipschitz Constant) = channel c 的卷积权重矩阵的 spectral norm。可证明 $= \sigma_{\max}(W_c)$ ——仅靠预训练权重即可计算, 无需任何数据。– TAC_c (Trigger Activation Change) = $\|\phi_c(\mathcal{A}(x; t, m)) - \phi_c(x)\|_2$, “加 trigger 前后 channel c 的 feature map 变化量”(需要数据 + 已知 trigger 才能算, 仅作为 ground truth)。

→ Zheng 等实验发现: UCLC 与 TAC 强正相关。UCLC 大的 channel, 往往就是“加上 trigger 后 feature 暴涨”的 channel ——即 后门 channel。

[TIP] 一句话洞察

后门 channel = 对 trigger 这种小扰动反应特别剧烈的 channel。Lipschitz 常数恰好刻画“最敏感的变化率”，因此大 UCLC = 后门嫌疑。

8.3 CLP 算法

[NOTE] 算法 9.6 (Channel Lipschitz Pruning)

输入：可疑模型 f_θ ，阈值因子 u (典型 $u = 3.0$)。完全不需要任何数据。1. 对每个 conv 层 l 、每个输出 channel c ，计算 UCLC：

$$\text{UCLC}_c^{(l)} = \sigma_{\max}(W_c^{(l)}) \quad (9.15)$$

其中 $W_c^{(l)}$ 是 channel c 对应卷积核展开成的矩阵。2. 在该层内求 median $\mu^{(l)}$ 和 $\text{MAD}^{(l)}$ 。3. 若 $\text{UCLC}_c^{(l)} > \mu^{(l)} + u \cdot \text{MAD}^{(l)} \rightarrow$ 把该 channel 置 0 (剪掉)。4. 全部层处理完 $\rightarrow$ 输出 purified 模型。

8.4 结果

CIFAR-10 / Tiny-ImageNet 实验：– 不需任何数据、不需 fine-tune；秒级完成 – ACC 通常仅下降 1–2% – ASR 在主流攻击 (BadNets, Trojan, CLA, SIG, WaNet 等) 上压到 0–5% – 与 ANP 同档，但 ANP 仍需 500 clean + 2000 iter；CLP 是真 zero-cost

[WARN] 局限

若攻击者刻意让后门 channel 的 spectral norm 与正常 channel 相近 (adaptive)，CLP 会失效；但目前未见能 bypass CLP 的 adaptive 攻击的有效案例。

9. 防御方法对比表

方法	阶段	需 clean data?	需重训?	计算成本	适用 trigger	主要弱点
Neural Trojans (AE)	推理输入预处理	★ ($\sim 10^3$, 训 AE)	否 (训 AE 不算模型重训)	中	小贴片 trigger	大 trigger / blended 失效, AE 降 BA
Neural Cleanse (NC)	训练后	★ (~clean val)	可选 (用于 unlearning)	高 (K 次优化)	小 patch trigger	类别多时贵; dynamic / clean-label 失败
STRIP	推理时	★ ($\sim 10^2$ for 叠加源)	否	中 (每输入 N 次前向)	显式 patch	adaptive 可绕过; large trigger 不稳
DP-SGD	训练时	自己的训练集 (假设大部分干净)	是 (本身就是训练)	中-高 (per-sample 梯度贵)	多数 poisoning	BA 掉 3–5%; sparse trigger 不一定能压
Fine-Pruning	训练后	★ (clean val)	是 (剪后 fine-tune)	低	静态 trigger	dynamic / hidden backdoor 失效
ANP	训练后	★ (500 张就够)	否 (mask + prune)	中 (2000 iter)	包括 dynamic	仍需少量 data; 阈值 τ 需调
CLP	训练后	0	否	极低 (一次 SVD)	多数 trigger	adaptive 未来可能突破

10. 期末复习要点

[TIP] 期末复习要点

1. 威胁模型 3 种 + 生命周期 4 阶段——能匹配每个防御到正确格子。2. **Neural Cleanse** 反演公式 (9.3) + **MAD** 检测 (9.4) 必背。注意 $\|m\|_1$ 是核心稀疏正则。3. **STRIP** 直觉: trigger 即“自信捷径”, 叠加后后门样本熵低、干净样本熵高。能写出公式 (9.5)。4. **DP-SGD** 抗投毒: 靠 (a) 单样本梯度 clipping 限影响 (b) 加高斯噪声淹没 outlier 方向。能写出更新 (9.6)(9.7)。5. 三种剪枝法的区别: – **Fine-Pruning** 一剪“clean 上沉默”神经元 – **ANP** 一剪“微小扰动就闯后门”的脆弱神经元 (式 9.12 mask + perturb 双重 min-max) – **CLP** 一剪“spectral norm 异常大”的 channel, 零数据 6. **ANP** 标志数字: 500 clean data + 2000 iters + CIFAR-10 + ACC 几乎无损 / ASR < 5%。7. **CLP** 关键定理: $UCLC = \text{spectral norm of channel weight} = \text{可从权重直接算} \rightarrow \text{data-free}$ 。8. 每种防御的失败场景: 能为每种方法举一个 adaptive 攻击/局限 (NC 失 dynamic, STRIP 失 adaptive, FP 失 IAB, DP 失 sparse, CLP 失“压平 spectral norm”的攻击)。

11. Anki Flashcards

[NOTE] Anki Q/A

1. Q: 后门攻击的三种典型威胁场景? A: 用第三方模型 / 用第三方数据 / 用第三方训练平台。
2. Q: 后门防御的四个生命周期阶段? A: 预处理 (pre-processing)、训练时 (training-time)、训练后 (post-training)、推理时 (inference-time)。
3. Q: Neural Trojans (Liu 2017) 的两种子方法各是? A: (a) 每类训一个 binary classifier 做输入异常检测; (b) auto-encoder 重构输入以破坏 trigger。
4. Q: 写出 Neural Cleanse 的触发器注入公式。A: $\mathcal{A}(x; t, m) = (1 - m) \odot x + m \odot t$ 。
5. Q: Neural Cleanse 反演阶段的优化目标? 为什么有 $\|m\|_1$ 项? A: $\min_{t, m} \mathcal{L}(f(\mathcal{A}(x; t, m)), y_t) + \lambda \|m\|_1$ 。 $\|m\|_1$ 让 mask 稀疏, 保证找到 最小 触发器, 便于与正常类的 mask 范数对比检测异常。
6. Q: 什么是 MAD? 某点是 outlier 的判据? A: $\text{MAD} = \text{median}|a_i - \text{median}(a)|$; 异常分数 = $|a_i - \text{median}| / (1.4826 \cdot \text{MAD})$, > 2 即视为异常。
7. Q: Neural Cleanse 怎么判断哪一类是后门目标类? A: 对每个候选目标类反演 trigger, 得 $\ell_t = \|m^*\|_1$; 用 MAD 检测离群点, 且 $\ell_t < \text{median} \rightarrow$ 后门类。
8. Q: STRIP 的核心算法步骤? A: 用 N 张 clean 图分别和可疑输入 x 叠加 $\rightarrow$ 算 softmax 平均 Shannon 熵 $\bar{H} \rightarrow \bar{H} < \tau$ 判为 trojaned。
9. Q: 为什么 trojaned 样本叠加后熵低? A: trigger 是非常自信的“捷径”, 叠加只是让 trigger 变半透明 $\rightarrow$ 后门仍触发 $\rightarrow$ softmax 集中在 $y_t \rightarrow$ 熵低。干净样本叠加后语义混合 $\rightarrow$ 熵高。
10. Q: 投毒样本的梯度有哪两个特殊性质? A: (1) ℓ_2 范数更大; (2) 方向偏离 clean 梯度主成分方向。即 outlier gradient。
11. Q: 写出 DP-SGD 的 per-sample clipping 公式与加噪更新。A: $\bar{g}_i = g_i / \max(1, \|g_i\|_2 / C)$; $\tilde{g} = \frac{1}{|B|} (\sum_i \bar{g}_i + \mathcal{N}(0, \sigma^2 C^2 I))$ 。
12. Q: Fine-Pruning 怎样找到要剪的神经元? A: 在 clean validation 上前向, 记录最后一个 conv 层每个神经元的平均激活; 升序排, 剪掉激活最小的前 p 部分。
13. Q: Fine-Pruning 失败的典型场景? A: dynamic backdoor / IAB —— 后门神经元在 clean 上也激活, FP 找不到。
14. Q: ANP 的核心 idea (一句话)? A: 剪那些“对很小神经元扰动就让模型闯出后门行为”的脆弱神经元 (而不是 sleeping 的)。
15. Q: 写出 ANP 神经元扰动的紧凑公式。A: $\tilde{w}_i^{(l)} = (1 + \delta_i^{(l)}) w_i^{(l)}$, $\tilde{b}_i^{(l)} = (1 + \xi_i^{(l)}) b_i^{(l)}$; δ, ξ 由 max-loss PGD 求得。
16. Q: ANP 的 mask + perturbation 双重优化是? A: $\min_m \max_{\delta, \xi} \sum \mathcal{L}(\tilde{f}(x), y) + \alpha \|m\|_1$; 交替 PGD/SGD 后按 $m_i < \tau$ 剪枝。
17. Q: ANP 论文标志性实验数字 (CIFAR-10)? A: 仅用 500 张 clean data + 2000 iters, ACC ~ 无损, ASR $< 5\%$ 。
18. Q: 定义 Channel Lipschitz Constant 的 upper bound (UCLC); 它在线性层等于什么? A: UCLC_c 是 channel c 子函数的 Lipschitz 上界; 等于该 channel 卷积权重矩阵的 spectral norm $\sigma_{\max}(W_c)$ 。
19. Q: CLP 为什么能完全 data-free? A: 因为 UCLC 只需对每个 channel 权重做一次 SVD 即可算出 $\rightarrow$ 不需要任何 clean / trigger 数据。
20. Q: CLP 的核心观察是? A: UCLC 与 TAC (trigger 前后 feature 变化量) 强相关 $\rightarrow$ UCLC 异常大的 channel 即为后门 channel。
21. Q: CLP 怎么判定一个 channel 该剪? A: 按层算 UCLC 的 median 与 MAD; 若某 channel $\text{UCLC} > \mu + u \cdot \text{MAD}$ (典型 $u = 3$) 则剪。
22. Q: 列三种“训练后剪枝防御”的代表方法及最大区别。A: FP (剪沉默神经元, 简单); ANP (剪脆弱神经元, 对 dynamic backdoor 鲁棒); CLP (剪 spectral norm 异常 channel, **zero-data**)。
23. Q: 哪种防御 不需要重新训练任何东西? A: STRIP / ANP / CLP。Neural Trojans 要训 AE; FP 要 fine-tune; DP-SGD 本身就是训练。
24. Q: 哪种防御 完全不需要任何 clean data? A: CLP (Zheng et al. 2022), 首个 data-free 后门防御。
25. Q: 防御者的最终目标用一句话怎么写? A: 在不大幅降低 BA (benign accuracy) 的前提下, 把 ASR (attack success rate) 压到尽可能低。

12. 记号速查表

符号	含义
f_θ	神经网络分类器
$\mathcal{A}(x; t, m)$	trigger 注入函数 (NC 公式 9.2)
t	触发器图案
m	mask, $\in [0, 1]^{H \times W}$
$\odot$	逐元素乘
$\ m\ _1$	mask 的 ℓ_1 范数 (trigger 大小代理)
$\mathcal{L}$	分类损失 (CE)
λ	NC 中 mask 稀疏项系数
ℓ_t	类 y_t 反演 trigger 的 $\ m\ _1$
$\tilde{\ell}$	ℓ 的中位数与 MAD
$A(y_t)$	类 y_t 的异常分数 (NC)
H	Shannon 熵
$\bar{H}$	STRIP 中 N 个叠加输入的平均熵
τ	STRIP / ANP / CLP 的判定阈值
C	DP-SGD 梯度 clipping 范数
σ	DP-SGD 噪声乘子
$w_i^{(l)}, b_i^{(l)}$	第 l 层第 i 个神经元的权重、偏置
$\delta_i^{(l)}, \xi_i^{(l)}$	ANP 中乘性扰动
$m_i^{(l)}$	ANP 中神经元 mask
α	ANP 中 mask 稀疏项系数
$\text{UCLC}_c^{(l)}$	层 l channel c 的 Lipschitz 上界 (CLP)
TAC_c	trigger 前后 channel c 的 feature 变化量
$\sigma_{\max}(W)$	矩阵 W 的最大奇异值 (spectral norm)
u	CLP 中剪枝阈值因子 (典型 3.0)
BA	Benign Accuracy (干净准确率)
ASR	Attack Success Rate
D_{val}	防御者持有的 clean validation 集

Lecture 10 —隐私 (Privacy)

[TIP] 学习指南

本节核心问题：机器学习模型在训练和推理过程中，会以哪些方式泄露训练数据中的私人信息？我们又有哪些技术手段可以系统性防御？前置知识：过拟合 (overfitting) 与泛化间隙 (generalization gap) → 见 [[lec02_ml_introduction]]；交叉熵 / softmax 置信度向量 → 见 [[lec02_ml_introduction]] 与 [[lec03_cnn]]；模型置信度梯度的可微性 (用于 model inversion) → 见 [[lec06_adversarial_attack]] 的 FGSM/PGD 直觉。差分隐私 (DP) 的“邻居数据集不可区分”思想，与 [[lec09_backdoor_defense]] 中的“鲁棒输入扰动”在数学结构上是同源的 (都是对输出加随机性以抑制对单点的敏感)。走完本节，你应当能：(1) 区分 **成员推理 (MIA)** 与 **特征重建 (Model Inversion)** 与 **模型提取 (Extraction)** 三类攻击；(2) 写出 Shokri 影子训练 4 步法；(3) 解释 Carlini 2021 用于排序 GPT-2 输出的 6 个指标；(4) 写出 (ϵ, δ) -DP 的形式化定义；(5) 比较 HE / SMPC / DP / FL 四类防御的代价与场景。

0. 核心问题 / Motivation

随着机器学习被部署到 **医疗、金融、信贷、推荐、人脸识别** 等行业，训练数据中常常包含 **病历、身份证、社交关系、银行流水** 等高度敏感信息。GDPR (EU, 2018)、CCPA (California, 2018)、《网络安全法》(China, 2017) 都把“个人信息”的收集、存储、再分发置于法律强约束之下。

核心洞见：与传统数据库不同，机器学习模型本身就是训练数据的一个 有损但常常超额的压缩——只要模型 **过拟合** 一点点，攻击者就能从模型参数 / 输出中反推出训练样本，甚至完整的 PII。“模型 $\neq$ 数据”是错觉，模型 = 会说话的数据库。

5 类常见隐私风险 (slide p.6 展开)：

风险	例子
冒用与盗用 (Impersonation & Theft)	攻击者拿到密码 → 转账；拿到信用卡号 → 网购“礼物”；拿到身份信息 → 修改账户绑定的邮箱/手机/地址或开设新账户
训练数据泄漏 (Training data leakage)	攻击者通过 MIA 推断“张三的医疗记录是否在医院模型训练集”；通过 model inversion 反推某类的“平均人脸”
不公平对待 (Unfair treatment)	基于性取向歧视；基于“疑似既有疾病”拒绝保险；基于与岗位无关的私人信息拒绝雇佣
模型本体泄漏	揭露保险公司定价模型的架构与参数；窃取银行信贷审批模型
行为画像滥用	浏览历史 / 购买记录被用于精准广告 → 进而被推断出健康状况、政治倾向等

整个第 10 讲从这一句话出发：“**不要存储你不需要的信息**”——这是治本的总原则，但实际工程几乎总要存一些必要数据，所以后半节讲 4 种 技术补救。

0.x 名词预热 (Glossary)

[NOTE] 一表读懂

所有以下术语会在正文反复出现。第一次见到不必精读，回头查表即可。

术语	1 句话定义	出现位置
GDPR (General Data Protection Regulation)	欧盟 2018 年生效的通用数据保护条例，对处理欧盟公民个人数据的全球企业设最高营收 4% 的罚款	§0
CCPA (California Consumer Privacy Act)	美国加州 2018 年通过的消费者隐私法，赋予加州居民知情权、删除权、退出销售权	§0
网络安全法	中国 2017 年生效，规定关键信息基础设施数据本地化、个人信息收集需明示同意	§0
PII (Personally Identifiable Information)	能单独或组合后识别到自然人的信息：姓名、身份证号、手机号、邮箱、家庭住址、银行账号	§1, §7
PHI (Protected Health Information)	个人健康信息：病历、诊断、用药、保险编号	§1
Inference attack / Confidentiality attack	推理攻击：攻击者通过查询接口推断本不应公开的训练数据属性或模型属性	§2
Shadow model	影子模型：攻击者在自己掌握的、与目标分布相近的数据上训练的“替身模型”，用来给攻击模型生成 (in, out) 标签	§4–5
Perplexity P	困惑度，语言模型对一段文本的不自信程度，越低表示模型越“熟悉”这段文本（公式见 §7.2）	§7
zlib entropy	一段字符串经 zlib (DEFLATE) 压缩后的字节数，可近似其 Kolmogorov 复杂度；高 zlib 熵 + 低 perplexity → 强记忆	§7
HE (Homomorphic Encryption)	同态加密：允许在密文上直接计算，解密后得到明文运算结果	§8–9
SMPC (Secure Multi-Party Computation)	安全多方计算：多方在不暴露各自输入的前提下合作算出函数值	§8, §10
DP (Differential Privacy)	差分隐私：给查询/算法输出加噪，使“个人是否在数据集”几乎不可区分	§8, §11
FL (Federated Learning)	联邦学习：数据留在本地，只交换模型权重/梯度的分布式训练范式	§8, §12
DP-SGD	差分隐私版本的 SGD：逐样本梯度裁剪 + 加高斯噪声 + 平均	§11
FedAvg	联邦学习经典聚合算法：本地多步 SGD → 上传权重 → 服务器加权平均	§12
Sensitivity Δf	敏感度：把任一条记录从数据库中替换/移除后，函数输出变化的上界	§11

1. 隐私基础

隐私 (Privacy) 个人对自己个人信息的 **采集 / 使用 / 传播** 的控制能力 (slide p.4)。三大支柱：

- **机密性 (Confidentiality)**：阻止未授权访问、盗窃、滥用。
- **数据保护 (Data Protection)**：技术 + 组织 + 法律手段，包括加密、访问控制、备份与恢复。
- **匿名性 (Anonymity)**：可以使用信息但无法识别到个人。

1.1 5 种隐私类别 (slide p.5)

类别	内涵	例子	主要法规
Personal Data Privacy	PII	姓名、住址、社保号	GDPR / CCPA
Location Privacy	物理位置	GPS 轨迹、基站定位	GDPR / CCPA
Communication Privacy	通信内容	邮件、短信、IM	GDPR
Healthcare Privacy	PHI	病历、诊断、处方	HIPAA (美) / GDPR
Behavioral Privacy	行为	浏览历史、购物记录	CCPA / GDPR (常用于个性化广告)

1.2 总原则：Do Not Store What You Do Not Need

(slide p.7)

- 删除非必要字段：注册时不需要血型就别要血型。
- 用随机 ID 替代真实姓名：UUID / 哈希后存储。
- 粗化 / 区间化：把”出生 1995-07-14”改成”90 后”。

[WARN] 注意

上述净化 本身 并不能提供严格隐私——Netflix-IMDb 攻击 (§1.3) 表明，看似匿名的数据在与外部信息交叉之后可被重新识别。这就是引入 形式化定义 (差分隐私) 的动机。

1.3 Worked Example: Netflix-IMDb 去匿名化 [Narayanan & Shmatikov, S&P 2008]

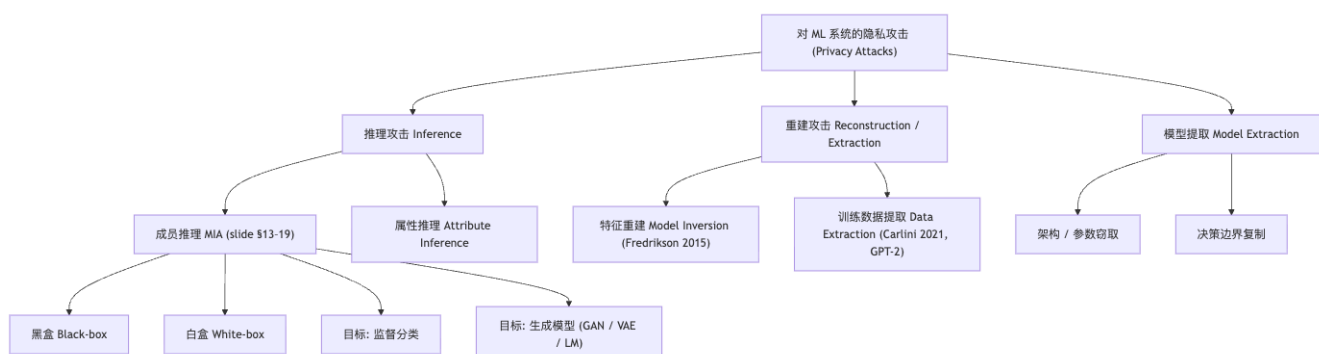
[EX] 例题—你 以为 你匿名了

背景：2006 年 Netflix 公开了一份”清洗过”的电影评分数据集 (50 万用户 × 1 亿评分)，用户名 → 随机 ID，作为推荐算法竞赛 (Netflix Prize) 的数据。

攻击者拥有：公开的 IMDb 评分 (有真实用户名)。攻击思路：对每个 IMDb 用户 u ，找出 Netflix 数据集中与 u 的 (电影, 评分, 时间戳) 匹配模式最相似的匿名 ID。关键观察：人的口味是 高维稀疏指纹——即使只匹配 8 部电影评分 (允许 14 天误差)，已足以 **唯一确定 99% 的用户**；只需 2 部小众电影即可锁定 68%。后果：被锁定的用户的政治倾向 (《华氏 911》)、性取向 (《断背山》) 等敏感属性立刻外泄。

教训：去标识 ≠ 隐私。任何高维 个人行为 数据本质上就是身份信息。这一发现直接推动 Dwork (2006) 提出差分隐私。

2. 攻击全景图



[NOTE] 阅读地图

§4–5: MIA; §6: Feature Inference; §7: Carlini GPT-2 Extraction (覆盖 D + F + G)。

3. 隐私泄漏的根因 (Causes of Leakage)

(slide p.11)

(R1) 过拟合 (Overfitting) 是隐私泄漏的头号元凶。

直觉论证：泛化间隙 $\text{gap} = L_{\text{test}} - L_{\text{train}}$ 越大，模型在 **训练集** 上的损失 系统性地 比测试集低 $\rightarrow$ 模型对训练样本表现出 可被区分的 自信度。MIA 正是利用这条 自信度差距。形式化地：若模型对 in-sample 与 out-sample 的输出分布可被一个分类器 g 区分，则攻击成功率 $> 50\%$ ，且与 gap 强正相关 (Yeom et al., CSF 2018 给出形式化界)。

(R2) 数据多样性 / 类别数。类别越多 $\rightarrow$ 每类样本越少 $\rightarrow$ 模型越倾向记忆个体。

- 二分类模型相对安全。
- 多分类 (e.g. 1000-class 人脸识别) 风险高。
- 离群样本 (OOD wrt 训练分布) 最易被记忆，也最易被攻击 (因其在 softmax 上呈“尖峰”)。

(R3) 模型复杂度 / 容量。参数量越大，记忆能力越强。GPT-2 XL 记忆量是 GPT-2 Small 的 $\sim 10\times$ (slide p.25)。深度残差网络比浅层 CNN 更易被 MIA。

[WARN] 推论

提升隐私的 两个免费午餐：(i) 强正则化 / Dropout / Early-stopping $\rightarrow$ 缩小 $\text{gap} \rightarrow$ 削弱 MIA；(ii) 模型蒸馏 / 减参 $\rightarrow$ 降低记忆容量。但都只是 启发式，没有形式化保证。形式化保证只有 §11 的 DP 给得出。

4. 成员推理攻击 (Membership Inference Attack, MIA)

4.1 形式化定义

设目标模型 $f_\theta: \mathcal{X} \rightarrow \Delta^{C-1}$ (输出 C 类的 softmax 向量)，训练集 $\mathcal{D}$ ，目标样本 x^* 。MIA 攻击者训练一个二分类器

$$g(f, x^*) \rightarrow \{\text{in}, \text{out}\}, \quad \text{in} \iff x^* \in \mathcal{D}. \quad (10.1)$$

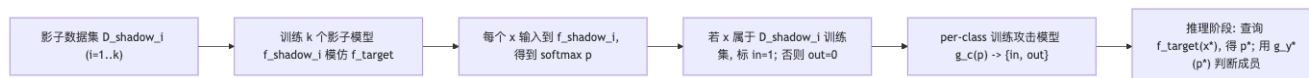
威胁模型 (Shokri et al., S&P 2017): – 黑盒查询访问 f (任意输入 $\rightarrow$ 任意 softmax 向量)。– 攻击者不知道 $\mathcal{D}$ ，但知道 $\mathcal{D}$ 的大致分布、 f 的任务、 f 的训练 API (云平台公开)。– 目标：对给定 x^* 判断成员身份。

4.2 攻击原理 (slide p.14, “Rationale”)

模型在 **训练样本** 上的 softmax 自信度系统性地高于在 **未见样本** 上——因为它见过这些样本、被 CE 推着把概率推到接近 one-hot。

例：目标模型做人脸识别。攻击者从互联网下载 Brad Pitt 的照片去查询。若返回的 Brad Pitt 类置信度 $\rightarrow 1.0$ ，则该照片很可能在训练集中；若置信度 = 0.4 (与其他人脸近)，则未在。

4.3 Shokri 影子训练 4 步法 (slide p.15–17)



Step 1 一构造影子数据： k 份与目标分布相近的影子集 $D_1, \dots, D_k$ (允许部分重叠)。**Step 2 一训练影子模型**：用与 f_{target} 相同或相似 的架构 + 训练流程。云 MLaaS API (Google, Amazon) 公开训练流程 $\rightarrow$ 易于复制。**Step**

3 —制造攻击模型训练集：对每个 x 收集 (softmax 输出向量 p , 标签 in/out)。关键假设：影子模型在影子训练集与影子测试集上的 softmax 分布不同，且这个差异与目标模型上的差异一致。Step 4 —训练 per-class 攻击模型：对每个类别 c 训练一个 g_c 二分类器 (因不同类的过拟合程度不同)。

[NOTE] 结果

在 CIFAR-10 / Texas hospital / MNIST 上, Shokri 报告攻击 precision $\in [0.6, 0.95]$, 远高于 0.5 的随机基线。模型越过拟合, 攻击越成功。

4.4 推广

- 已被推广到 GAN (检查生成器是否记忆) 和 VAE (Hayes et al., PETS 2019)。
- 已被推广到 联邦学习 (Nasr et al., S&P 2019): 恶意客户端可对其他客户端做 MIA。
- 白盒变体 (访问梯度) 更强, 可达 0.85+ AUC。

5. 影子训练数据从何而来 (slide p.19)

[WARN] 假设松绑

上一节假定攻击者能拿到与目标分布相近的真实样本。若 **拿不到**, 怎么办? Shokri 提出 3 种策略:

(S1) **Model-based synthesis (合成 / Hill-climbing)**。对每个目标类 c , 用爬山法不断微调随机初始化的样本, 直到目标模型对 c 的置信度 > 阈值:

```
def synthesize_for_class(c, f_target, T_max, conf_thresh):
    x = random_sample()          # e.g. uniform pixel
    best_conf = 0
    for t in range(T_max):
        x_new = randomly_perturb(x, k_features)  # 随机改 k 个特征
        p_new = f_target(x_new)[c]
        if p_new > best_conf:      # 接受改善
            x, best_conf = x_new, p_new
        if best_conf >= conf_thresh:
            return x
    return None
```

直觉: 高置信度样本 $\approx$ “训练分布上”典型代表”, 可充当合成的影子训练样本。

(S2) **Statistics-based**: 攻击者知道边缘统计 (e.g. 各特征的均值、方差、边际直方图), 从中独立采样合成。

(S3) **Noisy real data**: 用真实但与目标训练集不相交的数据 (允许 10–20% 标签噪声)。实验表明仍能保持攻击有效。

6. 特征推理 / 重建攻击 (Feature / Reconstruction Inference)

(slide p.21–23)

目标: 恢复 x^* 的某些特征, 或一类的代表样本。别名: Attribute Inference / Reconstruction Attack / Data Extraction。

6.1 Model Inversion via Confidence Gradient (Fredrikson, Jha, Ristenpart, CCS 2015)

经典案例: 从浅层神经网络的人脸识别器, 给定目标人名 y , 重建出这个人的平均面孔。

优化目标：在输入空间上找一个 x ，使目标类的置信度最大：

$$x^* = \arg \max_{x \in [0,1]^d} f_{\theta}(x)[y] - \lambda \cdot \text{prior}(x), \quad (10.2)$$

其中 $\text{prior}(x)$ 是一个正则项 (TV norm / 自然图像先验) 防止结果像对抗样本。

算法：梯度下降 / 上升 (white-box) 或基于查询的数值梯度 (black-box)。

```
x = init_random_image()
for t in range(T):
    grad = ∇x [ ftheta(x)[y] - λ·prior(x) ]
    x = clip01(x + η · grad)
```

[EX] 结果 (Fredrikson 2015)

在 AT&T 人脸数据集上，仅给定姓名 + 黑盒查询，重建出的人脸 可被志愿者以 75% 准确率匹配回原人。模型越浅、训练集越小，重建越清晰。

6.2 类平均 vs. 个体重建

攻击	输出	风险
Class average reconstruction	类 y 的“原型样本”	中：暴露训练集存在哪类敏感个体
Per-sample reconstruction	具体某个 x_i	极高：直接暴露训练样本本身 (见 §7 GPT-2)

7. 案例研究：GPT-2 训练数据提取 (Carlini et al., USENIX Security 2021)

论文：Extracting Training Data from Large Language Models (arXiv:2012.07805) 这是隐私领域 里程碑式 工作：在真实大型生产模型 (OpenAI GPT-2) 上展示训练数据可被字面提取。

7.1 目标与威胁模型 (slide p.24–26)

目标：从 GPT-2 中提取 被记忆的 训练样本。攻击者能力：仅黑盒查询访问 GPT-2 (任意 prefix → 续写)。强弱度量：被提取文本在训练集中 出现的文档数 越少 → 攻击越强。最强情形：文本只在单个文档出现过 (unintended memorization)。

模型规格：– GPT-2 Small / Medium / XL = 124M / 334M / 1.5B 参数；– 训练数据：40 GB 去重网页文本 (OpenAI 未公开数据集本身，但公开了爬取脚本)。– 关键发现：XL 记忆量 $\approx$ Small 的 10 倍—越大越泄漏。

7.2 6 个排序指标 (slide p.28)

攻击核心难点：GPT-2 一秒可生成上千 token，怎么从噪声中筛出 真正被记忆 的样本？Carlini 设计 6 个指标，每个指标取 top 100，去重后人工核验。

(M1) Perplexity ——模型对自己生成文本的“困惑度”：

$$P(x_1, \dots, x_n) = \exp\left(-\frac{1}{n} \sum_{i=1}^n \log f_{\theta}(x_i \mid x_1, \dots, x_{i-1})\right) \quad (10.3)$$

直觉： P 越小 → 模型越笃定下一个 token → 文本越可能是 死记硬背 而非生成。单独使用的问题：大量 固定模板 (网址前缀、版权声明) 也有低 P ，但并非“个性化的记忆”，会被误选。下面 5 个指标都是为了排除这种“全网到处都有”的低熵噪声。

(M2) 与小模型比较：定义比率 $r = P_{\text{XL}}(x) / P_{\text{Small}}(x)$ 。若 $r \ll 1 \rightarrow$ 只有 XL 记得，Small 不记得 $\rightarrow$ 强记忆信号 (越大的模型记得越多 的副产品)。直觉：通用的高频文本两种模型都熟；只有大模型 单独 自信的，更可能是它独自记下的稀有样本。

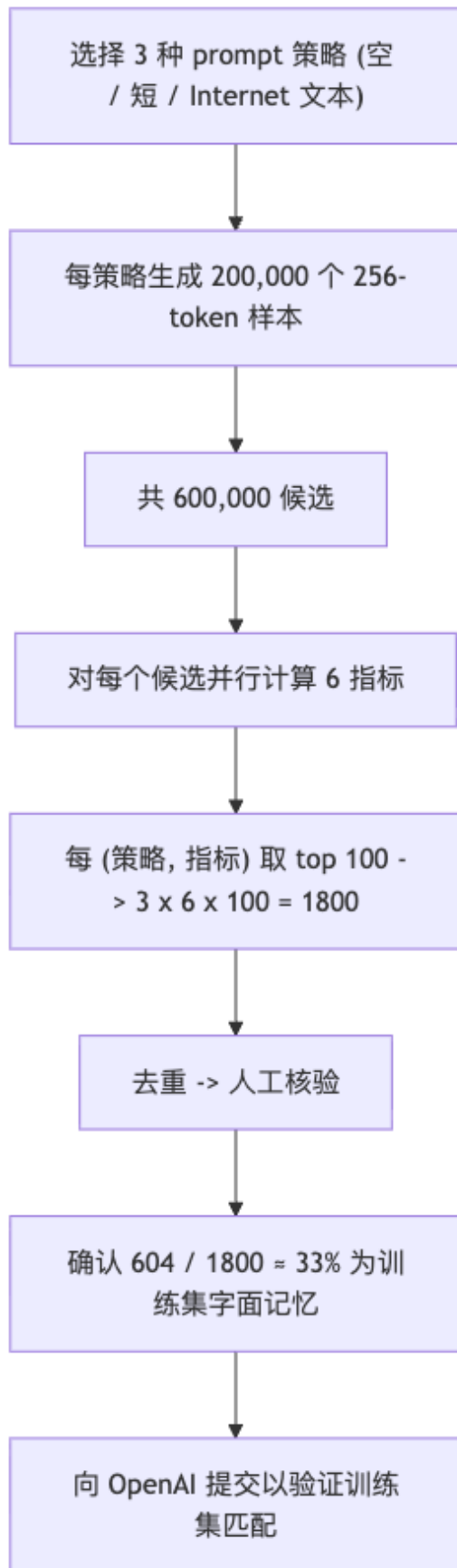
(M3) zlib entropy 比率： $z(x) = |\text{zlib.compress}(x)|$ (字节数)。用 $P(x) / z(x)$ 排序。直觉：zlib 压缩后大 $\rightarrow$ 字符串本身高熵 (难压缩)；同时 P 低 (模型自信)。高 zlib + 低 perplexity 是黄金组合：意味着这是一段 看似随机 却被模型 记牢 的字符串，几乎一定是被原文记忆的私人数据 (UUID、邮箱、地址等)。

(M4) Lowercase perplexity 比率： $P(x) / P(\text{lower}(x))$ 。模型对原始大小写 远 比小写更确定 $\rightarrow$ 表明模型记住了 精确大小写形式 $\rightarrow$ 是字面记忆。

(M5) Sliding-window perplexity (window=50 tokens)：取局部窗口中最小 perplexity。直觉：一段 256-token 长文本可能整体 perplexity 高，但其中藏一段 50 token 的精确背诵；滑窗能抓住这种局部记忆。

(M6) Internet-text prompts：用从公共网页随机抓取的句子作为 prompt (而非空 prompt)，迫使模型续写真实文档。提升发现训练样本的命中率。

7.3 完整攻击流水线



7.4 提取出的内容例子 (slide p.29–31)

[EX] Prefix prompt: “East Stroudsburg Stroudsburg...”

GPT-2 续写后输出了某真实个人的 **全名、电话号码、邮箱、家庭地址**——这段内容在训练集中 只出现在一个文档 中，被 XL 一字不差地记住。

按类别统计 (Carlini Table 1):

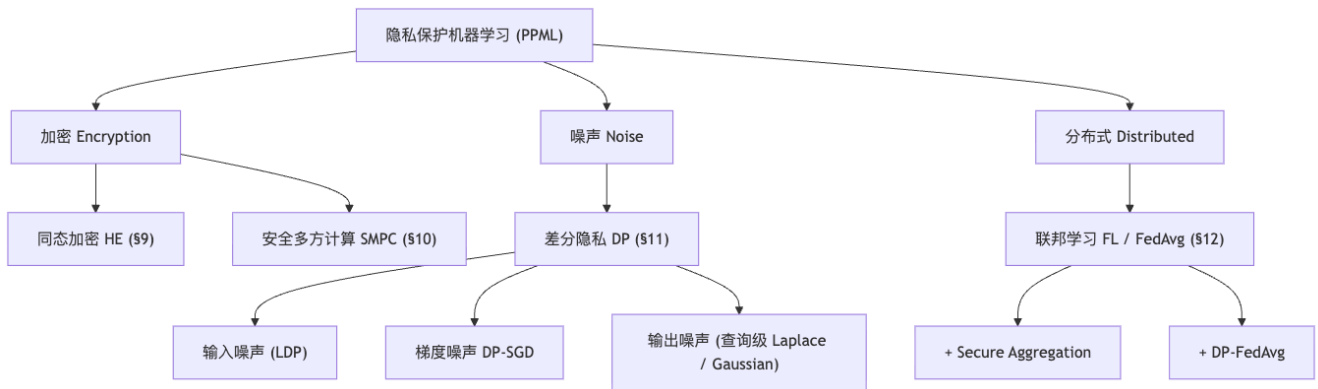
类别	提取数量	备注
PII (姓名 + 电话 / 邮箱 / 地址 / 社交账号)	78	最敏感
URL	50	部分指向已下线页面
代码片段	31	含 GitHub 私仓提交?
UUID / 不可读字符串	e.g. 1e4bd2a8-e8c8-4a62-adcd-40a936780059	低 perplexity 命中典型
新闻标题、日志、论坛对话	大量	包含 已从网络删除 的内容

总计: 604 / 1800 = 33.6% 的候选被确认为字面记忆。模型越大，记忆越多: GPT-2 XL ≈ 10× GPT-2 Small。

7.5 为什么越大的模型记得越多?

直觉: 参数量提升 → 模型容量过剩 → 优化器倾向 无损存储 训练样本而非压缩。理论 (Feldman 2020, Does Learning Require Memorization?) 进一步证明: 在长尾分布下, 最优泛化必然要求一定程度的字面记忆。这预示 GPT-3 / GPT-4 / Llama 等更大模型在隐私上 天然更危险。

8. 防御全景图



[NOTE] 三类思路的本质差异

– 加密 让 计算过程 看不到明文 (机密性)。代价: 计算开销巨大。– DP 让 输出 不依赖于任何单条记录 (匿名性形式化)。代价: 精度损失。– FL 让 原始数据不离开本地 (data sovereignty)。但梯度本身可能仍泄漏 → 需要 + DP / + SMPC 强化。

9. 同态加密 (Homomorphic Encryption, HE)

核心承诺: 密文上能直接计算。

$$\text{Dec}(\text{Enc}(a) \otimes \text{Enc}(b)) = a \star b. \tag{10.4}$$

其中 $\otimes$ 是密文域运算, $\star$ 是明文域对应运算。

9.1 Toy Example —加法同态 (Paillier, 1999)

定义加密函数 $\text{Enc}(m) = g^m \cdot r^N \bmod N^2$, 则

$$\text{Enc}(a) \cdot \text{Enc}(b) = g^{a+b} \cdot (r_1 r_2)^N \bmod N^2 = \text{Enc}(a + b). \quad (10.5)$$

也就是说, 密文相乘 = 明文相加。服务器拿到一堆密文, 能直接加总, 但永远看不到明文。

9.2 分类

类型	支持的操作	例子	代价
PHE Partial HE	仅加法 或 仅乘法	Paillier (加) / RSA-mult (乘)	低
SHE / LHE Somewhat / Leveled HE	有限次加 + 乘	BFV, BGV, CKKS	中
FHE Fully HE	任意次加 + 乘 (任意电路)	Gentry 2009, TFHE	极高 (每次乘法 10x– 1000x 慢)

9.3 在 ML 中的应用

CryptoNets (Microsoft, ICML 2016): 把 CNN 推理改写成多项式形式 (用 x^2 替代 ReLU, average-pool 替代 max-pool), 在 BFV 密文上跑。代价: MNIST 推理 **单张图 250 秒**。CKKS 改进后 < 1 秒 / 张。

[WARN] HE 的工程瓶颈

(i) 密文膨胀: 1 个 32-bit 明文 → 数 KB 密文。(ii) 乘法深度有限: 每层乘法消耗“噪声预算”, 超过后必须 bootstrapping (极慢)。(iii) 非线性激活难: ReLU / softmax 需多项式近似 → 精度损失。

10. 安全多方计算 (Secure Multi-Party Computation, SMPC)

10.1 Yao's 百万富翁问题 (slide p.37)

[EX] 引子

两个百万富翁都想知道 谁更富有, 但都不想暴露 自己有多少钱。在没有可信第三方的情况下如何完成比较? Yao (1982) 给出第一个正解—这就是 SMPC 的起源问题。

SMPC 通用形式: n 方各持私有输入 x_i , 共同计算 $y = f(x_1, \dots, x_n)$, 要求除 y 本身外不泄漏任何 x_i 的额外信息。

10.2 两大技术构件 (1 段话各)

Garbled Circuits (姚氏混淆电路): 将 f 表达为布尔电路, 由 Alice 把每个门的真值表用随机 label 加密“打乱”, Bob 通过 oblivious transfer 解一对正确的输入 label, 沿电路一路评估, 最终只能得到输出 label → 解密为明文输出。安全性: Bob 看到的 label 都是随机串。

Oblivious Transfer (不经意传输, OT): Sender 持两条消息 m_0, m_1 , Receiver 持选择位 b 。协议结束 Receiver 拿到 m_b 但不知 m_{1-b} , Sender 不知 b 。OT 是 SMPC 的基础原语。

Secret Sharing (秘密分享): 另一条路线 (Shamir 1979)。把 x 拆成 n 个 share $x^{(1)}, \dots, x^{(n)}$, 任意 $t-1$ 个 share 不泄漏 x 。各方对 share 直接做加法/乘法, 重组后得到原运算结果。SPDZ、ABY3 等现代 ML 框架基于此。

10.3 SMPC vs HE 对比 (slide p.38)

维度	HE	SMPC
参与方数	1 计算方 + 多数据方	≥ 2 方对等
通信量	低 (仅传密文)	高 (多轮交互)
计算量	极高 (尤其 FHE)	中 (主要瓶颈是通信)
信任假设	计算方半诚实即可	通常半诚实多数 (honest majority) 或恶 意敌手模型
典型场景	云端推理	多方联合训练 / 联邦统计

11. 差分隐私 (Differential Privacy, DP)

11.1 婚恋数据库 (slide p.39) — 必背直觉

[EX] 例题—张三的婚姻状况

假设有一个婚恋数据库：8 已婚 + 2 单身，只允许查“有多少人单身”。– 第一次查询：返回 2。– 张三去登记了自己的婚姻状况。– 第二次查询：返回 3。→ 攻击者立刻推断：**张三是单身**。
DP 的目标：让两次查询的结果 几乎不可区分，即使张三加入或离开数据库，攻击者也无法靠输出差判断他是否在。**怎么做？**——加入随机噪声。如果两次查询不再是确定的 2 与 3，而是分别从 $\mathcal{N}(2, \sigma^2)$ 与 $\mathcal{N}(3, \sigma^2)$ 采样：– 张三不在 → 可能得 2.5；张三在 → 也可能得 2.5。攻击者无法把这个 2.5 自信地 归到任一数据集 → 张三的隐私被保护。

11.2 (ϵ, δ) -DP 形式化定义

定义 相邻数据集 (neighboring datasets) $\mathcal{D} \sim \mathcal{D}'$ 两者仅相差一条记录 (添加 / 删除 / 替换)。

[NOTE] 定义 10.1 — (ϵ, δ) -差分隐私 (Dwork et al. 2006; Dwork & Roth 2014 教材)

一个随机算法 $\mathcal{M} : \mathcal{X}^n \rightarrow \mathcal{Y}$ 满足 (ϵ, δ) -DP，当且仅当对 任意 相邻 $\mathcal{D}, \mathcal{D}'$ 和 任意 输出子集 $S \subseteq \mathcal{Y}$ ：

$$\Pr[\mathcal{M}(\mathcal{D}) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{M}(\mathcal{D}') \in S] + \delta \quad (10.6)$$

– ϵ (隐私预算)：越小越严，常用 $\epsilon \in [0.1, 10]$ 。– δ (失败概率)：通常 $\delta \ll 1/n$ (e.g. 10^{-5})。– $\delta = 0$ 称纯 DP； $\delta > 0$ 称近似 DP，更便于高斯机制。

几何直觉：对任一输出，“在 $\mathcal{D}$ 下的概率”与“在 $\mathcal{D}'$ 下的概率”之比限制在 e^ϵ 之内 → 攻击者从输出区分 $\mathcal{D}$ 与 $\mathcal{D}'$ 的能力被有界。

11.3 敏感度与基础机制

[NOTE] 定义 10.2 — ℓ_p 敏感度

$$\Delta_p f = \max_{\mathcal{D} \sim \mathcal{D}'} \|f(\mathcal{D}) - f(\mathcal{D}')\|_p. \quad (10.7)$$

计数查询 $\Delta_1 f = 1$ ，均值查询 $\Delta_1 f = (\max - \min)/n$ 。

(M1) Laplace 机制—实现纯 ϵ -DP：

$$\mathcal{M}_{\text{Lap}}(\mathcal{D}) = f(\mathcal{D}) + \text{Lap}(0, \Delta_1 f / \epsilon). \quad (10.8)$$

(M2) Gaussian 机制—实现 (ϵ, δ) -DP:

$$\mathcal{M}_{\text{Gauss}}(\mathcal{D}) = f(\mathcal{D}) + \mathcal{N}(0, \sigma^2 I), \quad \sigma \geq \frac{\Delta_2 f \sqrt{2 \ln(1.25/\delta)}}{\epsilon}. \quad (10.9)$$

11.4 组合性 (Composition)

[NOTE] 定理 10.3 —基本组合

若 $\mathcal{M}_1$ 是 (ϵ_1, δ_1) -DP, $\mathcal{M}_2$ 是 (ϵ_2, δ_2) -DP, 则它们的联合输出是 $(\epsilon_1 + \epsilon_2, \delta_1 + \delta_2)$ -DP。
进阶 (Advanced Composition, Dwork–Rothblum–Vadhan 2010): k 次 (ϵ, δ) 机制的复合为 $(\sqrt{2k \ln(1/\delta')} \epsilon + k\epsilon(e^\epsilon - 1), k\delta + \delta')$ -DP, 把线性增长降到 $O(\sqrt{k})$ 。

11.5 DP-SGD (Abadi et al., CCS 2016)

把 DP 注入到神经网络训练:

```
for t in 1..T:
    Sample batch  $L_t$  of size B (Poisson sampling, prob B/n)
    For each  $i$  in  $L_t$ :
         $g_i = \nabla_{\theta} \ell(\theta, x_i, y_i)$  # 逐样本梯度
         $g_i \leftarrow g_i \cdot \min(1, C / \|g_i\|_2)$  # 裁剪到  $\ell_2$  范数  $C$ 
     $\tilde{g}_t = (1/B) [\sum_i g_i + N(0, \sigma^2 C^2 I)]$  # 加高斯噪声后平均
     $\theta \leftarrow \theta - \eta \cdot \tilde{g}_t$ 
```

关键三件套: 1. **Per-sample gradient clipping**: 限制单样本贡献的 ℓ_2 敏感度上界为 C 。2. **Gaussian noise**: 加噪标准差 σC 。3. **Subsampled mechanism + Moments Accountant**: 用 Rényi DP / moments accountant 跟踪总隐私预算 (ϵ, δ) 的累积。

代价: 相同 ϵ 下, DP-SGD 训出的模型准确率比标准 SGD 低 1–10%; ϵ 越小, 损失越大。收益: 模型对训练集中任意单条记录 几乎不可分辨 $\rightarrow$ MIA 攻击效力大幅下降; 可证明 worst-case 保证。

[WARN] 隐私预算耗尽—不能反复使用

(ϵ, δ) 是 一次性 预算: 用过就用过了。多轮训练 / 多次查询要走 composition 累加。这点与口令、密钥根本不同。

12. 联邦学习 (Federated Learning, FL)

12.1 FedAvg (McMahan et al., AISTATS 2017)

```
# 在服务器上
initialize global model  $\theta^0$ 
for round  $t = 0, 1, \dots$ :
    sample K clients  $S_t \subset \{1, \dots, N\}$ 
    broadcast  $\theta^t$  to  $S_t$ 

    # 在每个客户端  $k$  上 (本地, 数据不出本地)
     $\theta_k^{t+1} \leftarrow \text{LocalSGD}(\theta^t, D_k, \text{epochs}=E, \text{lr}=\eta)$ 
    upload  $\Delta_k = \theta_k^{t+1} - \theta^t$  to server

    # 在服务器上聚合
     $\theta^{t+1} \leftarrow \theta^t + \sum_{k \in S_t} (n_k / n_S) \cdot \Delta_k$  # 按样本数加权平均
```

交换的是：模型参数更新 Δ_k (或本地权重 θ_k)。永不交换的是：原始数据 D_k 。

典型场景：Google Gboard 输入法 (用户输入数据敏感，但要训练下一词预测) → 在手机本地训练，仅上传模型增量。

12.2 FL 的隐私漏洞

[WARN] FL ≠ 隐私安全

仅仅“数据不离开本地”是不够的。梯度本身就是数据的可微变换，可被反推。– **Deep Leakage from Gradients** (Zhu et al., NeurIPS 2019): 仅用一步梯度，可重建出客户端的 (图像, 标签) 对。在小 batch 上几乎完美还原。– **Membership Inference on FL** (Nasr et al., S&P 2019): 恶意服务器或恶意客户端可对其他客户端的训练集做 MIA。

12.3 防御加固：FL + DP + SMPC

- **DP-FedAvg**: 客户端在上传前对 Δ_k 做裁剪 + 加噪 → 提供 client-level DP (单个客户端是否参与不可分辨)。
- **Secure Aggregation** (Bonawitz et al., CCS 2017): 用 SMPC + 秘密分享，服务器只能拿到 $\sum_k \Delta_k$ ，看不到任一 Δ_k 。
- **FL + HE**: 把上传的 Δ_k 加密，服务器在密文上做加法聚合。

[NOTE] 工程组合

工业级隐私 FL 系统通常是 FL + Secure Aggregation + DP-noise + 客户端 TEE (Trusted Execution Environment) 的全栈组合。

13. 攻击 vs. 防御 对应表

攻击	削弱它的最有效防御	原因
Membership Inference (MIA)	DP / DP-SGD	(10.6) 直接保证相邻数据集输出不可分辨 → MIA 上界与 ϵ 线性相关
Model Inversion / Feature Reconstruction	DP + 移除 softmax 全向量返回 (只返 top-1)	限制置信度信息 → 优化目标 (10.2) 不可微
Training Data Extraction (GPT-2 类)	训练时 DP-SGD + 数据去重 + 早停	去重消除 unintended memorization 的 enabling condition; DP-SGD 给可证明上界
Model Extraction	查询限速 + 输出扰动 + 水印	提高重建成本，提供可追责
FL 中梯度反演	Secure Aggregation + DP-Noise + 大 batch	大 batch 平均稀释单样本梯度信号
Netflix-IMDb 类 link attack	DP 发布合成数据	k -anonymity / l -diversity 已被证不充分

14. 期末复习要点

[TIP] 必背 5 条

1. 隐私三大组件: Confidentiality / Data Protection / Anonymity; 总原则 Do not store what you do not need.
2. MIA 形式化 (10.1) + Shokri 影子训练 4 步: 能写出威胁模型、影子模型作用、攻击模型训练数据如何构造。
3. Carlini GPT-2 攻击的 6 排序指标: perplexity、与小模型对比、zlib 熵、小写化 perplexity、滑窗 perplexity、Internet prompts; 并能说出每个为何能筛出 字面记忆。
4. (ϵ, δ) -DP 定义 (10.6) + Laplace/Gaussian 机制 (10.8)(10.9): 能写出敏感度, 能解释婚恋数据库直觉。
5. 四类防御的代价/适用: HE (计算贵) / SMPC (通信贵) / DP (精度损失) / FL (数据本地但仍漏梯度) — 实际系统常组合使用。

15. Anki Flashcards

[NOTE] 复习闪卡 (Q → A)

Anki 导入: 每张为 front / back, 自由调成 cloze 也可。

- Q1: GDPR、CCPA、《网络安全法》分别由哪个法域颁布、哪一年生效? A1: 欧盟 2018 / 加州 2018 / 中国 2017。
- Q2: 列 5 种 PII。A2: 姓名、身份证号、手机号、邮箱、家庭住址 (信用卡号、社保号也算)。
- Q3: 5 种隐私类别? A3: Personal Data / Location / Communication / Healthcare / Behavioral。
- Q4: 隐私的三大组件? A4: Confidentiality, Data Protection, Anonymity。
- Q5: Netflix-IMDb 攻击的核心结论? A5: 去标识 $\neq$ 隐私; 高维稀疏行为本身就是身份指纹, 只需 8 部电影评分即可唯一识别 99% 用户。
- Q6: MIA 的形式化目标? A6: 训练分类器 $g(f, x^*) \rightarrow \{\text{in}, \text{out}\}$ 判断 x^* 是否在 f 的训练集中。
- Q7: MIA 攻击为什么有效 (一句话原因)? A7: 模型对训练样本的 softmax 自信度系统性高于未见样本 (过拟合导致)。
- Q8: Shokri 影子训练为何要 per-class 训练攻击模型? A8: 不同类别的过拟合程度不同, per-class 攻击模型能更精准地刻画该类的 “in vs out” softmax 分布差异。
- Q9: Hill-climbing 合成影子样本时, “接受 / 拒绝” 一次扰动的判据是什么? A9: 扰动后目标类置信度提升则接受, 否则拒绝; 直到置信度超过阈值。
- Q10: Model inversion (Fredrikson 2015) 优化的目标函数? A10: $x^* = \arg \max_x f_\theta(x)[y] - \lambda \cdot \text{prior}(x)$, 即最大化目标类置信度 - 自然图像正则。
- Q11: Perplexity 定义? A11: $P = \exp(-\frac{1}{n} \sum_i \log f_\theta(x_i | x_{<i}))$; 越低 = 模型越自信 = 越可能是记忆。
- Q12: 为什么 “高 zlib 熵 + 低 perplexity” 是字面记忆的强信号? A12: 高 zlib = 字符串本身复杂 (非模板); 低 perplexity = 模型仍非常自信 $\rightarrow$ 必然是逐字记忆而非生成的合理产物。
- Q13: GPT-2 XL 与 Small 在记忆量上的比例? A13: 约 10:1, 模型越大记忆越多 (slide p.25)。
- Q14: Carlini 2021 最终人工核验 1800 候选样本, 多少被确认为字面记忆? A14: 604 (约 33%)。
- Q15: 同态加密的核心承诺, 一行公式? A15: $\text{Dec}(\text{Enc}(a) \otimes \text{Enc}(b)) = a \star b$ 。
- Q16: Paillier 加法同态如何把 “密文相乘” 映射为 “明文相加”? A16: $\text{Enc}(m) = g^m r^N \bmod N^2$, 则 $\text{Enc}(a)\text{Enc}(b) = g^{a+b} \cdot (r_1 r_2)^N = \text{Enc}(a+b)$ 。
- Q17: Yao 百万富翁问题陈述? A17: 两个百万富翁想知道谁更富, 但不愿透露各自资产, 无可信第三方下如何完成?
- Q18: (ϵ, δ) -DP 的形式化定义? A18: $\forall \mathcal{D} \sim \mathcal{D}', \forall S: \Pr[\mathcal{M}(\mathcal{D}) \in S] \leq e^\epsilon \Pr[\mathcal{M}(\mathcal{D}') \in S] + \delta$ 。
- Q19: Laplace 机制的噪声标度? A19: $\text{Lap}(0, \Delta_1 f / \epsilon)$ 。

Q20: Gaussian 机制 σ 的下界 (实现 (ϵ, δ) -DP)? **A20:** $\sigma \geq \Delta_2 f \sqrt{2 \ln(1.25/\delta)}/\epsilon$ 。

Q21: DP-SGD 的三件套? **A21:** (i) 逐样本梯度裁剪到 ℓ_2 范数 C ; (ii) 加高斯噪声 $\mathcal{N}(0, \sigma^2 C^2)$; (iii) 用 moments accountant / Rényi DP 跟踪累积隐私预算。

Q22: 婚恋数据库直觉一句话? **A22:** 加噪让”张三在与不在”两次查询返回相同分布, 攻击者分不清。

Q23: FedAvg 三步? **A23:** 广播全局权重 $\rightarrow$ 客户端本地 SGD 多步 $\rightarrow$ 服务器按样本数加权平均上传的权重增量。

Q24: FL 中只交换梯度就安全吗? **A24:** 不安全。Deep Leakage from Gradients (Zhu 2019) 显示仅一步梯度可逆向出原始图像。需要 + Secure Aggregation + DP-noise 加固。

Q25: HE / SMPC / DP / FL 各自的代价? **A25:** HE = 计算开销极大; SMPC = 通信开销大、需多轮交互; DP = 精度损失; FL = 仍漏梯度, 需要叠加上述。

16. 记号速查表

符号	含义
f_θ	目标模型 (参数 θ)
$\mathcal{D}, \mathcal{D}'$	相邻数据集 (差一条记录)
$\mathcal{M}$	随机机制 / 算法
x^*	攻击者关心的样本
$g(f, x^*)$	MIA 攻击模型
$\mathcal{D}_{\text{shadow}, i}$	第 i 个影子训练集
$f_{\text{shadow}, i}$	第 i 个影子模型
$P(x_1, \dots, x_n)$	Perplexity
$z(x)$	x 的 zlib 压缩字节数
$\Delta_p f$	函数 f 的 ℓ_p 敏感度
(ϵ, δ)	DP 隐私预算 / 失败概率
$\text{Lap}(0, b)$	0 均值 Laplace 噪声, scale b
$\mathcal{N}(0, \sigma^2)$	0 均值高斯噪声
C	DP-SGD 中的逐样本梯度裁剪范数
σ	DP-SGD 的噪声乘子
Δ_k	FedAvg 中客户端 k 的权重增量
n_k	客户端 k 的样本数
Enc/Dec	HE 加密 / 解密
$\otimes, \star$	密文 / 明文域对应运算